Ministerio de Salud Pública

BANCO MUNDIAL

Manual de la API de integración

Módulo de Indicadores de Presidencia

Tecnología: Python · FastAPI · R Shiny · Nginx

Construcción, instalación, configuración, publicación, consumo y pruebas.

Elaborado por: Marcelo Chávez

Consultor Especialista en Protección Social - Banco Mundial

marcelo_chavez_ec@outlook.com

0983332687

Versión documental: 22 de octubre de 2026

Tabla de contenidos

Antes de empezar **5**
0.1 Control documental y alcance de esta edición . . . 5

1 Instalación y configuración del ecosistema **6**
1.1 Conceptos de API y FastAPI . . . 6
1.2 Vocabulario de integración . . . 6
1.3 Dónde se ejecuta cada componente . . . 7
1.4 Dirección pública y direcciones internas . . . 7
1.5 Ubicación del código y preparación . . . 8
1.6 Directorio del proyecto . . . 8
1.7 Mapa de archivos . . . 8
1.8 Dependencias y función de cada componente . . . 8
1.9 Instalación Linux: paso 1, comprobar el entorno . . . 9
1.10 Paso 2: identificar el intérprete y preparar una reproducción . . . 9
1.11 Paso 3: preparar directorios de ejecución . . . 10
1.12 Paso 4: configurar credenciales y variables . . . 10
1.13 Variables: finalidad y tratamiento . . . 11
1.14 Paso 5: instalar y arrancar los servicios . . . 12
1.15 Paso 6: comprobar antes de publicar . . . 13
1.16 Comprobación de dependencias R del adaptador . . . 13
1.17 Preparación del sistema consumidor . . . 13

2 Construcción de FastAPI en Python **14**
2.1 Paso 1: definir los datos que se aceptan . . . 14
2.2 Paso 2: crear la aplicación . . . 14
2.3 Paso 3: validación de acceso a los endpoints . . . 16
2.4 Paso 4: declarar el POST de sesiones . . . 17
2.5 Paso 5: obtener datos REST . . . 18
2.6 Paso 6: devolver el visualizador y conservar interacción . . . 19
2.7 Paso 7: iniciar Python con Uvicorn . . . 19

3 Autenticación, sesiones y capas de seguridad **22**
3.1 Los dos niveles de autenticación . . . 22
3.2 Validación del archivo auth.py . . . 22
3.3 Diferenciar secretos y URL temporal . . . 23
3.4 Controles y dónde se aplican . . . 23

4 Datos y conexión con el dashboard R **25**
4.1 Fuentes institucionales de datos . . . 25
4.2 Exportación del corte autorizado . . . 25
4.3 Acceso directo mediante identidad firmada . . . 25

5 Publicación y multiconcurrencia **27**
5.1 Publicación del código en GitHub y despliegue de servicios . . . 27
5.2 Paso 1: comprender las rutas de Nginx . . . 27
5.3 Paso 2: comprender el proxy y WebSocket . . . 27
5.4 Paso 3: instalar las rutas sin reemplazar SIVAS . . . 28
5.5 Paso 4: verificar el acceso público . . . 28
5.6 Configuración de las unidades systemd . . . 29
5.7 Iniciar, detener, reiniciar y actualizar . . . 29
5.8 Consulta de registros operativos . . . 30
5.9 Gestión de sesiones y conexiones simultáneas . . . 30
5.10 CI y despliegue del repositorio . . . 31

Marcelo Chávez
Consultor Especialista en Protección Social - Banco Mundial
marcelo_chavez_ec@outlook.com 0983332687

6 Consumo de la URL **32**
6.1 Direcciones que debe conocer . . . 32
6.2 Configurar el YAML privado . . . 32
6.3 Abrir la comprobación . . . 32
6.4 Probar en Swagger . . . 33
6.5 Respuesta del POST de sesiones . . . 33
6.6 Solicitud HTTP de prueba con PowerShell . . . 33
6.7 Ejemplo HTML del iframe . . . 34

7 TEST_API.R explicado línea por línea **35**
7.1 El archivo completo y su función . . . 35
7.2 Solicitud HTTP utilizada por TEST_API.R . . . 36
7.3 Construcción de la URL y los encabezados HTTP . . . 37
7.4 Ejecución del POST desde el cliente R . . . 37
7.5 Renderizado del iframe en TEST_API.R . . . 38

8 Pruebas y resolución de problemas **39**
8.1 Secuencia de prueba recomendada . . . 39
8.2 Pruebas automatizadas disponibles . . . 39
8.3 Antecedentes históricos y revisión vigente . . . 39
8.4 Error: Failed to create server en Windows . . . 40
8.5 Aviso: Firefox no encontrado al abrir la app . . . 40
8.6 Error: Firefox no permite mostrar la página incrustada . . . 40
8.7 Otros síntomas y qué revisar . . . 40
8.8 Rollback de un despliegue . . . 41

9 Trazabilidad técnica y adaptación a otro módulo **42**
9.1 Qué se conectó realmente . . . 42
9.2 Archivo, responsabilidad y conexión siguiente . . . 43
9.3 Cómo pasa cada autenticación . . . 43
9.4 Puertos, URLs y Swagger . . . 44
9.5 Qué está instalado y qué versión se inspeccionó . . . 44
9.6 Evidencia vigente y alcance de las pruebas . . . 45
9.7 Investigación de esperas agotadas y fichas PDF . . . 45
9.8 Cómo reproducir la revisión sin revelar secretos . . . 46
9.9 Adaptación razonada a otro módulo . . . 46

10 Editar y regenerar este manual, solo en PDF **48**
10.1 Archivo principal del manual . . . 48
10.2 Inclusión y mantenimiento de las explicaciones de código . . . 48
10.3 Repetir las comprobaciones de esta revisión . . . 48
10.4 Recursos gráficos y referencias . . . 49
10.5 Renderizado a PDF . . . 49

11 Anexo de evidencia verificable **50**
11.1 Entornos y trazabilidad temporal . . . 50
11.2 Matriz funcional de los cinco indicadores . . . 50
11.3 Casos: procedimiento, esperado y observado . . . 51
11.4 Revisión Git y huellas de las fuentes . . . 56
11.5 Pendientes y conclusión delimitada . . . 57

12 Anexos de código Python completo **59**
12.1 Guía de api.py y correspondencia de líneas . . . 59
12.2 Archivo api.py: código y explicación de cada línea . . . 60
12.3 Archivo auth.py: código y explicación de cada línea . . . 73
12.4 servidor.py: lectura línea por línea . . . 79
12.5 Bootstrap y actualización del corte . . . 79
12.6 Adaptador Shiny completo . . . 80
12.7 CI existente . . . 82

12.8 Actualización de las notas por línea 82

Referencias **83**

Listado de Figuras

1.1 Arquitectura de integración con un sistema externo. 7

3.1 Solicitud de sesión y visualización mediante iframe. 22
3.2 Capas de autenticación y acceso al visualizador institucional. 24

5.1 Gestión de conexiones simultáneas en el servicio publicado. 30

9.1 Recorrido del corte REST y del visualizador institucional. 42

Antes de empezar

Este manual describe la construcción, configuración, publicación y consumo de la API de Indicadores de Presidencia del **Sistema de Visualización Analítica en Salud (SIVAS)**.

El manual reúne construcción, instalación, configuración, publicación, consumo, pruebas y diagnóstico. No es necesario buscar las explicaciones en varios documentos. Los bloques marcados como **código del proyecto** se leen directamente desde los archivos fuente durante el renderizado; los ejemplos con marcadores son didácticos y no contienen secretos.

Guía de lectura: si solo desea comprobar el dashboard, vaya al capítulo de consumo de la URL y a la explicación línea por línea de `TEST_API.R`. Si debe mantener el servicio, lea también construcción, instalación Linux y operación. No vuelva a instalar sobre el servidor 10.64.100.197 para aprender: el servicio ya fue publicado; los pasos de instalación describen cómo reproducirlo en un entorno autorizado.

Alcance: cinco indicadores de Presidencia. El dashboard sigue desarrollado en **R Shiny** y conserva su diseño. Python administra la API, la autenticación de integración, las sesiones y el acceso al visualizador. No se convirtieron las gráficas a Python y no se integraron otros módulos.

La documentación describe el código local revisado y las verificaciones efectuadas durante esta implementación. El capítulo de pruebas identifica el alcance de cada comprobación y sus condiciones de ejecución.

0.1. Control documental y alcance de esta edición

Dato	Valor
Fecha inicial / portada	22 de octubre de 2026.
Última revisión de contenido y evidencia	26 de octubre de 2026; horas de los casos en UTC y America/Guayaquil.
Versión documental	2.0, con revisión de contenido del 26/10/2026. No confundir con la versión 1.0.0 declarada por la API.
Código incluido	Fuentes locales leídas durante el render. La comparación con el 197 se registra por archivo y SHA256.
Operación realizada	Lecturas, sesiones de prueba, consulta y descarga; sin reinstalar, desplegar, sincronizar o reiniciar servicios del 197.

El anexo de evidencia forma parte de este PDF. Las afirmaciones históricas sin soporte primario completo se identifican como antecedentes; no se reconstruyen horas ni resultados. Las instrucciones de instalación describen cómo reproducir el entorno, no acciones ejecutadas nuevamente sobre producción.

1 Instalación y configuración del ecosistema

1.1. Conceptos de API y FastAPI

Una **API** es una interfaz para que un programa solicite una operación o datos a otro programa. En este proyecto, el backend de un sistema externo solicita una sesión autorizada a FastAPI. La API devuelve un JSON con la URL temporal que ese sistema incorpora en un iframe. El ejemplo `TEST_API.R` ilustra este contrato; no limita el lenguaje ni el sistema operativo del consumidor.

FastAPI es el framework de Python utilizado para declarar las rutas, validar entradas y respuestas y generar la documentación interactiva. Una ruta combina un método HTTP, como GET o POST, con una dirección. El decorador `@app.post(...)` vincula una ruta con una función Python. Esa relación está documentada en el tutorial oficial de FastAPI (FastAPI, s. f.-a).

Uvicorn es el programa que escucha conexiones y ejecuta la aplicación FastAPI. Instalar FastAPI no inicia un servicio por sí solo: también se necesita ejecutar Uvicorn y mantenerlo activo. La documentación oficial diferencia la aplicación y el servidor ASGI que la ejecuta (FastAPI, s. f.-b).

En esta implementación, la API nace de la función `create_app()` de `deploy/presidencia/api.py`. La instrucción `FastAPI(...)` crea la aplicación. El archivo `deploy/presidencia/linux/servidor.py` inicia Uvicorn y le indica que debe llamar a esa función.

1.2. Vocabulario de integración

Término	Significado en este proyecto
Backend	Código del sistema externo o del servicio API que se ejecuta fuera del navegador.
Endpoint	Ruta que atiende una operación de la API.
GET	Solicitud para leer un recurso o consultar datos.
POST	Solicitud para enviar un cuerpo y ejecutar una operación, como crear sesión.
JSON	Texto estructurado con nombres y valores que intercambian R y Python.
Header	Encabezado HTTP; aquí transporta credenciales de la API.
Bearer	Forma de presentar el token: `Authorization: Bearer <token>`.
SID	Identificador aleatorio de una sesión temporal de visualización.
Iframe	Elemento HTML que muestra otra página dentro de la aplicación consumidora.
WebSocket	Conexión persistente que mantiene la interacción del dashboard Shiny.
Proxy	Componente que recibe una conexión y la dirige a otro servicio.
Loopback	Dirección del mismo equipo: `127.0.0.1`.
systemd	Administrador de servicios Linux que inicia y supervisa los procesos.
Entorno Conda `msp_01`	Herramientas locales; no es el intérprete de systemd ni el utilizado para pytest en esta revisión.

1.3. Dónde se ejecuta cada componente

Un **sistema externo** consume la integración a través del dominio `analiticadesalud.msp.gob.ec`. El servicio FastAPI en Python y el visualizador institucional se ejecutan en el servidor Linux `10.64.100.197`; Nginx enruta sus solicitudes. El acceso público utiliza el HTTPS institucional existente. El sistema externo puede ejecutarse en cualquier plataforma con conectividad y autorización para ese dominio.

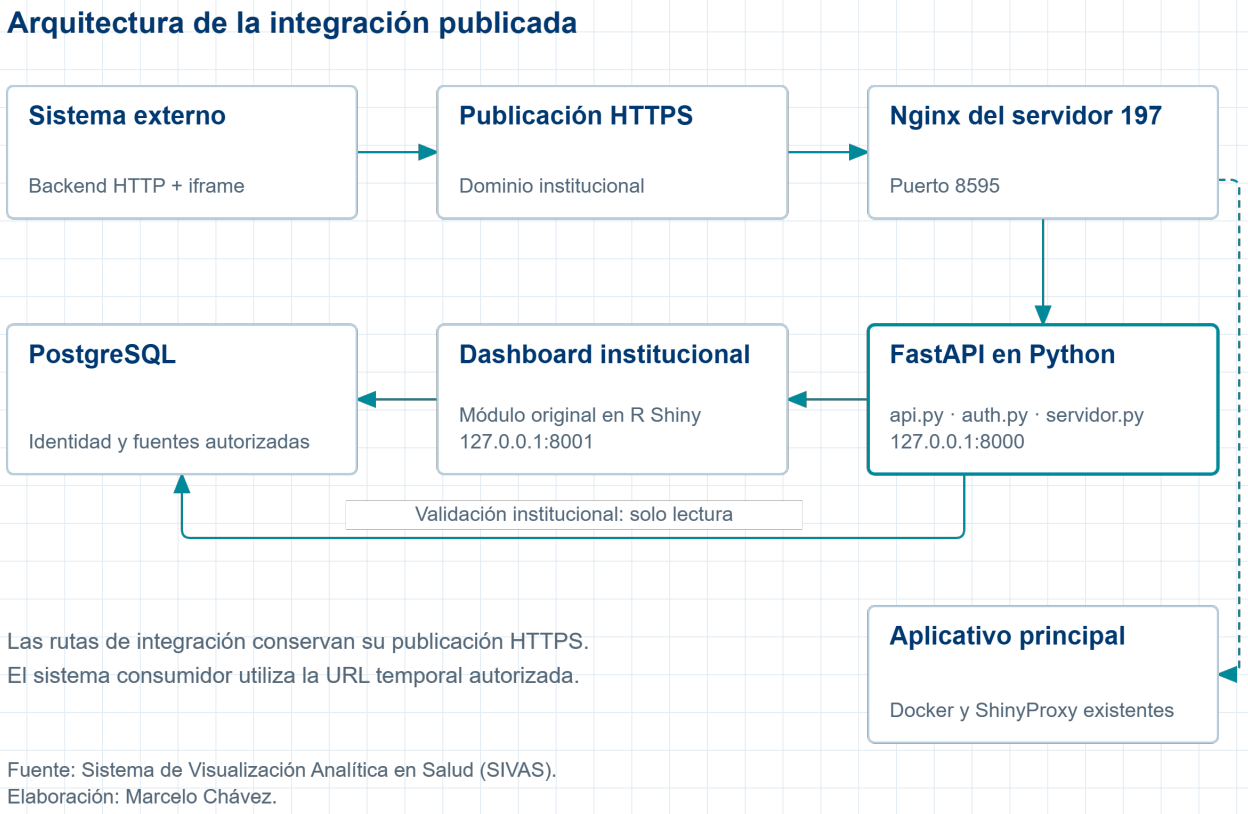


Figura 1.1: Arquitectura de integración con un sistema externo.

El camino de integración de la Figura 1.1 es:

1. El backend del sistema externo obtiene las credenciales desde su configuración privada.
2. El sistema externo envía una petición HTTPS al dominio institucional.
3. La publicación existente conduce al Nginx del servidor 10.64.100.197, que escucha en 8595.
4. Las rutas de integración se dirigen al FastAPI interno en `127.0.0.1:8000`.
5. FastAPI verifica las credenciales de integración, la cuenta institucional y la sesión; después comunica el acceso firmado al visualizador institucional interno en `127.0.0.1:8001`.
6. Los procesos autorizados consultan PostgreSQL mediante las conexiones institucionales.

El navegador del sistema externo se conecta al dominio por HTTPS. Los puertos 8000 y 8001 son internos del servidor y no son direcciones públicas del iframe. El consumidor tampoco necesita acceso directo a PostgreSQL.

1.4. Dirección pública y direcciones internas

El sistema externo utiliza la URL pública `https://analiticadesalud.msp.gob.ec`. La dirección `127.0.0.1:8000` identifica FastAPI solo dentro del servidor 10.64.100.197. La dirección local del ejemplo `TEST_API.R`, `http://127.0.0.1:5010/`, pertenece al equipo que ejecuta ese ejemplo.

El valor de `base_url` en el consumidor debe ser el dominio público. Una dirección localhost siempre se refiere al propio equipo que hace la solicitud; por ello no sustituye a la URL publicada del servidor.

1.5. Ubicación del código y preparación

1.6. Directorio del proyecto

En este equipo Windows, la raíz es:

```
C:/Users/MARCELO/OneDrive/Documentos/MSP/health_analytics_msp
```

En el servidor 10.64.100.197:

```
/home/marcelo.chavez/PROYECTO_BM/APPs/health_analytics_msp
```

Los comandos que indican **desde la raíz** deben ejecutarse después de situarse en esa carpeta. Un archivo no se encuentra solo por estar abierto en el editor: también importa el directorio de trabajo.

1.7. Mapa de archivos

```
health_analytics_msp/
  deploy/presidencia/
    api.py                        aplicación FastAPI y endpoints
    auth.py                       autenticación y sesiones
    requirements.txt              dependencias Python fijadas
    configurar.py                 preparación privada de desarrollo
    bootstrap.R                   conexiones y corte de datos agregados
    exportar.R                    ejecución del exportador
    bridge.R                      validación HMAC en R
    shiny_app.R                   adaptador al dashboard original
    linux/
      servidor.py                 inicia Uvicorn
      sivas-presidencia-api.service
      sivas-presidencia-shiny.service
      sivas-presidencia-exportar.service
      sivas-presidencia-exportar.timer
      nginx-proxy.conf
      nginx-rutas.conf
    examples/shiny_windows/cliente.R
    tests/test_api.py
  modulos/comprobacion_api/
    TEST_API.R                    botón e iframe
    config_credenciales_api.yml   privado; no versionado
  modulos/indicadores_presidencia/
    global.R                      fuentes, reglas y metas
    indicadores_presidencia_ui.R
    indicadores_presidencia_server.R
    indicadores_presidencia_assets.R
  .github/workflows/presidencia-integracion.yml
  docs/manuales/api_presidencia/
    Manual_integral_API_Presidencia.qmd
    bibliography.bib
    _quarto.yml
    incluir_codigo.lua
    latex/                        diseño y créditos
    recursos/                     logos y estilo APA
    diagramas/                    figuras y fuentes editables
```

El exportador de datos se implementa en `deploy/presidencia/exportar.R` y el arranque de Python en `deploy/presidencia/linux/servidor.py`. La carpeta `output/` contiene evidencias y auxiliares locales; no es un componente ejecutable de la API.

1.8. Dependencias y función de cada componente

Componente	Para qué sirve
FastAPI y Pydantic	Rutas, validación y documentación OpenAPI.
Uvicorn	Escuchar solicitudes y ejecutar FastAPI.
httpx y websockets	Proxy HTTP y comunicación interactiva con Shiny.
psycopg	Consulta de identidad en PostgreSQL desde Python.
argon2-cffi y PyNaCl	Verificación de contraseñas; no descifrado de hashes.
python-dotenv	Lectura de configuración privada en desarrollo.
pytest	Pruebas automatizadas de API y seguridad.
R, Shiny y paquetes del módulo	Dashboard original y exportación de agregados.
Nginx y systemd	Publicación y operación continua en Linux.

`requirements.txt` fija dependencias; un archivo de requisitos no demuestra por sí solo qué está instalado. En la revisión se consultaron el venv del servicio y los intérpretes locales: FastAPI 0.159.1 y Uvicorn 0.54.0 en ambos venv; Python 3.12.15 en el 197 y 3.12.14 en el venv Windows. Conda `msp_01` tiene Python 3.14.8; R es 4.6.0 en Windows y 4.5.1 en el 197. Las versiones de paquetes y la evidencia están en el anexo. Python 3.14 en CI es configuración del workflow, no prueba de ejecución de esa revisión. No es necesario instalar FastAPI localmente para usar solo el cliente R.

1.9. Instalación Linux: paso 1, comprobar el entorno

Tipo: comprobación de prerrequisitos y guía de reproducción; no reinstalación del 197. **Equipo:** Linux, cuenta autorizada del proyecto. **Directorio:** raíz del checkout. **Archivos:** `requirements.txt` y unidades de `deploy/presidencia/linux/`. **Propósito:** comprobar intérpretes y puertos antes de una instalación nueva. En otro servidor deben ajustarse usuario, rutas, HTTPS y conectividad. Los comandos siguientes son reproducibles; la evidencia lista cuáles se ejecutaron en esta revisión.

```
cd /home/marcelo.chavez/PROYECTO_BM/APPs/health_analytics_msp
python3.12 --version
Rscript --version
ss -ltn | grep -E ':8000|:8001' || true
```

Resultado esperado: Python y R disponibles. Antes de iniciar una instalación nueva, 8000 y 8001 deben estar libres. Si el sistema ya está publicado, los listeners en loopback son normales: se debe consultar systemd, no iniciar copias adicionales.

El visualizador reutiliza el módulo institucional. Los paquetes R deben estar disponibles en las bibliotecas del intérprete que utilizará el servicio; esta revisión no instaló ni actualizó ninguno. El siguiente paso distingue el venv realmente usado por systemd del entorno Conda local.

1.10. Paso 2: identificar el intérprete y preparar una reproducción

Tipo: guía para un entorno nuevo autorizado. **Equipo:** Linux de desarrollo o servidor de destino nuevo, no el 197 durante esta revisión. **Directorio:** raíz del checkout. **Archivo:** `deploy/presidencia/requirements.txt`. **Propósito:** instalar dependencias en el intérprete que después declarará `ExecStart`.

La instalación existente utiliza un **venv**, no Conda, para el proceso de FastAPI. La unidad apunta a `/home/marcelo.chavez/.sivas-presidencia/venv/bin/python`; se verificó Python 3.12.15 en esa ruta. En otro equipo, un ejemplo de reproducción es:

```
# SOLO instalación nueva; ajustar la raíz de la cuenta autorizada.
python3.12 -m venv /RUTA/PRIVADA/sivas-presidencia/venv
/RUTA/PRIVADA/sivas-presidencia/venv/bin/python -m pip install -r deploy/presidencia/requirements.txt
/RUTA/PRIVADA/sivas-presidencia/venv/bin/python -m pip check
/RUTA/PRIVADA/sivas-presidencia/venv/bin/python -c "import sys; print(sys.executable); print(sys.version)"
```

Esperado y comprobación: importación de FastAPI/Uvicorn correcta y dependencias compatibles; `pip check` debe finalizar sin incompatibilidades. Estos comandos de creación/instalación **no se ejecutaron sobre**

el 197 en esta revisión. El valor absoluto del intérprete debe coincidir con `ExecStart` en la unidad que se instale posteriormente.

Conda `msp_01` es una alternativa de laboratorio y un entorno local de herramientas (Conda, s. f.). No se debe presentar como el intérprete instalado de systemd. En Windows se observó `C:/ProgramData/anaconda3/envs/msp_01/python.exe`, Python 3.14.8, sin pytest; las pruebas usaron el venv existente de Presidencia, Python 3.12.14. No copiar entornos binarios de Windows a Linux.

```
# Lectura de un entorno Conda de laboratorio, si se elige esa alternativa.
conda run -n msp_01 python -c "import sys; print(sys.executable); print(sys.version)"
```

Si se escoge Conda en una futura instalación, debe adaptarse la ruta absoluta de `ExecStart` y repetirse la validación. El manual no afirma que ese cambio ya esté desplegado.

1.11. Paso 3: preparar directorios de ejecución

Tipo: instalación nueva, no mantenimiento de carpetas existentes. **Equipo:** Linux autorizado. **Directorio:** raíz del checkout. **Recursos:** runtime de la cuenta, `deploy/presidencia/runtime` y `/etc/sivas-presidencia`. **Propósito:** separar recursos y configuración privada. **Esperado:** directorios restringidos y rutas compatibles con las unidades; comprobar propietario/permisos mediante `ls -ld`, sin mostrar contenidos privados.

```
install -d -m 700 /home/marcelo.chavez/.sivas-presidencia/runtime
install -d -m 700 deploy/presidencia/runtime
sudo install -d -m 700 /etc/sivas-presidencia
```

La primera carpeta almacena el JSON agregado REST. La segunda es necesaria para los recursos de fichas del adaptador. La carpeta de `/etc` guarda configuración privada que systemd leerá como administrador. No se guardan esos secretos en los archivos de código.

1.12. Paso 4: configurar credenciales y variables

Instalación existente: conservar el archivo `/etc/sivas-presidencia/environment` y los secretos vigentes. El usuario institucional autorizado y su contraseña ya existen. No ejecutar `preparar_cuenta.R --generar` para desplegar: ese procedimiento puede cambiar una contraseña.

Equipo/directorio/archivo: Linux autorizado, raíz del checkout; archivo privado `/etc/sivas-presidencia/environment`. **Propósito:** aprovisionar identidad cliente, firma y conexión por un canal seguro. **Esperado:** configuración válida y privada, sin sobrescribir una existente.

Primera instalación autorizada: sustituya `/RUTA/DEL/ENTORNO/bin/python` por el intérprete elegido en el paso 2. El siguiente ejemplo genera secretos API nuevos en archivos privados. Se incluye para reproducir la solución; no se ejecutó al redactar este manual. Se niega a reemplazar una configuración existente. No imprime los secretos.

```
sudo /RUTA/DEL/ENTORNO/bin/python - <<'PY'
from pathlib import Path
import json, os, secrets
from argon2 import PasswordHasher
os.umask(0o077)
base = Path('/etc/sivas-presidencia')
env = base / 'environment'
cliente = base / 'credenciales_integracion.json'
if env.exists() or cliente.exists():
    raise SystemExit('Ya existe configuración; conservarla o planificar una rotación.')
clave = secrets.token_urlsafe(32)
token = secrets.token_urlsafe(48)
valores = {
    'PRESIDENCIA_API_KEY': token,
    'PRESIDENCIA_CLIENT_USER': 'integracion_presidencia',
    'PRESIDENCIA_CLIENT_HASH': PasswordHasher().hash(clave),
    'PRESIDENCIA_BRIDGE_KEY': secrets.token_urlsafe(48),
```

```
    'PRESIDENCIA_SIVAS_USER': '<USUARIO_INSTITUCIONAL_AUTORIZADO>',
    'PRESIDENCIA_ENVIRONMENT': 'producción',
    'PRESIDENCIA_VIEW_SECONDS': '28800',
    'PRESIDENCIA_SNAPSHOT': '/home/marcelo.chavez/.sivas-presidencia/runtime/presidencia.json',
    'PRESIDENCIA_SHINY_HOST': '127.0.0.1',
    'PRESIDENCIA_SHINY_PORT': '8001',
    'PRESIDENCIA_SHINY_INTERNAL_URL': 'http://127.0.0.1:8001',
    'PRESIDENCIA_SHINY_PUBLIC_URL': 'http://127.0.0.1:8001',
    'PRESIDENCIA_ALLOWED_HOSTS': 'analiticadesalud.msp.gob.ec,127.0.0.1,localhost',
    'PRESIDENCIA_FRAME_ANCESTORS': "'self' http://127.0.0.1:5010 http://localhost:5010 http://127.0.0.1:18760
↪  http://localhost:18760",
    'PRESIDENCIA_MAX_AGE_HOURS': '24',
    'PRESIDENCIA_DOCS_URL': '/presidencia-api/docs',
    'PRESIDENCIA_OPENAPI_URL': '/presidencia-api/openapi.json',
    'PRESIDENCIA_HEALTH_PUBLIC_PREFIX': '/presidencia-api',
    'PGHOST': '', 'PGPORT': '5432', 'PGUSER': '',
    'PGPASSWORD': '', 'PGSSLMODE': 'prefer'
}
env.write_text('\n'.join(k+'='+json.dumps(v, ensure_ascii=False)
                      for k,v in valores.items())+'\n', encoding='utf-8')
cliente.write_text(json.dumps({'usuario_api':'integracion_presidencia',
    'clave_api':clave,'token':token}, ensure_ascii=False, indent=2), encoding='utf-8')
print('Archivos privados preparados. Completar conexión PostgreSQL por canal seguro.')
PY
sudoedit /etc/sivas-presidencia/environment
sudo chmod 600 /etc/sivas-presidencia/environment
```

En el editor privado completar PGHOST, PGUSER y PGPASSWORD con la conexión autorizada. No pegar esas claves en comandos que queden en el historial. La cuenta institucional de la instalación debe existir y tener permiso para PRESIDENCIA; escribir su nombre en un archivo no crea ese permiso.

El archivo adicional de credenciales API generado sirve para aprovisionar al consumidor mediante un canal seguro. No contiene la contraseña institucional. La clave HMAC, el hash del cliente y la contraseña PostgreSQL no se entregan al usuario del iframe.

`EnvironmentFile` es configuración leída por systemd; no es un script Bash. Evitar cargarlo con `source`, especialmente porque los hashes contienen signos de dólar. En el checkout Linux publicado no se copió el `.env` de Windows: `bootstrap.R` lo leería si existiera y podría sustituir variables de producción.

1.13. Variables: finalidad y tratamiento

Los nombres siguientes proceden del código. Los secretos se muestran con marcadores; nunca copiar valores reales al manual o a Git. La existencia de una variable documentada no acredita que un valor de ejemplo esté instalado.

Variable o grupo	Función	Valor observado o tratamiento
`PRESIDENCIA_API_KEY`	Bearer del cliente de integración.	`<TOKEN_PRIVADO>`; no se imprime.
`PRESIDENCIA_CLIENT_USER`, `PRESIDENCIA_CLIENT_HASH`	Identidad del cliente y verificación de su clave.	Usuario configurado y hash privado; el consumidor suministra su clave, no el hash.
`PRESIDENCIA_SIVAS_USER`	Cuenta institucional específica permitida.	`<USUARIO_AUTORIZADO>`; necesita asignación vigente a PRESIDENCIA.
`PRESIDENCIA_BRIDGE_KEY`	HMAC interno API → adaptador.	`<CLAVE_INTERNA_PRIVADA>`; no se entrega al iframe.
`PRESIDENCIA_VIEW_SECONDS`	Vencimiento absoluto de SID.	28800 s observados; ocho horas configuradas, no duración real ensayada.
`PRESIDENCIA_SNAPSHOT`, `PRESIDENCIA_MAX_AGE_HOURS`	Corte REST y vigencia máxima admitida.	Runtime privado del servicio; 24 h observadas.
`PRESIDENCIA_SHINY_INTERNAL_URL`	Destino del proxy HTTP/WSS.	`http://127.0.0.1:8001` en el 197.

Variable o grupo	Función	Valor observado o tratamiento
`PRESIDENCIA_SHINY_PUBLIC_URL`	Nombre histórico usado como destino/fallback por el código.	No confundirlo con la URL pública del iframe; esta se devuelve en el POST bajo el dominio institucional.
`PRESIDENCIA_SHINY_HOST`, `PRESIDENCIA_SHINY_PORT`	Escucha del adaptador.	Socket observado en loopback, puerto 8001.
`PRESIDENCIA_ALLOWED_HOSTS`, `PRESIDENCIA_FRAME_ANCESTORS`	Hosts admitidos y orígenes que pueden embeber.	Configuración explícita; 5010 se probó. No usar comodines para simular aprobación de otros sistemas.
`PRESIDENCIA_DOCS_URL`, `PRESIDENCIA_OPENAPI_URL`	Rutas de Swagger y del contrato.	`/presidencia-api/docs` y `/presidencia-api/openapi.json`, verificadas.
`PRESIDENCIA_HEALTH_PUBLIC_PREFIX`	Prefijo de enlaces públicos de salud.	Rutas observadas bajo `/presidencia-api/health/`.
`PGHOST`, `PGPORT`, `PGSSLMODE`	Conectividad PostgreSQL del servidor.	`10.64.100.191:5432`; modo prefer observado. Eso no certifica por sí solo cifrado efectivo de la conexión PostgreSQL.
`PGUSER`, `PGPASSWORD`	Credenciales de consulta institucional.	`<USUARIO_BD>` / `<CLAVE_BD>`; privadas.
`SIVAS_DEV_CONFIG`, `R_LIBS_USER`	Alternativas de configuración/bibliotecas R.	Opcionales según el código; no reemplazar el environment del servicio con un archivo local.

La clave institucional del consumidor se envía en `clave_sivas` durante el POST; no se almacena en el entorno del iframe. El SID es una capacidad temporal y se debe eliminar de logs y capturas compartidas. También se depuraron cookies de mensajes de error de las herramientas.

1.14. Paso 5: instalar y arrancar los servicios

Tipo: guía de instalación nueva; los comandos que instalan, habilitan o arrancan servicios no se ejecutaron en el 197 durante esta revisión. **Archivos:** las cuatro unidades y el temporizador de `deploy/presidencia/linux/`. **Propósito:** supervisar API, adaptador y exportación con el intérprete configurado. **Esperado:** verificación de unidades correcta, exportación exitosa y procesos activos. La instalación existente se comprobó únicamente con consultas de estado y lectura de archivos.

Dónde: Linux, desde la raíz del proyecto. Revisar primero las rutas y usuario escritos en las unidades. La unidad systemd describe qué proceso se supervisa y bajo qué condiciones (The systemd Project, s. f.).

```
sudo install -m 644 deploy/presidencia/linux/sivas-presidencia-*.service /etc/systemd/system/
sudo install -m 644 deploy/presidencia/linux/sivas-presidencia-exportar.timer /etc/systemd/system/
sudo systemd-analyze verify /etc/systemd/system/sivas-presidencia-*.service
sudo systemctl daemon-reload
sudo systemctl start sivas-presidencia-exportar.service
sudo systemctl enable --now sivas-presidencia-shiny.service
sudo systemctl enable --now sivas-presidencia-api.service
sudo systemctl enable --now sivas-presidencia-exportar.timer
```

El orden tiene una finalidad: primero comprobar que se puede generar el corte, después levantar el dashboard, luego la API y finalmente activar el refresco horario. Si el exportador falla, resolver la conexión o los datos antes de declarar disponible la integración.

Las unidades usan una cuenta de servicio existente, `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=strict`, máscara 0077 y rutas de escritura limitadas. No se modificó SELinux en esta revisión ni se prescribe deshabilitarlo para reproducir la integración. Su estado actual no se utilizó como evidencia de funcionamiento.

1.15. Paso 6: comprobar antes de publicar

Tipo: comprobación de lectura. **Equipo:** Linux donde corran los servicios; `127.0.0.1` identifica ese equipo. **Directorio:** raíz del checkout. **Componentes:** rutas de salud y sockets de API/Shiny. **Propósito:** separar disponibilidad del proceso de autenticación e interactividad. Las rutas públicas verificadas desde Windows se registran en el anexo; no se presume que todo comando de esta guía fue ejecutado.

```
curl -f http://127.0.0.1:8000/health/live
curl -f http://127.0.0.1:8000/health/ready
curl -f http://127.0.0.1:8000/health/shiny
ss -ltn | grep -E ':8000|:8001'
sudo systemctl is-active sivas-presidencia-api sivas-presidencia-shiny
```

Resultado esperado: respuestas de salud correctas y puertos únicamente en loopback. Esto aún no prueba autenticación, gráficos, descarga o WebSocket. La publicación HTTPS y las pruebas de consumo se explican más adelante.

1.16. Comprobación de dependencias R del adaptador

Tipo: preflight de lectura para una reproducción. **Equipo/directorio:** Linux de destino autorizado, raíz del checkout. **Archivos:** `bootstrap.R`, `shiny_app.R` y UI/server/global/assets del módulo. **Propósito:** comprobar las bibliotecas que realmente cargará R. **Esperado:** lista vacía de faltantes; si no, aprovisionar el entorno de laboratorio mediante el procedimiento institucional, no instalar automáticamente en el 197.

```
paquetes <- c("shiny", "dplyr", "highcharter", "glue", "gt", "shinyjs",
              "DBI", "RPostgres", "config", "jsonlite", "digest", "openxlsx")
faltantes <- paquetes[!vapply(paquetes, requireNamespace, logical(1), quietly = TRUE)]
print(faltantes)
```

Para ejecutar el adaptador en un laboratorio, configurar allí sus variables privadas y usar `Rscript deploy/presidencia/shiny_app.R`; debe escuchar en el host/puerto configurado. En el 197 no se lanzó otra copia: se inspeccionó el servicio existente. La lectura de `/health/shiny` acredita respuesta HTTP; la prueba de iframe/WSS del anexo acredita el recorrido interactivo observado.

1.17. Preparación del sistema consumidor

El sistema consumidor necesita un cliente HTTP en su backend y un componente capaz de mostrar un iframe. Puede estar escrito en R, Python, Java, .NET u otra tecnología. El ejemplo `TEST_API.R` usa una instalación R ya disponible; su finalidad es demostrar el contrato HTTPS y la inserción del iframe, no instalar un ecosistema nuevo.

2 Construcción de FastAPI en Python

2.1. Paso 1: definir los datos que se aceptan

Archivo: `deploy/presidencia/api.py`. Este es el modelo real del cuerpo del POST:

```
80 class SolicitudSesion(BaseModel):
81     model_config = ConfigDict(extra='forbid')
82     usuario_sivas: str = Field(min_length=1, max_length=80)
83     clave_sivas: SecretStr = Field(min_length=1, max_length=256)
84     indicador: IndicatorId = 'primer-nivel'
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
80	Declara `SolicitudSesion`: modelo de datos validado de FastAPI/Pydantic.
81	En `módulo`, rechaza campos ajenos al contrato; cuando corresponde también impide NaN e infinito.
82	Define `usuario_sivas`: nombre institucional obligatorio, de uno a ochenta caracteres.
83	Define `clave_sivas`: contraseña institucional tratada como SecretStr, de uno a 256 caracteres.
84	Define `indicador`: indicador inicial autorizado; por defecto primer-nivel.

`BaseModel` establece el contrato. `extra='forbid'` rechaza campos no previstos. `usuario_sivas` y `clave_sivas` son obligatorios y tienen longitudes máximas. `SecretStr` evita que la contraseña aparezca normalmente como texto claro al representar el modelo, aunque la verificación necesita extraerla dentro del backend. `indicador` tiene un valor inicial y solo admite los cinco identificadores autorizados.

No se debe enviar el JSON completo de un archivo de credenciales si también contiene campos como fecha o procedencia. Enviar únicamente los campos del contrato.

2.2. Paso 2: crear la aplicación

Archivo: `api.py`, función `create_app`. Este fragmento real crea FastAPI y declara sus tres entradas de autenticación API:

```
87  def create_app(*, key: str | None = None, snapshot: Path | None = None,
88                 shiny_url: str | None = None, max_age: float | None = None,
89                 gateway: Gateway | None = None) -> FastAPI:
90      api_key = key if key is not None else os.getenv('PRESIDENCIA_API_KEY', '')
91      if len(api_key) < 32:
92          raise RuntimeError('Configure PRESIDENCIA_API_KEY con al menos 32 caracteres aleatorios.')
93      gate = gateway or Gateway(os.getenv('PRESIDENCIA_CLIENT_USER',''),
94          os.getenv('PRESIDENCIA_CLIENT_HASH',''), os.getenv('PRESIDENCIA_BRIDGE_KEY',''),
95          PostgresIdentity(os.getenv('PRESIDENCIA_SIVAS_USER','')),
96          username=os.getenv('PRESIDENCIA_SIVAS_USER',''),
97          seconds=int(os.getenv('PRESIDENCIA_VIEW_SECONDS','28800')))
98      source = snapshot or Path(os.getenv('PRESIDENCIA_SNAPSHOT', str(BASE / 'runtime/presidencia.json')))
99      shiny = (shiny_url or os.getenv('PRESIDENCIA_SHINY_PUBLIC_URL', 'http://127.0.0.1:8001')).rstrip('/')
100     parsed = urlsplit(shiny)
101     if parsed.scheme not in ('http', 'https') or not parsed.hostname or parsed.username or parsed.password or
        ↪ parsed.query or parsed.fragment:
102         raise RuntimeError('La URL pública Shiny debe ser HTTP/HTTPS sin credenciales, query ni fragmento.')
103     ttl = max_age if max_age is not None else float(os.getenv('PRESIDENCIA_MAX_AGE_HOURS', '24'))
104     if ttl <= 0:
105         raise RuntimeError('La vigencia del corte debe ser positiva.')
106     ancestors = os.getenv('PRESIDENCIA_FRAME_ANCESTORS', "'self' http://127.0.0.1:8000 http://localhost:8000")
107     if any(c in ancestors for c in (';', '\n', '\r', '*')):
108         raise RuntimeError('Configure orígenes iframe explícitos, sin comodines.')
109     environment = os.getenv('PRESIDENCIA_ENVIRONMENT', 'desarrollo')
110     app = FastAPI(title=f'SIVAS - Integración de Presidencia ({environment})', version='1.0.0',
```

```
                description='Solo Presidencia. Cliente API y token permanente revocable; sesión iframe tras validar la cuenta institucional.',
                docs_url=os.getenv('PRESIDENCIA_DOCS_URL', '/docs'),
                openapi_url=os.getenv('PRESIDENCIA_OPENAPI_URL', '/openapi.json'), redoc_url=None)
    app.add_middleware(TrustedHostMiddleware, allowed_hosts=os.getenv(
        'PRESIDENCIA_ALLOWED_HOSTS', '127.0.0.1,localhost,testserver').split(','))
    bearer = HTTPBearer(auto_error=False)
    client_user = APIKeyHeader(name='X-API-Usuario',scheme_name='UsuarioAPI',auto_error=False)
    client_password = APIKeyHeader(name='X-API-Clave',scheme_name='ClaveAPI',auto_error=False)
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
87	Define `create_app`, que construye la aplicación FastAPI, su configuración, el gateway y sus rutas.
88	Completa los parámetros de `create_app`: shiny_url: str | None = None, max_age: float | None = None,.
89	Completa los parámetros de `create_app`: gateway: Gateway | None = None) -> FastAPI:.
90	En `create_app`, obtiene el Bearer del argumento de pruebas o de PRESIDENCIA_API_KEY.
91	Exige un Bearer de al menos 32 caracteres antes de crear la aplicación.
92	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
93	Usa el gateway recibido en pruebas o crea uno con el usuario del cliente API configurado.
94	Entrega al gateway el hash Argon2 de la clave API y la clave HMAC del puente; no son la contraseña institucional.
95	Construye PostgresIdentity para consultar únicamente la cuenta institucional autorizada.
96	Fija en el gateway el nombre institucional que podrá generar sesiones de Presidencia.
97	Lee PRESIDENCIA_VIEW_SECONDS como entero; el valor por defecto es 28.800 segundos.
98	En `create_app`, resuelve la ubicación del corte JSON de indicadores desde la configuración.
99	En `create_app`, normaliza la dirección del visualizador quitando la barra final.
100	En `create_app`, separa esquema, host y otras partes de la URL para validarlas.
101	Rechaza una dirección del visualizador sin host o protocolo HTTP/HTTPS, o que incluya credenciales, consulta o fragmento.
102	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
103	En `create_app`, obtiene la antigüedad máxima permitida al corte, expresada en horas.
104	Comprueba que la antigüedad máxima del corte sea mayor que cero.
105	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
106	En `create_app`, lee la lista explícita de orígenes que pueden embeber el visualizador.
107	Impide comodines, saltos de línea y punto y coma en la configuración de orígenes iframe.
108	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
109	En `create_app`, lee el nombre del ambiente para identificarlo en FastAPI y OpenAPI.
110	Crea FastAPI con título identificable del ambiente y versión 1.0.0.
111	Describe en OpenAPI el alcance del módulo y la diferencia entre credenciales API y sesión temporal.
112	Lee la ruta de Swagger desde PRESIDENCIA_DOCS_URL; su valor inicial es /docs.
113	Configura la URL de OpenAPI y desactiva la interfaz ReDoc.
114	Instala el control de host de FastAPI para rechazar dominios no autorizados.
115	Convierte la cadena de hosts permitidos en una lista separando por comas.
116	En `create_app`, prepara el lector del encabezado Authorization sin emitir un rechazo automático.
117	En `create_app`, prepara el lector del encabezado X-API-Usuario.
118	En `create_app`, prepara el lector del encabezado X-API-Clave.

La variable `app` es la aplicación. Las variables de entorno determinan el nombre del ambiente y las rutas de Swagger/OpenAPI. `TrustedHostMiddleware` restringe los nombres de host aceptados. `HTTPBearer` lee Authorization; las dos instancias `APIKeyHeader` leen usuario y clave API.

La función es una **fábrica**: primero prepara la aplicación y finalmente la devuelve. Por eso Uvicorn se inicia con `factory=True` o `--factory`. No basta con importar el archivo y esperar que aparezca una página pública.

2.3. Paso 3: validación de acceso a los endpoints

Archivo: `api.py`, función `authorize`. El decorador del POST incluye `Depends(authorize)`: FastAPI ejecuta esa comprobación antes de entregar el control a la función que crea la sesión.

```
120     def authorize(request: Request, credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(bearer)],
121                   username: Annotated[str | None, Depends(client_user)], password: Annotated[str | None,
    ↪ Depends(client_password)]):
122         try:
123             gate.throttle(request.client.host if request.client else 'unknown')
124             if credentials is None or credentials.scheme.lower() != 'bearer' or not
                ↪ secrets.compare_digest(credentials.credentials.encode(), api_key.encode()):
125                 raise AccessDenied()
126             gate.verify_client(username,password)
127             gate.principal()
128         except AccessDenied:
129             try:
130                 gate.throttle(request.client.host if request.client else 'unknown',fail=True)
131             except AccessDenied:
132                 pass
133            raise HTTPException(401, 'Credencial de integración inválida.', headers={'WWW-Authenticate': 'Bearer'})
134         except IdentityUnavailable:
135             raise HTTPException(503,'No se pudo comprobar la autorización institucional.') from None
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
120	Declara authorize como dependencia de acceso: recibe la petición y el Bearer que FastAPI extrajo de Authorization.
121	Recibe X-API-Usuario y X-API-Clave mediante Depends; ninguno se obtiene del JavaScript del iframe.
122	Agrupa operaciones de `authorize` cuyos errores se traducen a una respuesta controlada.
123	Consulta o incrementa el contador de fallos del origen, antes de aceptar más intentos.
124	Rechaza un Bearer ausente, un esquema distinto de Bearer o un token diferente al configurado; compara el token en tiempo constante.
125	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
126	Verifica X-API-Usuario y X-API-Clave contra el cliente y su hash configurados.
127	Comprueba que la cuenta institucional configurada siga activa, vigente y autorizada para PRESIDENCIA.
128	Captura `AccessDenied` en `authorize` para responder sin mostrar detalles internos ni credenciales.
129	Agrupa operaciones de `authorize` cuyos errores se traducen a una respuesta controlada.
130	Registra el intento fallido asociado al origen para limitar repeticiones dentro de la ventana configurada.
131	Captura `AccessDenied` en `authorize` para responder sin mostrar detalles internos ni credenciales.
132	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
133	Devuelve 401 y el encabezado WWW-Authenticate: Bearer, sin especificar qué secreto falló.
134	Captura `IdentityUnavailable` en `authorize` para responder sin mostrar detalles internos ni credenciales.
135	Devuelve 503 cuando la identidad institucional no puede comprobarse; no concede acceso por omisión.

Primero se revisa el límite de intentos; después se compara el Bearer y se verifica el cliente. También se comprueba que la cuenta institucional configurada conserve autorización. Un error de acceso devuelve 401; una indisponibilidad de la fuente de identidad devuelve 503. El limitador actual cuenta fallos por origen durante un minuto; no es una prueba de capacidad o un sistema completo de protección perimetral.

2.4. Paso 4: declarar el POST de sesiones

Ubicación exacta: `deploy/presidencia/api.py`, línea 201 declara el prefijo; línea 203 registra `POST /api/v1/presidencia/sesiones`; desde la línea 204, `open_session()` recibe el cuerpo validado. Cualquier backend externo puede llamar este endpoint. `TEST_API.R` es uno de esos consumidores:

```
201  prefix = '/api/v1/presidencia'
202
203  @app.post(prefix + '/sesiones', dependencies=[Depends(authorize)],tags=['Presidencia'])
204  def open_session(body: SolicitudSesion, request: Request):
205      try:
206          sid = gate.issue(body.usuario_sivas,body.clave_sivas.get_secret_value(),body.indicador)
207      except AccessDenied:
208          try:
209              gate.throttle(request.client.host if request.client else 'unknown',fail=True)
210          except AccessDenied:
211              pass
212          raise HTTPException(401,'Cuenta institucional no autorizada.') from None
213      except IdentityUnavailable:
214          raise HTTPException(503,'No se pudo comprobar la autorización institucional.') from None
215      base = str(request.base_url).rstrip('/') + '/vista/' + sid
216      return {'url':base+'/embed/presidencia','demo':base+'/demo',
217              'vigencia_segundos':gate.seconds,'indicadores':[
218                  {'id':i,'url':base+'/embed/presidencia/'+i} for i in IDS]}
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
201	En `create_app`, define /api/v1/presidencia, base de los endpoints REST.
203	Registra POST /api/v1/presidencia/sesiones y exige que authorize termine correctamente antes de ejecutar open_session.
204	Recibe el cuerpo SolicitudSesion ya validado y Request para conocer la dirección pública de la solicitud.
205	Agrupa operaciones de `open_session` cuyos errores se traducen a una respuesta controlada.
206	Extrae la contraseña de SecretStr solo dentro del backend y llama a Gateway.issue con usuario, contraseña e indicador inicial.
207	Captura `AccessDenied` en `open_session` para responder sin mostrar detalles internos ni credenciales.
208	Agrupa operaciones de `open_session` cuyos errores se traducen a una respuesta controlada.
209	Consulta o incrementa el contador de fallos del origen, antes de aceptar más intentos.
210	Captura `AccessDenied` en `open_session` para responder sin mostrar detalles internos ni credenciales.
211	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
212	Interrumpe esta ruta con HTTP 401: rechaza la autenticación o sesión inválida. No devuelve el secreto recibido.
213	Captura `IdentityUnavailable` en `open_session` para responder sin mostrar detalles internos ni credenciales.
214	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
215	Construye la base pública /vista/ usando esquema y host de Request; Nginx debe aportar los encabezados correctos.
216	Devuelve url para el módulo y demo para la página de prueba; ambas incorporan el SID temporal.
217	Incluye vigencia_segundos y empieza la lista de enlaces individuales de los cinco indicadores.
218	Asocia cada ID autorizado con su URL /embed/presidencia/; no devuelve contraseñas ni hashes.

1. `prefix` aporta `/api/v1/presidencia`.
2. `@app.post` agrega `/sesiones` y vincula esa URL con `open_session`.
3. `body` contiene el JSON validado; `request` contiene información de la petición.
4. `gate.issue(...)` valida usuario y contraseña institucionales y crea el SID.
5. `request.base_url` se utiliza para construir la dirección HTTPS pública. Nginx debe comunicar correctamente el esquema y host.

6. El `return` entrega un diccionario; FastAPI lo convierte en una respuesta JSON. No devuelve el dashboard como una imagen.

2.5. Paso 5: obtener datos REST

`catalog()` devuelve el catálogo de cinco indicadores y los nombres de sus rutas. La dirección de dashboard que aparece en ese catálogo incluye un marcador de sesión: para visualizar se debe crear una sesión real mediante el POST.

`series()` devuelve una serie agregada y admite `anio`, `mes_desde` y `mes_hasta`. En la implementación documentada el año admitido es **2026**. Un rango mensual invertido se rechaza con 422. No asumir que cambiar el año en la URL habilita automáticamente datos de 2027.

Ubicación exacta: `deploy/presidencia/api.py`, línea 290 registra `GET /api/v1/presidencia` y línea 296 registra `GET /api/v1/presidencia/{indicator_id}/serie`. Ambas rutas ejecutan `authorize` y leen el corte mediante `read_snapshot()`.

```
290     @app.get(prefix, response_model=Catalogo, dependencies=[Depends(authorize)], tags=['Presidencia'])
291     def catalog():
292         cut = read_snapshot()
293         return {'modulo': cut.modulo, 'generado_en': cut.generado_en,
294                 'indicadores': [dict(id=i.id, titulo=i.titulo,
                    ↪ dashboard=f'/vista/{{sesion}}/embed/presidencia/{i.id}', datos=f'{prefix}/{i.id}/serie') for
                    ↪ i in cut.indicadores]}
295
296     @app.get(prefix + '/{indicator_id}/serie', response_model=Serie, dependencies=[Depends(authorize)],
        ↪ tags=['Presidencia'])
297     def series(indicator_id: IndicatorId, anio: Annotated[int, Query(ge=2026, le=2026)] = 2026,
298                mes_desde: Annotated[int, Query(ge=1, le=12)] = 1,
299                mes_hasta: Annotated[int, Query(ge=1, le=12)] = 12):
300         if mes_desde > mes_hasta:
301             raise HTTPException(422, 'mes_desde debe ser menor o igual a mes_hasta.')
302         cut = read_snapshot()
303         indicator = next(i for i in cut.indicadores if i.id == indicator_id)
304         data = indicator.model_dump(mode='json')
305         data['datos'] = [p.model_dump(mode='json') for p in indicator.datos if p.fecha.year == anio and mes_desde
    ↪ <= p.fecha.month <= mes_hasta]
306         return {'generado_en': cut.generado_en, 'origen': cut.origen, **data}
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
290	Registra GET /api/v1/presidencia, aplica authorize y valida su salida con el modelo Catalogo.
291	Define `catalog`, que entrega el catálogo REST protegido de los cinco indicadores.
292	En `catalog`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
293	Devuelve módulo, fecha del corte y enlaces del catálogo REST.
294	Construye un enlace REST y una plantilla de URL de dashboard para cada indicador; la plantilla exige reemplazar la sesión por un SID real.
296	Registra GET /api/v1/presidencia/{indicator_id}/serie; exige authorize y valida la respuesta como Serie.
297	Limita indicator_id a los cinco IDs y anio a 2026 mediante Query; no habilita otros años por cambiar la URL.
298	Declara mes_desde entre uno y doce; su valor inicial es uno.
299	Declara mes_hasta entre uno y doce; su valor inicial es doce.
300	Comprueba que el primer mes del rango no sea posterior al último.
301	Interrumpe esta ruta con HTTP 422: rechaza parámetros fuera de contrato. No devuelve el secreto recibido.
302	En `series`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
303	En `series`, localiza el indicador cuyo ID coincide con el solicitado.
304	En `series`, convierte el indicador validado a tipos JSON y conserva únicamente los puntos del rango pedido.

Línea	Explicación
305	Conserva únicamente puntos cuyo año y mes pertenecen al rango autorizado de la petición.
306	Devuelve los puntos del indicador dentro del año y meses solicitados, con origen y fecha del corte.

Los GET protegidos requieren los tres encabezados API y la cuenta configurada debe continuar autorizada. La contraseña institucional se verifica al crear la sesión de visualización; no se envía como parámetro GET.

2.6. Paso 6: devolver el visualizador y conservar interacción

La ruta `/vista/<sid>/embed/presidencia` valida la capacidad y devuelve una página HTML. Esa página contiene un iframe interno con una ruta `/vista/<sid>/shiny/`. FastAPI actúa como proxy hacia el Shiny de 8001.

```
308     @app.get('/vista/{sid}/embed/presidencia', response_class=HTMLResponse, include_in_schema=False)
309     def module(sid: str):
310         session = view(sid)
311         return embed(sid,session['indicator'])
312 
313     @app.get('/vista/{sid}/embed/presidencia/{indicator_id}', response_class=HTMLResponse,
        ↪  include_in_schema=False)
314     def embed(sid: str, indicator_id: IndicatorId):
315         view(sid)
316         return page(f'<iframe title="{html.escape(LABELS[indicator_id])}"
            ↪  src="/vista/{sid}/shiny/?indicador={indicator_id}"></iframe>')
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
308	Aplica el decorador `app.get('/vista/{sid}/embed/presidencia', response_class=HTMLResponse, include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
309	Define `module`, que abre el indicador inicial de una sesión autorizada.
310	En `module`, consulta la sesión actual del SID o recibe una copia de sus datos ya validados.
311	Invoca la vista del indicador inicial guardado en la sesión.
313	Aplica el decorador `app.get('/vista/{sid}/embed/presidencia/{indicator_id}', response_class=HTMLResponse, include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
314	Define `embed`, que construye el iframe interno de un indicador después de validar el SID.
315	Ejecuta `view(sid)` dentro de `embed` para construye el iframe interno de un indicador después de validar el SID.
316	Construye el iframe interno con indicador y SID; escapa el título para que su texto no se interprete como HTML.

`shiny_http()` reenvía HTML, recursos y solicitudes HTTP autorizadas. `shiny_websocket()` mantiene los mensajes de interacción y comprueba la sesión cada diez segundos. Sin este segundo canal el dashboard puede mostrar una portada y luego quedar desconectado o sin responder.

El contenido sigue calculándose y renderizándose en Shiny. El proxy no reemplaza el server R ni sus consultas por una captura estática.

2.7. Paso 7: iniciar Python con Uvicorn

Archivo completo: `deploy/presidencia/linux/servidor.py`.

```python
"""Servidor de un trabajador; oculta capacidades temporales en los logs."""
import logging
import re

import uvicorn


class OcultarSesion(logging.Filter):
    def filter(self, record):
        record.msg = re.sub(r'/vista/[A-Za-z0-9_-]{43}(?=/)',
                            '/vista/[SESION]', record.getMessage())
        record.args = ()
        return True


if __name__ == '__main__':
    config = uvicorn.config.LOGGING_CONFIG
    config['filters'] = {'sesiones': {'()': OcultarSesion}}
    for handler in config['handlers'].values():
        handler['filters'] = ['sesiones']
    uvicorn.run('deploy.presidencia.api:create_app', factory=True,
                host='127.0.0.1', port=8000, workers=1,
                access_log=False, log_config=config,
                proxy_headers=True, forwarded_allow_ips='127.0.0.1')
```

Lectura línea por línea: deploy/presidencia/linux/servidor.py

Línea	Explicación
1	Texto descriptivo del componente; no ejecuta una operación de red ni abre un servicio.
2	Importa logging para filtrado de mensajes antes de escribir registros.
3	Importa re para reconocimiento de rutas que contienen un SID.
5	Importa uvicorn para servidor ASGI que ejecuta la aplicación FastAPI.
8	Declara `OcultarSesion`: filtro de logging que oculta SID.
9	Define `filter`, que oculta el SID de las rutas antes de escribir un mensaje de registro.
10	Actualiza el registro de log para ocultar el SID y evitar un segundo formateo con argumentos originales.
11	Continuación de la instrucción de la línea 10: Actualiza el registro de log para ocultar el SID y evitar un segundo formateo con argumentos originales.
12	Actualiza el registro de log para ocultar el SID y evitar un segundo formateo con argumentos originales.
13	Permite procesar el registro después de ocultar el SID; no cambia la sesión real del navegador.
16	Ejecuta el arranque de Uvicorn únicamente cuando servidor.py se invoca como programa principal.
17	En `módulo`, toma la configuración de registro de Uvicorn para instalar el filtro de SID.
18	Registra o aplica el filtro OcultarSesion en los manejadores de salida de Uvicorn.
19	Recorre `config['handlers'].values()` asignando cada elemento a `handler` dentro de `módulo`.
20	Registra o aplica el filtro OcultarSesion en los manejadores de salida de Uvicorn.
21	Inicia Uvicorn con la fábrica create_app y las opciones de puerto, trabajador, proxy y registro del bloque.
22	Continuación de la instrucción de la línea 21: Inicia Uvicorn con la fábrica create_app y las opciones de puerto, trabajador, proxy y registro del bloque.
23	Continuación de la instrucción de la línea 21: Inicia Uvicorn con la fábrica create_app y las opciones de puerto, trabajador, proxy y registro del bloque.
24	Continuación de la instrucción de la línea 21: Inicia Uvicorn con la fábrica create_app y las opciones de puerto, trabajador, proxy y registro del bloque.

Las importaciones cargan logging, expresiones regulares y Uvicorn. `OcultarSesion` modifica los mensajes antes de escribirlos para no conservar SIDs de 43 caracteres en las rutas. No modifica el SID que utiliza el navegador.

El bloque `if __name__ == '__main__'` se ejecuta al iniciar el módulo con Python. La cadena `deploy.presidencia.api:create_app` significa: buscar el paquete `deploy`, luego `presidencia`, importar `api.py` y llamar a `create_app`.

`host='127.0.0.1'` mantiene el puerto interno. `port=8000` fija el puerto. `workers=1` es importante: las sesiones se almacenan en memoria de un único proceso. `access_log=False` reduce el registro de URLs. Solo se confían encabezados de proxy provenientes de loopback.

La ejecución Linux real es:

```
python -m deploy.presidencia.linux.servidor
```

Debe usarse el intérprete declarado por la unidad: en el 197 es el venv nativo documentado, con las variables privadas que carga systemd. Conda solo es una alternativa de laboratorio si se elige y configura expresamente. No ejecutar otra copia en 8000 mientras el servicio existente esté activo.

3 Autenticación, sesiones y capas de seguridad

3.1. Los dos niveles de autenticación

El primer nivel identifica al **cliente API** mediante usuario, contraseña API y Bearer. El segundo comprueba al **usuario institucional** que tiene permiso para PRESIDENCIA. Tener un token API correcto no sustituye la contraseña institucional al crear la sesión.

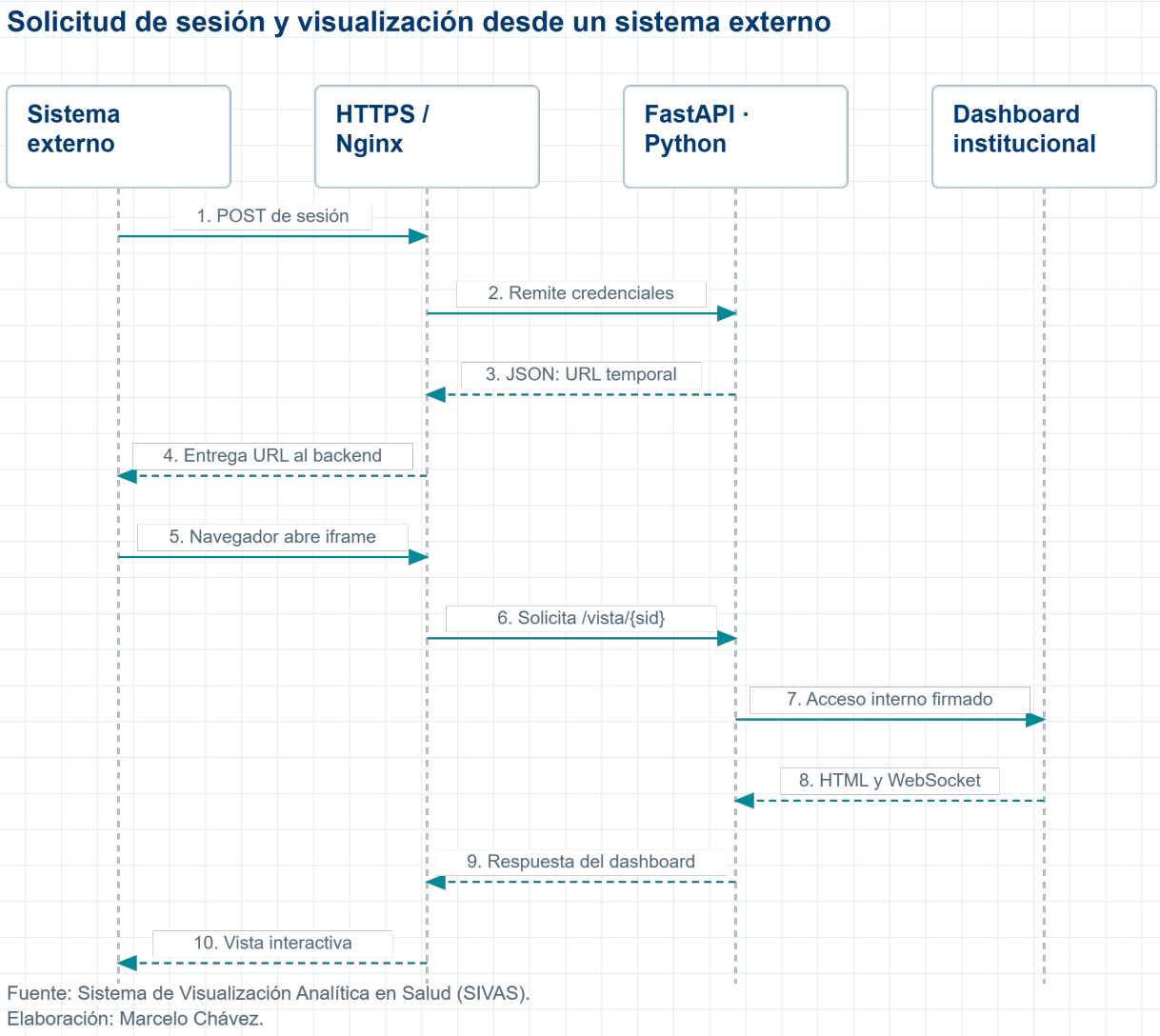


Figura 3.1: Solicitud de sesión y visualización mediante iframe.

La Figura 3.1 presenta el orden de integración. El backend del sistema externo envía el POST con las credenciales y recibe la URL temporal. Después su navegador abre esa URL dentro del iframe y mantiene la conexión con el visualizador autorizado.

3.2. Validación del archivo auth.py

Ruta en el repositorio: `deploy/presidencia/auth.py`. Este archivo contiene `PostgresIdentity`, `verify_institutional_password` y `Gateway`. La consulta `SQL_IDENTITY` verifica actividad, vigencia y asignación

al módulo; `verify_client()` valida la identidad API; `issue()` comprueba la contraseña institucional y crea la sesión. El anexo presenta cada línea del archivo junto con su explicación.

`PostgresIdentity` consulta `productos_bm.credenciales.usuarios`, `usuario_modulo`, `modulos` y `roles`. La consulta está parametrizada. Exige usuario activo y vigente, asignación activa a PRESIDENCIA y rol USUARIO o ADMINISTRADOR; la cuenta no debe ser superusuario. No crea usuarios, migra tablas o cambia permisos.

El hash institucional no se devuelve al sistema consumidor. Las contraseñas scrypt se verifican mediante PyNaCl y las Argon2 mediante PasswordHasher. **Verificar un hash no significa recuperar la contraseña original.**

`Gateway.verify_client()` comprueba al cliente API; `issue()` genera la sesión; `resolve()` revisa vigencia y cambios de identidad; `revoke()` elimina la capacidad y `headers()` firma la identidad que recibirá R. El apéndice contiene `auth.py` completo.

3.3. Diferenciar secretos y URL temporal

Elemento	Duración y destino
Bearer API	Permanente y revocable; solo backend consumidor y servicio API.
Contraseña API	Backend consumidor; en el servicio se conserva su hash.
Contraseña institucional	Backend consumidor; se verifica contra el hash institucional.
Clave HMAC	Solo comunicación interna FastAPI-Shiny; no se entrega al consumidor.
SID de iframe	Temporal; aparece en la URL y permite visualizar mientras esté vigente.

La sesión dura por defecto 28.800 segundos: ocho horas. No se prolonga con actividad. El cliente mínimo solicita otra sesión al pulsar nuevamente el botón, pero no revoca las anteriores al cerrarse. Estas vencen según el servicio. Un reinicio de FastAPI pierde sus sesiones en memoria.

El gateway limita el registro a 256 sesiones vigentes. El valor controla la admisión de sesiones en memoria; la planificación de capacidad utiliza las mediciones operativas del entorno. La cuenta y su huella de contraseña se revalidan cada diez segundos de uso, y el WebSocket también revisa la vigencia periódicamente.

3.4. Controles y dónde se aplican

La Figura 3.2 separa los controles de transporte, credenciales, capacidad temporal, puente interno y datos. Las transacciones de consulta utilizan modo de solo lectura; esta configuración no reemplaza la revisión de permisos del rol de base de datos (PostgreSQL Global Development Group, s. f.).

La política CSP `frame-ancestors` decide qué páginas pueden incrustar el contenido. El origen incluye **protocolo, host y puerto**. `http://127.0.0.1:5010` y `http://127.0.0.1:18760` son orígenes distintos. La especificación CSP establece el control de ancestros del marco (W3C, s. f.).

Se autorizaron explícitamente 127.0.0.1 y localhost en los puertos 5010 y 18760, además del propio sitio. No se utiliza un comodín. Las llamadas POST desde el backend del sistema externo no requieren CORS del navegador; esto es diferente del permiso para mostrar un iframe.

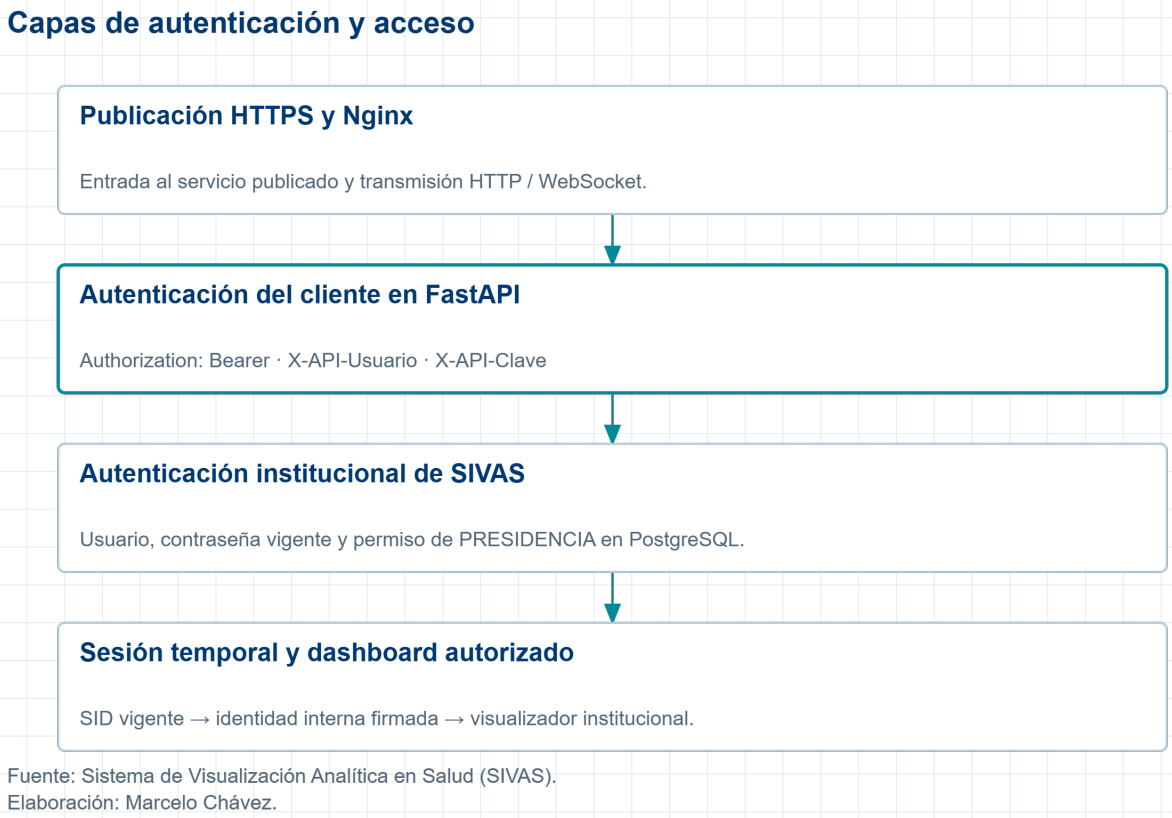


Figura 3.2: Capas de autenticación y acceso al visualizador institucional.

4 Datos y conexión con el dashboard R

4.1. Fuentes institucionales de datos

Las consultas y metas se reutilizan de `modulos/indicadores_presidencia/global.R` (Sistema de Visualización Analítica en Salud (SIVAS), 2026). Las fuentes leídas por esta implementación son:

Base	Esquema y tabla	Uso
kpis	`public.indicador_primer_nivel`	Cobertura del primer nivel.
kpis	`public.indicador_salud_mental`	Cobertura de salud mental.
kpis	`public.indicador_paquete_ninos`	Paquete priorizado de niños.
kpis	`public.indicador_gestantes`	Paquete priorizado de gestantes.
productos_bm	`presidencia.medicamentos`	Abastecimiento de medicamentos.
productos_bm	`credenciales.*`	Identidad y autorización institucional.

El sistema externo no consulta estas tablas. `bootstrap.R` crea conexiones de lectura para R; `auth.py` utiliza su conexión de lectura para identidad.

4.2. Exportación del corte autorizado

`exportar.R` llama a `presidencia_exportar()`, que une los datos y las metas institucionales y escribe un JSON agregado. Cada conjunto contiene fecha, mes, valor, meta mensual, numeradores, denominador, corte y actualización. No se exportan cédulas o nombres de pacientes.

El temporizador Linux está configurado con `OnUnitActiveSec=1h` para regenerar el corte aproximadamente cada hora. Se comprobó su estado y última ejecución, no una serie prolongada de ciclos. REST lo lee y valida su estructura, cinco indicadores, año, fechas y antigüedad máxima de 24 horas. Un corte vencido o inválido genera 503. En Linux, el archivo final se sustituye mediante renombrado atómico para que una lectura concurrente no encuentre un JSON incompleto.

El dashboard Shiny mantiene sus consultas originales; REST y el iframe son consumos distintos. El anexo contrasta ambas salidas y Excel en una muestra real. `ready` comprueba la antigüedad del archivo agregado, no que todos los indicadores tengan datos del último mes de negocio. En la muestra, primer nivel/salud mental llegan a junio; paquetes a septiembre y medicamentos a agosto de 2026. No se alteraron fuentes para completar períodos ausentes.

4.3. Acceso directo mediante identidad firmada

`shiny_app.R` carga la UI y el server originales, retira el panel de login solo de esta instancia adaptadora y suministra una identidad externa verificada. Antes de mostrarla exige la firma interna de FastAPI. No se eliminó el login del SIVAS convencional.

La firma se calcula con HMAC-SHA256. Incluye usuario, módulo, tiempo y SID. R verifica que corresponda a PRESIDENCIA, a la cuenta autorizada y a una ventana temporal de 60 segundos. El acceso directo al adaptador sin esa firma no entrega el dashboard protegido.

Archivo real: `deploy/presidencia/bridge.R`.

```r
# Solo el adaptador local acepta identidad firmada por FastAPI.
presidencia_validar_puente<-function(request) {
 clave<-Sys.getenv('PRESIDENCIA_BRIDGE_KEY')
 usuario<-request$HTTP_X_PRESIDENCIA_USUARIO %||% ''
 modulo<-request$HTTP_X_PRESIDENCIA_MODULO %||% ''
 tiempo<-request$HTTP_X_PRESIDENCIA_TIEMPO %||% ''
 sesion<-request$HTTP_X_PRESIDENCIA_SESION %||% ''
```

```
  firma<-request$HTTP_X_PRESIDENCIA_FIRMA %||% ''
  segundos<-suppressWarnings(as.numeric(tiempo))
  valido<-nchar(clave)>=32L&&identical(usuario,toupper(Sys.getenv('PRESIDENCIA_SIVAS_USER')))&&
   identical(modulo,'PRESIDENCIA')&&is.finite(segundos)&&abs(as.numeric(Sys.time())-segundos)<=60&&
   grepl('^[A-Za-z0-9_-]{43}$',sesion)&&grepl('^[a-f0-9]{64}$',firma)
  if(!valido)return(list(ok=FALSE))
  esperada<-digest::hmac(clave,paste(usuario,modulo,tiempo,sesion,sep='\n'),algo='sha256',serialize=FALSE)
  # Comparar los 64 bytes completos, sin terminar en la primera diferencia.
  diferencia<-bitwXor(as.integer(charToRaw(esperada)),as.integer(charToRaw(firma)))
  if(sum(diferencia)!=0L)return(list(ok=FALSE))
  list(ok=TRUE,usuario=data.frame(nombre_usuario=usuario,stringsAsFactors=FALSE))
}
```

5 Publicación y multiconcurrencia

5.1. Publicación del código en GitHub y despliegue de servicios

La publicación del código se realiza en GitHub. Para ejecutar esa versión en el servidor se configura el intérprete, las dependencias, las variables privadas, las unidades systemd y las rutas de Nginx. Los pasos siguientes describen esa configuración operativa.

La integración publicada utiliza servicios nativos Linux. Se conservan Docker y ShinyProxy para el aplicativo convencional, pero no se agregaron contenedores de Presidencia. Los Dockerfile y compose del directorio de integración son alternativas de desarrollo y CI; no describen el modo de ejecución que se dejó activo en el servidor 10.64.100.197.

5.2. Paso 1: comprender las rutas de Nginx

El archivo `linux/nginx-rutas.conf` agrega únicamente los endpoints de integración. El resto de las rutas mantiene su destino convencional.

```
# Incluir dentro del server 8595, antes del location / existente.
location = /api/v1/presidencia {
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
location ^~ /api/v1/presidencia/ {
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
location ^~ /vista/ {
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
location = /presidencia-api/docs {
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
location = /presidencia-api/openapi.json {
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
location ~ ^/presidencia-api/health/(live|ready|shiny)$ {
    rewrite ^/presidencia-api/(.*)$ /$1 break;
    include /etc/nginx/sivas-presidencia-proxy.conf;
}
```

El primer bloque atiende exactamente el catálogo. El segundo permite sus rutas hijas, como sesiones y series. El bloque `/vista/` conduce el HTML y WebSocket del dashboard temporal. Swagger y OpenAPI tienen nombres propios para no sustituir otras páginas de SIVAS.

La ruta pública de salud empieza por `/presidencia-api/health/`. El rewrite la adapta a `/health/` antes de pasarla a Python. OpenAPI se configura con el prefijo público para que el botón de prueba de Swagger no intente consultar una ruta equivocada.

5.3. Paso 2: comprender el proxy y WebSocket

Archivo real: `linux/nginx-proxy.conf`.

```
# Incluir únicamente en los location de la integración.
proxy_pass http://127.0.0.1:8000;
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Forwarded-Proto https;
proxy_set_header X-Forwarded-For $remote_addr;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection $connection_upgrade_analytics_msp;
proxy_read_timeout 3600s;
```

```
proxy_send_timeout 3600s;
proxy_buffering off;
proxy_redirect off;
# No registrar capacidades temporales de /vista/ ni claves de solicitudes.
access_log off;
error_log /var/log/nginx/sivas-presidencia-error.log crit;
# Declarar headers aquí impide heredar X-Frame-Options SAMEORIGIN del sitio.
# La API agrega su propia lista explícita CSP frame-ancestors.
add_header X-Content-Type-Options nosniff always;
add_header Referrer-Policy no-referrer always;
add_header X-Robots-Tag "noindex, nofollow, noarchive" always;
```

`proxy_pass` entrega las peticiones a 8000, no a Shiny directamente. `Host` y `X-Forwarded-Proto` permiten generar URLs públicas correctas. El valor HTTPS se usa porque estas rutas se publican mediante el frontal HTTPS institucional existente; no representa la instalación de un certificado nuevo en este fragmento.

Los encabezados Upgrade y Connection mantienen la transición a WebSocket, que Nginx requiere reenviar expresamente (NGINX, s. f.). En el servidor 10.64.100.197 ya existe el mapa `connection_upgrade_analytics_msp` en su configuración. En una reproducción nueva debe existir en el contexto HTTP:

```
map $http_upgrade $connection_upgrade_analytics_msp {
    default upgrade;
    '' close;
}
```

No duplicar un mapa que ya existe. Las directivas de encabezados del location evitan heredar el SAMEORIGIN del aplicativo convencional; FastAPI aporta su lista explícita de ancestros permitidos. No se quitaron las protecciones de todas las interfaces.

5.4. Paso 3: instalar las rutas sin reemplazar SIVAS

Tipo: reproducción de una instalación nueva o futuro despliegue autorizado; no ejecutado en esta revisión. **Equipo:** Linux con administración de Nginx. **Directorio:** raíz del checkout. **Archivos:** las dos plantillas de `deploy/presidencia/linux/`, sus includes en `/etc/nginx/` y la configuración del server 8595. **Propósito:** enrutar únicamente la integración. En el 197 se comprobó por lectura la inclusión en `/etc/nginx/conf.d/analytics-shinyproxy-8595.conf`, línea 96; no se editó ni recargó. Los comandos siguientes son instrucciones, no evidencia de haberlos ejecutado.

```
sudo cp /etc/nginx/conf.d/analytics-shinyproxy-8595.conf
 ↪  /etc/nginx/conf.d/analytics-shinyproxy-8595.conf.respaldo
sudo install -m 644 deploy/presidencia/linux/nginx-proxy.conf /etc/nginx/sivas-presidencia-proxy.conf
sudo install -m 644 deploy/presidencia/linux/nginx-rutas.conf /etc/nginx/sivas-presidencia-rutas.conf
sudoedit /etc/nginx/conf.d/analytics-shinyproxy-8595.conf
```

Agregar dentro del server 8595, antes del `location /` convencional:

```
include /etc/nginx/sivas-presidencia-rutas.conf;
```

Después:

```
sudo nginx -t
sudo systemctl reload nginx
```

Resultado esperado: `nginx -t` confirma sintaxis correcta; la recarga aplica la nueva configuración. No reinicia Docker ni ShinyProxy. Si la validación falla, no recargar: restaurar el archivo respaldado o corregir el error concreto.

5.5. Paso 4: verificar el acceso público

Dónde: PowerShell Windows, sin credenciales para las rutas de salud.

```
Invoke-RestMethod 'https://analiticadesalud.msp.gob.ec/presidencia-api/health/live'
Invoke-RestMethod 'https://analiticadesalud.msp.gob.ec/presidencia-api/health/ready'
Invoke-RestMethod 'https://analiticadesalud.msp.gob.ec/presidencia-api/health/shiny'
```

`live` comprueba que el proceso atiende; `ready` que el corte REST está disponible y vigente; `shiny` que el adaptador HTTP responde. Estas comprobaciones no reemplazan el POST autenticado ni una prueba interactiva del iframe.

5.6. Configuración de las unidades systemd

La unidad de la API es el vínculo entre Linux y el Python que se construyó:

```
[Unit]
Description=SIVAS - API de integración de Presidencia
After=network-online.target sivas-presidencia-shiny.service
Wants=network-online.target sivas-presidencia-shiny.service

[Service]
Type=simple
User=marcelo.chavez
Group=marcelo.chavez
WorkingDirectory=/home/marcelo.chavez/PROYECTO_BM/APPs/health_analytics_msp
EnvironmentFile=/etc/sivas-presidencia/environment
ExecStart=/home/marcelo.chavez/.sivas-presidencia/venv/bin/python -m deploy.presidencia.linux.servidor
Restart=on-failure
RestartSec=5
UMask=0077
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ReadWritePaths=/home/marcelo.chavez/.sivas-presidencia

[Install]
WantedBy=multi-user.target
```

`WorkingDirectory` sitúa el proceso en la raíz del proyecto. `EnvironmentFile` carga la configuración privada. `ExecStart` elige el Python aislado y ejecuta `servidor.py` como módulo. `Restart=on-failure` reinicia el proceso si falla; no conserva las sesiones que estaban en su memoria. Las últimas directivas restringen privilegios y escritura.

La unidad Shiny ejecuta `Rscript deploy/presidencia/shiny_app.R`. El exportador es una tarea `oneshot`: termina al producir el corte. El timer lo invoca cada hora, por lo que verlo inactivo después de una ejecución correcta es normal; lo que debe estar activo para repetirlo es el temporizador.

5.7. Iniciar, detener, reiniciar y actualizar

Consultar:

```
sudo systemctl status sivas-presidencia-api sivas-presidencia-shiny
sudo systemctl list-timers sivas-presidencia-exportar.timer
sudo systemctl show sivas-presidencia-exportar.service -p ExecMainStatus
```

Actualizar solo los datos REST:

```
sudo systemctl start sivas-presidencia-exportar.service
```

Reiniciar únicamente la integración después de un cambio autorizado:

```
sudo systemctl restart sivas-presidencia-shiny.service sivas-presidencia-api.service
```

Esto desconecta los visualizadores de integración y obliga a solicitar nuevas sesiones. No hace falta reiniciar ambos si solo se cambió la lista CSP: en ese caso reiniciar únicamente la API.

Detener la integración:

```
sudo systemctl stop sivas-presidencia-exportar.timer sivas-presidencia-api.service
↪  sivas-presidencia-shiny.service
```

Las rutas públicas seguirán presentes mientras no se retiren de Nginx; mostrarán indisponibilidad si sus procesos están detenidos. Detener systemd no desinstala archivos ni elimina credenciales.

5.8. Consulta de registros operativos

```
sudo journalctl -u sivas-presidencia-api.service --since '10 minutes ago'
sudo journalctl -u sivas-presidencia-shiny.service --since '10 minutes ago'
sudo journalctl -u sivas-presidencia-exportar.service -n 30
```

La API redacta SID en sus rutas de registro; Nginx no registra access_log de la integración y sus errores se limitan a nivel crítico. Los registros operativos siguen siendo información de acceso restringido. No compartirlos sin revisión.

5.9. Gestión de sesiones y conexiones simultáneas

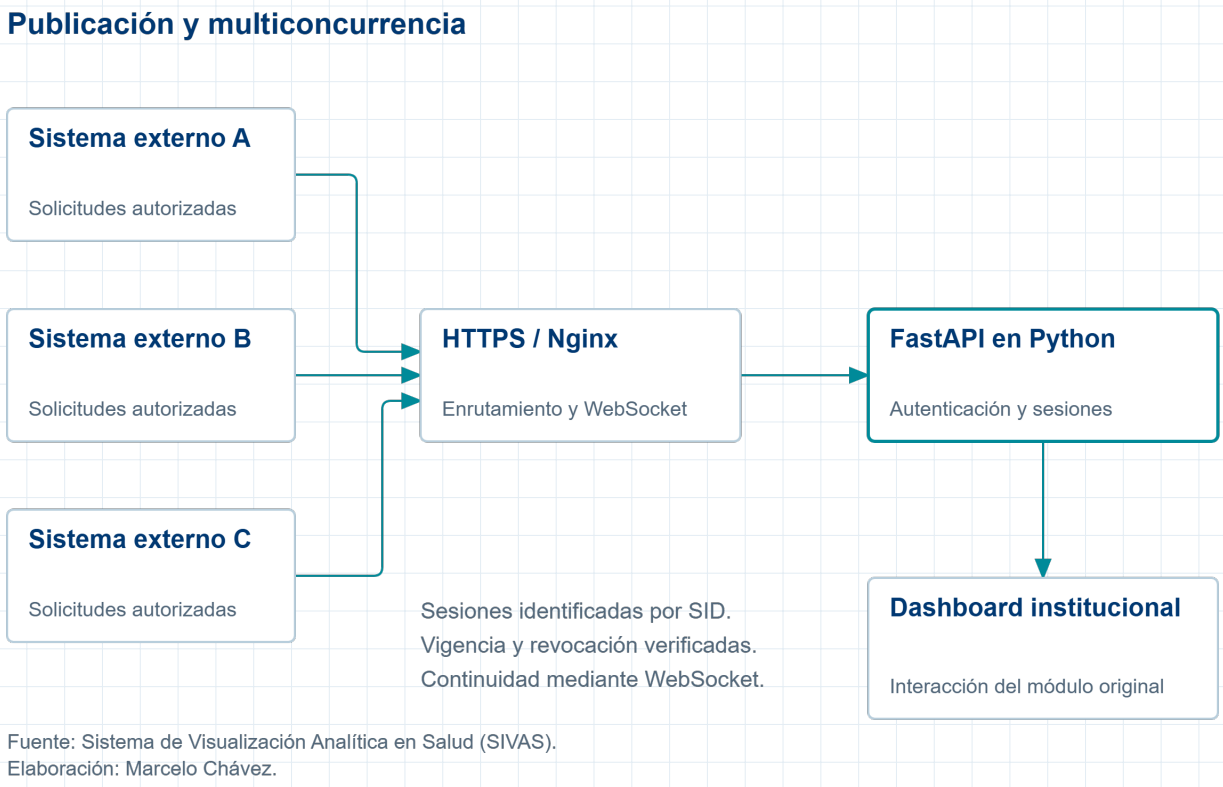


Figura 5.1: Gestión de conexiones simultáneas en el servicio publicado.

La Figura 5.1 muestra cómo entran solicitudes simultáneas desde sistemas externos al mismo servicio publicado. Nginx recibe las conexiones; FastAPI atiende las operaciones HTTP y las sesiones autorizadas; el visualizador mantiene la interacción del dashboard mediante WebSocket. El aplicativo principal continúa con su despliegue existente.

Las sesiones de visualización se conservan en la memoria del proceso FastAPI configurado en `servidor.py`. El gateway valida vencimiento, revocación y el límite configurado de sesiones. Al planificar réplicas o trabajadores adicionales, se debe conservar una resolución coherente del SID mediante afinidad o almacenamiento compartido. El control operativo revisa conexiones PostgreSQL, CPU, memoria, tiempos HTTP y continuidad de WebSocket. Las cifras de capacidad se obtienen del procedimiento de pruebas del entorno autorizado.

5.10. CI y despliegue del repositorio

Archivo: `.github/workflows/presidencia-integracion.yml`. Cuando cambian las rutas contempladas, GitHub Actions instala dependencias Python, ejecuta pytest y construye una imagen API de desarrollo. Un disparo manual puede empaquetar esa imagen y, opcionalmente, construir Shiny.

El workflow no contiene un despliegue automático al servidor 10.64.100.197 ni aprovisionamiento de sus secretos. La instalación nativa documentada es una operación distinta. El código copiado mediante Git debe coincidir con el publicado, y los secretos deben conservarse fuera de Git.

6 Consumo de la URL

6.1. Direcciones que debe conocer

Función	URL o ruta
Base del cliente	`https://analiticadesalud.msp.gob.ec`
Swagger	`/presidencia-api/docs`
Catálogo REST	`/api/v1/presidencia`
Crear sesión	`POST /api/v1/presidencia/sesiones`
Serie de un indicador	`/api/v1/presidencia/<id>/serie`
Ejemplo de comprobación `TEST_API.R`	`http://127.0.0.1:5010/`

Las rutas de la tabla, salvo la prueba local, se agregan a la base HTTPS. Los cinco IDs válidos son `primer-nivel`, `salud-mental`, `paquete-ninos`, `paquete-gestantes` y `medicamentos`.

6.2. Configurar el YAML privado

Archivo del ejemplo `TEST_API.R`: `modulos/comprobacion_api/config_credenciales_api.yml`. Esta es su estructura **sin secretos reales**:

```yaml
servicio:
  base_url: https://analiticadesalud.msp.gob.ec
  timeout_segundos: 30
fastapi:
  usuario: integracion_presidencia
  clave: '<CONTRASEÑA_API_PRIVADA>'
  token: '<BEARER_PRIVADO>'
sivas:
  usuario: daniel.azin.0698
  clave: '<CONTRASEÑA_INSTITUCIONAL_PRIVADA>'
```

Los marcadores no son credenciales funcionales. El archivo privado existente ya se completó durante la implementación. No cambiar sus valores para resolver un error de puerto o de CSP.

La configuración privada de desarrollo de la integración utiliza también este archivo, excluido del repositorio:

```
deploy/presidencia/runtime/credenciales_integracion.json
```

La contraseña institucional privada está en `credenciales_sivas.json`, en la misma carpeta. `TEST_API.R` lee el YAML, no esos JSON. Si se actualizan credenciales en un medio, se debe mantener la configuración consumidora coherente mediante el canal seguro correspondiente.

El `.gitignore` excluye los YAML cuyo nombre empieza por `config_credenciales` dentro del módulo de comprobación. Excluir de Git no cifra el archivo: se debe proteger también la carpeta Windows, OneDrive y sus respaldos.

6.3. Abrir la comprobación

Dónde: PowerShell Windows, desde la raíz del proyecto.

```
& 'C:/Program Files/R/R-4.6.0/bin/x64/Rscript.exe' .\modulos\comprobacion_api\TEST_API.R
```

Abrir `http://127.0.0.1:5010/` y pulsar **Mostrar dashboard**. Debe aparecer el módulo original, sin un formulario institucional adicional. El selector interior permite navegar los cinco indicadores.

Alternativa desde la consola R de Positron, con codificación UTF-8 y el proyecto como directorio de trabajo:

```
app <- source('modulos/comprobacion_api/TEST_API.R', encoding = 'UTF-8')$value
shiny::runApp(app, host = '127.0.0.1', port = 5010)
```

Si el puerto ya está ocupado, detener la instancia propia anterior desde su terminal. No iniciar una segunda en el mismo puerto ni terminar indiscriminadamente todos los procesos R. Ctrl+C detiene la ejecución en terminal; en la sesión R, utilizar el control de parada de la aplicación.

6.4. Probar en Swagger

1. Abrir `https://analiticadesalud.msp.gob.ec/presidencia-api/docs`.
2. Pulsar **Authorize**.
3. En Bearer ingresar solo el token; Swagger agrega la palabra Bearer.
4. Completar UsuarioAPI y ClaveAPI con las credenciales API.
5. Abrir el POST de sesiones y pulsar **Try it out**.
6. Completar únicamente usuario_sivas, clave_sivas e indicador en el JSON.
7. Ejecutar. Con ambas autenticaciones válidas se espera HTTP 200 y cinco URLs.
8. Usar la URL temporal recibida; no pegarla en documentación o repositorios.

Una respuesta 401 al abrir el catálogo directamente en el navegador, sin encabezados, es normal. No significa que el servicio esté caído.

6.5. Respuesta del POST de sesiones

Este ejemplo es ilustrativo; `SESION_TEMPORAL` no es una capacidad real ni una URL para abrir:

```
{
  "url": "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia",
  "demo": "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/demo",
  "vigencia_segundos": 28800,
  "indicadores": [
    {"id": "primer-nivel", "url":
     ↪  "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia/primer-nivel"},
    {"id": "salud-mental", "url":
     ↪  "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia/salud-mental"},
    {"id": "paquete-ninos", "url":
     ↪  "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia/paquete-ninos"},
    {"id": "paquete-gestantes", "url":
     ↪  "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia/paquete-gestantes"},
    {"id": "medicamentos", "url":
     ↪  "https://analiticadesalud.msp.gob.ec/vista/SESION_TEMPORAL/embed/presidencia/medicamentos"}
  ]
}
```

`url` abre el módulo con el indicador inicial. `demo` es una página HTML con un iframe. La lista `indicadores` permite abrir cada dashboard directamente. La vigencia corresponde a la sesión de visualización, no a la fecha del dato estadístico.

6.6. Solicitud HTTP de prueba con PowerShell

Para este ejemplo se requieren las variables del consumidor cargadas desde un gestor o archivo privado. El cargador existente `deploy/presidencia/examples/shiny_windows/cargar_entorno.ps1` puede leer los dos JSON privados anteriores, si siguen correspondiendo a las credenciales vigentes. No imprime sus valores. Esta es una alternativa de consumo; no es la forma en que `TEST_API.R` lee su YAML.

```powershell
. .\deploy\presidencia\examples\shiny_windows\cargar_entorno.ps1
$base = $env:PRESIDENCIA_API_BASE_URL
$headers = @{
  Authorization = 'Bearer ' + $env:PRESIDENCIA_API_KEY
  'X-API-Usuario' = $env:PRESIDENCIA_CLIENT_USER
  'X-API-Clave' = $env:PRESIDENCIA_CLIENT_PASSWORD
}
$body = @{
  usuario_sivas = $env:PRESIDENCIA_SIVAS_USER
  clave_sivas = $env:PRESIDENCIA_SIVAS_PASSWORD
  indicador = 'primer-nivel'
} | ConvertTo-Json
$sesion = Invoke-RestMethod -Method Post -Uri "$base/api/v1/presidencia/sesiones" -Headers $headers -ContentType
 ↪  'application/json' -Body $body
$serie = Invoke-RestMethod -Uri "$base/api/v1/presidencia/primer-nivel/serie?anio=2026&mes_desde=1&mes_hasta=6"
 ↪  -Headers $headers
# No imprimir encabezados, body o sesión: contienen secretos o una URL de acceso.
Start-Process $sesion.url
```

6.7. Ejemplo HTML del iframe

El backend sustituye el marcador por la URL real que acaba de recibir. El HTML no contiene el Bearer ni las dos contraseñas.

```html
<iframe
  src="URL_TEMPORAL_DEVUELTA_POR_LA_API"
  title="Indicadores de Presidencia"
  referrerpolicy="no-referrer"
  style="width:100%;height:85vh;border:0;">
</iframe>
```

No abrir el HTML con `file://` esperando que tenga el mismo origen que Shiny Windows. La integración se prueba en el origen HTTP o HTTPS autorizado; un origen nuevo requiere su autorización explícita en CSP.

7 TEST_API.R explicado línea por línea

7.1. El archivo completo y su función

Ubicación: `modulos/comprobacion_api/TEST_API.R`. El siguiente bloque proviene del archivo real. Shiny organiza una aplicación en interfaz, server y objeto de app (Posit, s. f.-b). Aquí esos componentes se mantienen juntos, como fue solicitado.

```
1  if (.Platform$OS.type == 'windows') Sys.setlocale('LC_CTYPE', 'Spanish_Ecuador.utf8')
2
3  carpeta <- if (file.exists('modulos/comprobacion_api/config_credenciales_api.yml')) 'modulos/comprobacion_api'
    ↪  else '.'
4  datos <- yaml::read_yaml(file.path(carpeta, 'config_credenciales_api.yml'), eval.expr = FALSE)
5  source(file.path(carpeta, '../../deploy/presidencia/examples/shiny_windows/cliente.R'), encoding = 'UTF-8', local
    ↪  = TRUE)
6  config <- list(base = datos$servicio$base_url, timeout = datos$servicio$timeout_segundos,
7    usuario_api = datos$fastapi$usuario, clave_api = datos$fastapi$clave, token = datos$fastapi$token,
8    usuario_sivas = datos$sivas$usuario, clave_sivas = datos$sivas$clave)
9
10 ui <- shiny::fluidPage(
11   shiny::titlePanel('Comprobación API - Indicadores de Presidencia'),
12   shiny::actionButton('mostrar', 'Mostrar dashboard', class = 'btn-primary'),
13   shiny::uiOutput('dashboard')
14 )
15
16 server <- function(input, output, session) {
17   output$dashboard <- shiny::renderUI({
18     shiny::req(input$mostrar)
19     tryCatch({
20       sesion <- presidencia_solicitar_sesion(config)
21       shiny::tags$iframe(src = sesion$url, title = 'Indicadores de Presidencia',
22         referrerpolicy = 'no-referrer', style = 'width:100%;height:85vh;border:0;margin-top:12px;')
23     }, error = function(e) shiny::div(class = 'alert alert-danger', conditionMessage(e)))
24   })
25 }
26
27 app <- shiny::shinyApp(ui, server, options = list(host = '127.0.0.1', port = 5010, launch.browser = if
    ↪  (.Platform$OS.type == 'windows') function(url) shell.exec(url) else FALSE))
28 if (sys.nframe() == 0L) shiny::runApp(app, host = '127.0.0.1', port = 5010, launch.browser =
    ↪  app$options$launch.browser) else app
```

Línea	Explicación
1	En Windows configura la interpretación de caracteres en español. `.Platform$OS.type` consulta el sistema y `Sys.setlocale()` cambia la configuración del proceso R.
2	Línea en blanco: separa bloques para facilitar la lectura; no ejecuta ninguna operación.
3	Busca el YAML desde la raíz del repositorio. Si no existe allí, usa el directorio actual. `<-` asigna el resultado a `carpeta`.
4	Lee la configuración privada como una lista R. `file.path()` arma la ruta; `eval.expr = FALSE` impide evaluar expresiones R incluidas en YAML. No envía una solicitud todavía.
5	Carga las funciones de `cliente.R` con UTF-8. `local = TRUE` las incorpora al entorno que ejecuta el archivo. Aquí nace la función que se llamará en la línea 20.
6	Inicia la lista `config`: guarda la URL pública y el tiempo máximo de espera. `$` accede a un elemento de la lista anidada que produjo YAML.
7	Añade el usuario, la clave del cliente API y el token Bearer. Se mantienen en el servidor R; no se convierten en campos del iframe.
8	Añade el usuario y la contraseña institucionales y cierra la lista. Son las credenciales del segundo nivel.
9	Línea en blanco: separa bloques para facilitar la lectura; no ejecuta ninguna operación.
10	Define la interfaz con `fluidPage()`. `shiny::` selecciona una función del paquete Shiny sin cargar todo su espacio de nombres.

Línea	Explicación
11	Dibuja el título de la pequeña aplicación de comprobación. No cambia el diseño del dashboard institucional.
12	Crea el botón cuyo identificador es `mostrar`. Su contador comienza en cero y aumenta con cada clic.
13	Reserva el lugar donde el servidor insertará el iframe o un mensaje de error. El identificador coincide con `output$dashboard`.
14	Cierra la definición de la interfaz.
15	Línea en blanco: separa bloques para facilitar la lectura; no ejecuta ninguna operación.
16	Define el servidor de cada sesión de usuario. `input` contiene entradas, `output` las salidas y `session` el contexto Shiny.
17	Registra una salida reactiva de interfaz. `renderUI()` permite crear una etiqueta HTML después de obtener la URL.
18	`req()` detiene silenciosamente esta salida mientras el contador del botón sea cero. Al pulsarlo se permite continuar; cada nuevo clic solicita otra sesión.
19	Abre un bloque de manejo de errores. Si falla la conexión o la autenticación, se mostrará el mensaje de la línea 23.
20	Llama al cliente HTTP real. La función construye las tres cabeceras, envía el POST con ambas credenciales SIVAS, valida el estado y devuelve el JSON como lista R.
21	Crea el iframe con la URL temporal devuelta por FastAPI. El navegador carga esa URL; no recibe la clave API ni la contraseña institucional.
22	Evita enviar el referente y configura ancho, alto y borde. `85vh` ocupa el 85 % de la altura visible. Cierra la etiqueta iframe.
23	Si ocurre un error, genera un aviso rojo con el mensaje de la condición. No imprime la configuración privada ni los encabezados.
24	Cierra el bloque de renderizado y su asignación a la salida.
25	Cierra la función del servidor.
26	Línea en blanco: separa bloques para facilitar la lectura; no ejecuta ninguna operación.
27	Crea la aplicación en Windows, puerto 5010. `launch.browser` usa `shell.exec(url)` para abrir el navegador predeterminado de Windows, sin depender de que Firefox esté en PATH. En otros sistemas no abre un navegador automáticamente. El destino API sigue siendo el dominio público.
28	Si se ejecuta directamente, inicia Shiny y pasa explícitamente la función de apertura del navegador configurada en la línea 27. Si se carga desde otro contexto, devuelve `app`. No ejecutar dos instancias simultáneas en el mismo puerto.

7.2. Solicitud HTTP utilizada por TEST_API.R

La línea que dice `sesion <- presidencia_solicitar_sesion(config)` no llama a una función incorporada en R. Esa función fue cargada por `source(...)` desde `deploy/presidencia/examples/shiny_windows/cliente.R`.

El trayecto completo es:

```
TEST_API.R
  source(cliente.R)                  carga las funciones; todavía no hay POST
  pulsar Mostrar dashboard
  presidencia_solicitar_sesion(config)
    presidencia_http_cliente(..., metodo='POST')
      httr::POST(...)                 aquí sale la petición por internet
        Nginx del servidor 10.64.100.197
          api.py / open_session(...)  Python valida y devuelve JSON
    presidencia_validar_respuesta(...) valida URLs y contrato
  sesion$url                          R toma la URL de la respuesta
  tags$iframe(src=sesion$url)          el navegador abre el dashboard
```

Este es el código real de la primera función del helper:

```r
presidencia_solicitar_sesion <- function(config = presidencia_config_cliente(), indicador = 'primer-nivel') {
  if (length(indicador) != 1L || !indicador %in% presidencia_ids_cliente) stop('Seleccione un indicador autorizado.', call. = FALSE)
  inicio <- Sys.time()
  datos <- presidencia_http_cliente(config, '/api/v1/presidencia/sesiones', metodo = 'POST',
    body = list(usuario_sivas = config$usuario_sivas, clave_sivas = config$clave_sivas, indicador = indicador))
  presidencia_validar_respuesta(datos, config, inicio)
}
```

La validación del indicador limita la petición a los cinco IDs. `inicio` registra el momento del envío. `presidencia_http_cliente` recibe la ruta, el método POST y el cuerpo institucional. Por último se comprueba que las URLs devueltas pertenezcan al origen y sesión esperados.

7.3. Construcción de la URL y los encabezados HTTP

En `presidencia_http_cliente` se concatena la base con la ruta y se preparan los encabezados. No se envían como parte de `src` del iframe.

```r
presidencia_http_cliente <- function(config, ruta, body = NULL, metodo = 'GET', autorizada = TRUE, origin = FALSE, json = TRUE) {
  url <- paste0(config$base, ruta)
  presidencia_origen(url)
  headers <- c(Accept = 'application/json')
  if (autorizada) headers <- c(headers, Authorization = paste('Bearer', config$token),
    'X-API-Usuario' = config$usuario_api, 'X-API-Clave' = config$clave_api)
```

El resultado es una solicitud a la base HTTPS más `/api/v1/presidencia/sesiones`. El Bearer se toma de `config$token`, el usuario API de `config$usuario_api` y la clave de `config$clave_api`.

7.4. Ejecución del POST desde el cliente R

```r
  respuesta <- tryCatch({
    opciones <- list(httr::add_headers(.headers = headers), httr::timeout(config$timeout), httr::config(followlocation = 0L))
    if (identical(metodo, 'POST')) {
      do.call(httr::POST, c(list(url = url, body = body, encode = 'json'), opciones))
    } else do.call(httr::GET, c(list(url = url), opciones))
  }, error = function(e) stop('No se pudo conectar con la API. Revise servicio, URL, certificado, red o VPN.', call. = FALSE))
  status <- httr::status_code(respuesta)
  if (status != 200L) {
```

`do.call(httr::POST, ...)` llama a `httr::POST` con los argumentos reunidos en una lista. `encode='json'` serializa el cuerpo; `httr::timeout` limita la espera y `followlocation=0` evita entregar credenciales siguiendo una redirección. Después el helper comprueba estado HTTP, tipo JSON y contenido.

Para comprenderlo sin la llamada indirecta, este ejemplo didáctico expresa la misma petición con argumentos visibles:

```r
respuesta <- httr::POST(
  url = paste0(config$base, '/api/v1/presidencia/sesiones'),
  httr::add_headers(
    Authorization = paste('Bearer', config$token),
    'X-API-Usuario' = config$usuario_api,
    'X-API-Clave' = config$clave_api
  ),
  body = list(usuario_sivas = config$usuario_sivas,
              clave_sivas = config$clave_sivas, indicador = 'primer-nivel'),
  encode = 'json',
  httr::timeout(config$timeout),
  httr::config(followlocation = 0L)
)
```

Este ejemplo no es un segundo cliente que se deba incorporar al proyecto. Ayuda a leer el helper que ya existe. La app reutiliza ese helper para no duplicar la lógica de seguridad.

7.5. Renderizado del iframe en TEST_API.R

El iframe necesita una URL que aún no existe al cargar la página. Primero hay que pulsar el botón y solicitar una sesión. `renderUI` produce el elemento HTML después de recibirla; `uiOutput` marca dónde colocarlo. `req(input$mostrar)` evita pedir una sesión antes del clic.

No hay observers, estado adicional o temporizadores en la versión mínima. El navegador interactúa con el dashboard dentro del iframe; ese contenido no vuelve a ejecutar el POST del cliente por cada filtro interior.

El archivo usa rutas relativas. Ejecutarlo desde la raíz del proyecto o desde la carpeta del módulo. El ejemplo de Positron muestra cómo obtener el objeto de app mediante `source(... )$value`; la última línea del archivo distingue ese caso de una ejecución directa con Rscript.

8 Pruebas y resolución de problemas

8.1. Secuencia de prueba recomendada

1. Comprobar salud pública: debe responder JSON con estado activo.
2. Comprobar corte REST y adaptador: deben estar disponibles.
3. Consultar catálogo sin credenciales: se espera 401, no acceso anónimo.
4. En un entorno de prueba autorizado, comprobar el rechazo de Bearer, clave API o contraseña institucional incorrectos. No repetir fallos rápidamente: existe limitación de intentos.
5. Enviar el POST con ambas autenticaciones correctas: se esperan cinco URLs HTTPS.
6. Abrir una URL y comprobar que no aparece otro login.
7. Navegar los cinco indicadores, verificar títulos, KPIs, gráficas y tablas, y probar los controles que se vayan a utilizar.
8. Comprobar que WebSocket se mantiene conectado; una imagen visible por sí sola no demuestra interacción.
9. Cerrar explícitamente una sesión de prueba y comprobar que su URL se rechaza.
10. Para certificar duración o carga, diseñar pruebas específicas. No deducir su resultado de las anteriores.

8.2. Pruebas automatizadas disponibles

Ejecutado en Windows: venv existente de Presidencia, con pytest instalado. Desde la raíz del repositorio:

```
& '.\deploy\presidencia\.venv\Scripts\python.exe' -m pytest .\deploy\presidencia\tests -q
```

Esta revisión obtuvo 32 aprobaciones. Conda `msp_01` no tiene pytest instalado en el equipo comprobado; no se le atribuyen esas pruebas. No hace falta modificar ese entorno para reproducir la ejecución con el venv disponible.

Guía para un clon Linux de laboratorio, no ejecutada en el 197: preparar allí un venv de pruebas y usar su intérprete. No reutilizar el servicio productivo para ensayos negativos.

```
/RUTA/DEL/CLON/.venv/bin/python -m pytest deploy/presidencia/tests -q
```

Los tests crean datos e identidades de prueba. Revisan autenticación, contratos, vigencia y revocación, validación de parámetros, rutas, configuración de OpenAPI y redacción de sesiones. No equivalen a una prueba completa del navegador, del frontal HTTPS o de una base real.

El cliente reutilizable también tiene pruebas R con respuestas controladas:

```
& 'C:/Program Files/R/R-4.6.0/bin/x64/Rscript.exe' .\deploy\presidencia\examples\shiny_windows\validar_cliente.R
```

La opción `--http` de ese script está limitada deliberadamente al API local de desarrollo. No utilizarla para suponer una prueba automática contra producción.

8.3. Antecedentes históricos y revisión vigente

El registro anterior describía 32 pruebas Windows/Linux, renderizado de cinco indicadores y otras comprobaciones de despliegue. No contenía hora, versión y registro primario por caso. Se conserva como antecedente depurado en `recursos/registros/antecedente_previo.json`; no se reconstruyen resultados Linux ni se convierten en aprobaciones actuales.

La revisión vigente diferencia:

1. **Sintéticas:** 32 casos Python en Windows, cliente R controlado y firma HMAC de prueba. Las identidades incorrectas se ensayaron en fixtures, sin bloquear cuentas reales.

2. **Locales:** versiones, sintaxis y lectura de las cinco descargas con Excel en modo de solo lectura.
3. **Reales del 197:** autenticación correcta, series REST, cinco enlaces embebidos, WebSocket, gráficos/KPIs/tablas, controles, XLSX y cierre. La evidencia y su alcance por indicador están en el anexo.
4. **Incidencia PDF:** lectura directa de cuatro archivos aprobada; vista embebida bloqueada acreditada en dos casos y no concluida en dos. El botón Descargar ficha no se certifica con una lectura de su URL estática.

No se probó duración efectiva de ocho horas, inactividad real, carga, otra red, Firefox ni todas las funciones de exportación/gestos de Highcharts. El panel Direcciones provinciales continúa En construcción. La conclusión del anexo conserva estas limitaciones.

8.4. Error: Failed to create server en Windows

Este error ocurrió porque se dejó una instancia de prueba escuchando en 5010 y después se intentó iniciar otra. Es un conflicto del servidor HTTP local, no un fallo de las credenciales FastAPI.

Comprobar el listener:

```
Get-NetTCPConnection -LocalPort 5010 -State Listen
```

Identificar su proceso con el PID mostrado. Detener únicamente la instancia propia conocida desde su terminal o control de parada. No terminar procesos R ajenos ni cambiar la contraseña institucional. Una vez libre el puerto, ejecutar nuevamente la app. Al finalizar la tarea de diagnóstico se detuvo la instancia del agente y se confirmó que 5010 quedaba libre.

Si se abre `TEST_API.R` con Rscript, mantener esa terminal activa mientras se prueba. Para editar y volver a ejecutar, detener primero la instancia propia; en Windows un proceso puede mantener abierto el archivo que está ejecutando.

8.5. Aviso: Firefox no encontrado al abrir la app

El aviso `"firefox" not found` aparece cuando R intenta abrir un ejecutable de navegador que no está disponible en PATH. Afecta la apertura automática, no demuestra una falla de autenticación o del API. En Windows, `TEST_API.R` utiliza `shell.exec(url)` como función de apertura: Windows elige su navegador predeterminado. No se modifica la configuración global de R ni se exige instalar Firefox.

Si Positron decide mostrar su propio visor, también se puede abrir manualmente `http://127.0.0.1:5010` mientras la app esté ejecutándose. Esta dirección corresponde al consumidor Windows, no al servicio FastAPI del servidor 10.64.100.197.

8.6. Error: Firefox no permite mostrar la página incrustada

La autenticación puede haber terminado correctamente y, aun así, el navegador impedir el iframe. En el caso observado se ejecutaba la app en 5010, mientras el servicio permitía 18760. Faltaba el origen exacto en `frame-ancestors`.

1. Identificar el origen real en la barra del navegador.
2. Consultar CSP y X-Frame-Options de la respuesta del iframe y de su contenido.
3. Agregar el origen exacto autorizado a `PRESIDENCIA_FRAME_ANCESTORS` del servicio.
4. Respaldar y reiniciar solo la API para aplicar esa configuración.
5. Pulsar Mostrar dashboard para crear una sesión nueva y recargar el iframe.

No resolverlo quitando la seguridad de todo SIVAS, usando `*`, deshabilitando protecciones del navegador o abriendo puertos internos a internet.

8.7. Otros síntomas y qué revisar

Síntoma	Revisión concreta
401	Distinguir Bearer, usuario/clave API, contraseña institucional y sesión vencida.
403 al cerrar	Origin debe coincidir con la base HTTPS de la API.
422	Nombres del JSON, ID de indicador, año 2026 y rango mensual.
503 de autorización	PostgreSQL accesible, cuenta vigente y consulta de identidad disponible.
503 de datos	Exportador, permisos del JSON, estructura, fecha y vigencia del corte.
502 del proxy	Estado de API/Shiny y destinos de Nginx.
JSON esperado pero llega HTML	URL apunta al aplicativo convencional o a una ruta incorrecta.
Pantalla que carga y se desconecta	WebSocket, proxy Upgrade, vigencia y conexión de datos.
YAML no encontrado	Directorio de trabajo y nombre actual del archivo.
Importación Python falla	Identificar el intérprete de ExecStart o el venv de laboratorio y verificar sus dependencias; no instalar paquetes automáticamente en producción.
URL temporal contiene localhost	Esquema/Host propagados por proxy o base configurada incorrectamente.
429	Puede corresponder a un proxy; el limitador actual del gateway utiliza 401 al denegar.

Un 503 no debe convertirse en un permiso para omitir autenticación. Un 401 no se corrige creando usuarios ficticios o cambiando permisos sin diagnóstico.

8.8. Rollback de un despliegue

Conservar respaldo y diferencias antes de actualizar. Si debe retirarse la integración, restaurar únicamente la configuración Nginx anterior correspondiente, validar con `nginx -t`, recargar y detener/deshabilitar sus servicios nuevos. No restaurar ciegamente un archivo viejo si TI hizo cambios posteriores.

El respaldo inicial documentado quedó bajo:

```
/home/marcelo.chavez/.sivas-presidencia/respaldo_20261023_023238/
```

La ampliación de CSP para 5010 generó además un archivo privado `environment.pre5010_<fecha>`. Los nombres de respaldo están en UTC; pueden mostrar el día siguiente a la fecha local de Ecuador. No se incluye su contenido secreto en este manual.

9 Trazabilidad técnica y adaptación a otro módulo

9.1. Qué se conectó realmente

La integración actual comprende **solo los cinco indicadores de Presidencia**. No incorpora BAI, EAIS, Enfermería, indicadores corporativos ni otros módulos. FastAPI no vuelve a dibujar los gráficos: publica datos agregados y controla el acceso a un adaptador que carga las funciones originales de Presidencia en R Shiny.

En el 197 existe un proceso Shiny específico de esta integración, además del aplicativo convencional. Por tanto, es preciso decir que **se añadieron servicios y rutas de integración**, y no que no se modificó nada en el servidor. Durante la auditoría del 26 de octubre de 2026 no se instalaron servicios, no se reiniciaron procesos y no se sincronizó código.

Hay dos recorridos diferentes:

1. **Datos REST:** `exportar.R` utiliza `bootstrap.R` para consultar las fuentes con las funciones del módulo y publicar un corte JSON. Los endpoints leen ese archivo; no ejecutan el dashboard para responder cada GET. El temporizador instalado refresca el corte aproximadamente cada hora. Esto no equivale a actualizar las tablas fuente cada hora.
2. **Visualización:** el iframe utiliza una URL de sesión de FastAPI. FastAPI valida el SID y reenvía HTTP y WebSocket al adaptador Shiny. Este carga la UI y el server institucionales y consulta sus fuentes mediante el bootstrap. El iframe conserva la presentación y la lógica del módulo; el corte REST es una salida distinta.

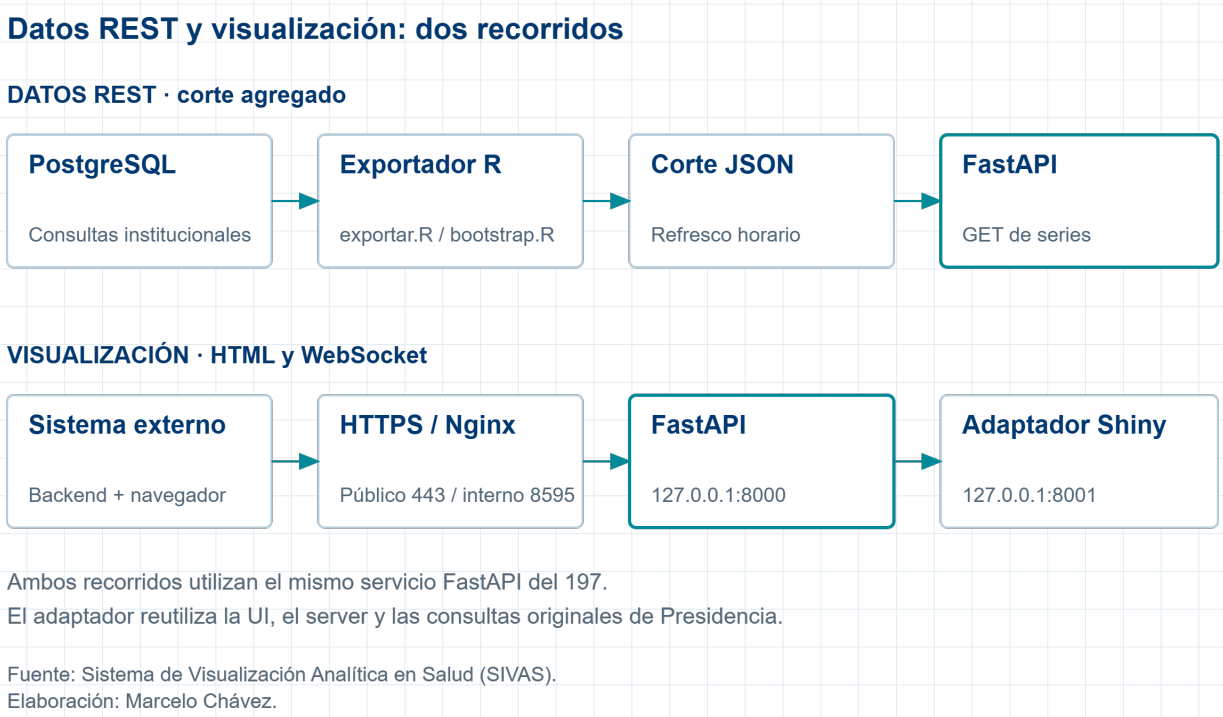


Figura 9.1: Recorrido del corte REST y del visualizador institucional.

La Figura 9.1 representa los dos usos del mismo servicio FastAPI. Las dos cajas con ese nombre no corresponden a dos procesos. El detalle del proxy y la autenticación se desarrolla en Figura 1.1, Figura 3.1 y Figura 3.2.

9.2. Archivo, responsabilidad y conexión siguiente

Las rutas de la tabla son relativas a la raíz `health_analytics_msp`. Cada archivo existe en la revisión local comprobada; los bloques de código de este manual se incluyen desde sus fuentes mediante `incluir_codigo.lua`.

Archivo	Qué hace y con qué se conecta
`modulos/comprobacion_api/TEST_API.R`	Lee la configuración privada, carga `cliente.R` y muestra un botón. Su `renderUI()` solicita la sesión y utiliza `sesion$url` como `src` del iframe.
`deploy/presidencia/examples/shiny_windows/cliente.R`	`presidencia_solicitar_sesion()` llama a `presidencia_http_cliente()`, hace el POST HTTPS y valida las cinco URLs recibidas. Es el enlace real de R con FastAPI.
`deploy/presidencia/linux/nginx-rutas.conf`	Declara las rutas que Nginx dirige a la integración. El resto del sitio continúa con su enrutamiento propio.
`deploy/presidencia/linux/nginx-proxy.conf`	Envía esas solicitudes a `127.0.0.1:8000` y configura el transporte de WebSocket.
`deploy/presidencia/linux/servidor.py`	Inicia Uvicorn y la fábrica `create_app()` de `api.py`, con un trabajador en el puerto interno 8000.
`deploy/presidencia/api.py`	Define contratos Pydantic, endpoints, autorización, lectura del corte, creación de sesión, HTML del iframe y proxy HTTP/WebSocket.
`deploy/presidencia/auth.py`	Valida cliente API, cuenta institucional, contraseña, permisos, vigencia y sesiones. Su consulta parametrizada comprueba usuarios y asignaciones en el esquema `credenciales`.
`deploy/presidencia/bridge.R`	Verifica el usuario, módulo, tiempo, SID y firma HMAC que llegan al adaptador. No acepta una identidad declarada libremente por el navegador.
`deploy/presidencia/shiny_app.R`	Carga el bootstrap, los assets y la UI/server originales; exige firma válida antes de entregar el visualizador. Pasa `autenticacion_externa` al server de Presidencia.
`deploy/presidencia/bootstrap.R`	Abre conexiones de solo lectura y reutiliza consultas, normalización y metas del módulo. También define el exportador agregado.
`deploy/presidencia/exportar.R`	Ejecuta la generación del corte REST. Lo inicia el servicio y temporizador de exportación.
`modulos/indicadores_presidencia/`	Contiene `global.R`, UI, server y assets que proporcionan el dashboard institucional reutilizado.

9.3. Cómo pasa cada autenticación

El consumidor no envía las credenciales al iframe. El recorrido es el siguiente:

1. El **backend del sistema externo** lee su configuración privada. En el ejemplo, lo hace R; un backend en otra tecnología debe cumplir el mismo contrato HTTP.
2. Envía `POST/api/v1/presidencia/sesiones` al dominio público. Presenta `Authorization: Bearer`, `X-API-Usuario` y `X-API-Clave`; el cuerpo JSON contiene `usuario_sivas`, `clave_sivas` e `indicador`.
3. Nginx recibe la ruta publicada y la dirige a FastAPI. Nginx es el proxy de transporte; las comprobaciones de identidad descritas aquí están implementadas en Python.
4. `authorize()` comprueba Bearer y cliente API. `Gateway` y `PostgresIdentity` comprueban la cuenta institucional configurada. La consulta exige usuario activo y vigente, asignación activa a PRESIDENCIA y rol permitido. El password institucional se verifica contra su hash; el hash no se entrega al cliente.
5. El endpoint crea un SID temporal y devuelve la URL del módulo y sus cinco enlaces individuales. Una autenticación API válida por sí sola no habilita cualquier otro módulo del sistema.

6. El navegador coloca la URL recibida en el iframe. Las llamadas siguientes usan el SID; no repiten un formulario de login visible ni incluyen las claves en JavaScript.
7. Al acceder al visualizador, FastAPI revalida la sesión e incorpora cabeceras internas firmadas. `bridge.R` valida esa firma en el proceso Shiny. El adaptador habilita la identidad externa únicamente si la validación es correcta.
8. HTTP y WebSocket pasan por el proxy. Una sesión vencida o inválida debe rechazarse. La vigencia del SID no es la misma que la duración de la clave de integración.

El contrato de seguridad actual está especializado en PRESIDENCIA tanto en Python como en R. No debe retirarse para facilitar la integración de otro módulo.

9.4. Puertos, URLs y Swagger

Tramo	Dirección o puerto	Uso y comprobación
Sistema externo → dominio	HTTPS, puerto público 443	El consumidor usa `https://analiticadesalud.msp.gob.ec`, sin exponer puertos internos. Acceso HTTPS comprobado.
Publicación institucional → Nginx del 197	Puerto 8595	Escucha confirmada en el 197. La configuración completa del equipo que termina TLS no se inspeccionó en esta auditoría.
Nginx → FastAPI	`127.0.0.1:8000` en el 197	Escucha confirmada; proxy instalado dirigido a ese destino.
FastAPI → adaptador Shiny	`127.0.0.1:8001` en el 197	Escucha y servicio confirmados. El usuario externo no debe abrir este puerto.
Procesos autorizados → PostgreSQL	Conexión institucional del bootstrap y del autenticador	Configuración privada del servidor; no se necesita acceso PostgreSQL desde el navegador.
Ejemplo local `TEST_API.R`	`http://127.0.0.1:5010/`	Solo aloja el cliente de demostración en Windows; no es el servidor FastAPI.

Swagger está publicado en **https://analiticadesalud.msp.gob.ec/presidencia-api/docs** y su contrato OpenAPI en **https://analiticadesalud.msp.gob.ec/presidencia-api/openapi.json**. Ambos respondieron HTTP 200 en esta auditoría. Para el consumidor, Swagger sale por HTTPS/443; lo atiende FastAPI detrás del proxy, no un servicio público adicional en 8000. Las rutas raíz `/docs` y `/health/live` no son las rutas públicas de esta instalación: el prefijo publicado es `/presidencia-api/`.

9.5. Qué está instalado y qué versión se inspeccionó

Se consultó el 197 en modo de solo lectura. `sivas-presidencia-api.service` y `sivas-presidencia-shiny.service` estaban activos. Sus unidades se ubican en `/etc/systemd/system/`; las plantillas versionadas están en `deploy/presidencia/linux/`. El temporizador `sivas-presidencia-exportar.timer` estaba activo y la última ejecución del exportador tenía `Result=success`. Un servicio de exportación de tipo `oneshot` puede aparecer inactivo después de finalizar correctamente.

Se leyeron las rutas instaladas `/etc/nginx/sivas-presidencia-rutas.conf` y `/etc/nginx/sivas-presidencia-proxy.conf`: coinciden en el enrutamiento descrito. Se verificaron los sockets de escucha; no se cambió configuración de Nginx ni se reinició ningún proceso.

La comparación actual de diez archivos encontró:

1. `api.py`, `auth.py`, `bootstrap.R`, `bridge.R`, `linux/servidor.py` y el `global.R` de Presidencia son idénticos byte a byte entre Windows y el 197.

2. La UI y el server de Presidencia coinciden en contenido después de normalizar saltos de línea.
3. `shiny_app.R` tiene una diferencia funcional de ruta: el servidor publicado lee `modulos/dneaisns/roles_usuarios/roles_usuarios_global.R`; Windows lee `modulos/roles_usuarios/roles_usuarios_global.R`. Esta diferencia procede de la reorganización local del módulo. No se sincronizó durante la auditoría. Los fragmentos incluidos en este manual muestran el archivo local, por lo que una futura publicación debe trasladar coordinadamente sus dependencias.

El nombre exacto es `shiny_app.R`. El adaptador lee el archivo de Roles y Usuarios durante su arranque para extraer funciones permitidas, antes de atender la UI; una ruta ausente puede impedir iniciar el adaptador aunque el acceso externo ya use FastAPI. Por eso no debe sustituirse automáticamente solo una referencia: un futuro cambio requiere revisar la carpeta y sus dependencias en un clon, probar y publicar de forma coordinada. No se aplicó esa operación en el 197.

El manual muestra la implementación existente y su procedimiento de uso; no sustituye un inventario de versiones inmutable. Antes de otra publicación se debe repetir la comparación y registrar la revisión desplegada.

9.6. Evidencia vigente y alcance de las pruebas

El registro depurado `recursos/evidencia_20261026.json` y su anexo dentro de este PDF contienen casos con equipo, fecha/hora disponible, procedimiento, esperado, observado, estado y referencia al soporte. Se conservaron los intentos fallidos y las limitaciones de los instrumentos; no se equipara HTTP 200 con funcionamiento integral.

Se demostraron cinco recorridos autorizados de REST y visualización: iframe sin segundo login, WebSocket con mensajes de ida/vuelta, gráficos, KPIs, tablas, controles y archivos XLSX. Los cinco libros se abrieron en Microsoft Excel 16.0 sin guardar. El cierre REST y el rechazo de reutilización se observaron por cada indicador; el clic Salir se comprobó en el primer indicador.

Se compararon REST, valores no nulos del gráfico y Excel: 6, 6, 9, 9 y 3 filas respectivamente, con tolerancia de 0.011 puntos porcentuales por redondeo. El corte REST muestreado fue generado el 26/10/2026 a las 11:07:27 UTC. Esta concordancia puntual no garantiza que dos lecturas futuras coincidan durante el intervalo entre exportaciones ni certifica todos los cálculos de negocio de manera independiente.

Persisten el problema de PDF y los ensayos no realizados que se detallan a continuación. La expiración de 28800 segundos fue leída como configuración y probada con reloj simulado; no se mantuvo una sesión real durante ocho horas.

9.7. Investigación de esperas agotadas y fichas PDF

Se separaron cuatro observaciones:

1. **Contexto del navegador:** el instrumento anterior inspeccionaba marcos envolventes y no acreditaba el DOM del Shiny remoto. Con Playwright se identificaron tres marcos y se observaron los cinco gráficos. La causa del `Page.navigate` original agotado no quedó determinada solamente con ese cambio.
2. **Contadores de la herramienta:** se esperaba una clave sin el sufijo de tipo que usa Shiny. La condición permanecía falsa con el WebSocket conectado. Al leer el contador tipado, las cinco consultas registraron contador 1 y respuestas del servidor. Se corrigió únicamente el instrumento, no el aplicativo.
3. **Descargas:** una prueba anterior no dejó evidencia del href exacto antes de pulsar. La nueva prueba esperó el binding habilitado y completó los cinco XLSX. Esto mejora la prueba, pero no demuestra que el momento del clic haya sido la causa única de todos los fallos previos. Hubo además esperas de lectura de PDF agotadas después de recibir cabeceras HTTP 200; su causa de transporte no se determinó.
4. **PDF embebido:** en primer nivel y paquete de niños el navegador registró `ERR_BLOCKED_BY_RESPONSE` y violación de `frame-ancestors 'self'`. Las cuatro rutas estáticas respondieron PDF completo con `X-Frame-Options: SAMEORIGIN` y esa CSP. El código de la vista usa una ruta absoluta `/fichas/`, que no pasa por la ruta protegida del gateway. Los ancestros incluyen el sistema externo, por lo que esas respuestas no autorizan esa cadena de iframe. Salud mental y gestantes no quedaron concluidos en la prueba de vista previa; no se extrapola un resultado observado a todas las funciones.

Impacto demostrado: la ficha no se visualiza en los dos casos acreditados dentro del cliente externo. La descarga desde el botón de Shiny sigue pendiente de una secuencia completa; no se deduce su funcionamiento

del GET estático. El flujo nacional de gráficos/datos tiene evidencia independiente aprobada.

Corrección propuesta, no aplicada: en un clon de pruebas, revisar la construcción de `src` en `output$ficha_pdf` para usar un recurso relativo al adaptador bajo la URL de sesión; comprobar que la lista permitida de fichas y la validación de SID se mantengan. Ensayar vista previa, Range, botón de descarga y cierre en los cuatro indicadores. No relajar globalmente CSP/X-Frame-Options ni modificar Nginx como solución automática. Esta propuesta no está desplegada.

Conectividad: hubo intentos SSH agotados antes de lecturas posteriores exitosas. No se determinó su causa ni se atribuyó una caída del aplicativo. Las versiones y hashes actuales se apoyan en la captura exitosa posterior.

9.8. Cómo reproducir la revisión sin revelar secretos

1. Abrir la raíz del repositorio y activar el entorno que tiene las dependencias. El entorno local `msp_01` no tiene instalado pytest en la revisión comprobada; las 32 pruebas se ejecutaron con el `.venv` existente de la integración. En Linux, la unidad versionada apunta a `/home/marcelo.chavez/.sivas-presidencia/venv/bin/python`. No se deben confundir esos tres intérpretes.
2. Ejecutar las pruebas Python, cliente R y puente indicadas en la tabla anterior. No usar contraseñas reales en casos negativos repetitivos contra producción.
3. Abrir Swagger y revisar el POST de sesiones, los encabezados de seguridad y los modelos de solicitud/respuesta. Introducir secretos solo en un equipo autorizado y no publicar capturas de sus valores.
4. Ejecutar `TEST_API.R` con el YAML privado configurado y pulsar Mostrar dashboard. Comprobar visualmente gráficos y controles, no solo la aparición del iframe.
5. Un administrador puede comprobar en el 197, sin reiniciar: `systemctl is-active sivas-presidencia-api sivas-presidencia-shiny sivas-presidencia-exportar.timer`, `ss -ltn` y las rutas de Nginx citadas. No copiar el contenido de `/etc/sivas-presidencia/environment` al informe.
6. Comparar versiones antes de atribuir al servidor un cambio realizado únicamente en Windows. Registrar fecha, revisión y resultado, excluyendo URLs con SID y credenciales.

9.9. Adaptación razonada a otro módulo

Este procedimiento es una **guía de desarrollo**, no una funcionalidad ya instalada para todos los módulos. Copiar la carpeta o cambiar una URL no habilita automáticamente otro dashboard. La arquitectura es reutilizable; el contrato, datos, identidad y adaptador deben revisarse para cada módulo.

Paso	Qué adaptar	Punto real de partida
1. Delimitar acceso	Elegir módulo, indicadores y datos autorizados; decidir si son agregados o nominales. No trasladar permisos de Presidencia.	Catálogo institucional y reglas del módulo destino.
2. Revisar el módulo Shiny	Identificar su global, UI, server, helpers, recursos y conexión; comprobar cómo admite una identidad externa verificada.	Patrón de carga de `shiny_app.R` y parámetro `autenticacion_externa` del server actual.
3. Definir el contrato REST	Adaptar IDs, campos, unidades, rangos de año y modelos. Hoy `IndicatorId`, `Corte` y `Catalogo` restringen PRESIDENCIA; las series restringen el año 2026.	`api.py`: modelos, `IDS`, `LABELS`, prefijo y `series()`.
4. Preparar datos	Reutilizar las consultas del módulo destino, construir su corte y verificar actualidad, tamaño y ausencia de datos no autorizados.	`bootstrap.R`, `exportar.R`, `PRESIDENCIA_SNAPSHOT` y `read_snapshot()`.
5. Autorizar la identidad	Validar la asignación y rol del módulo nuevo con SQL parametrizado; definir cuenta y privilegios propios.	`auth.py::SQL_IDENTITY`, hoy fijado a PRESIDENCIA.

Paso	Qué adaptar	Punto real de partida
6. Adaptar el puente	Firmar y verificar el identificador del módulo correcto; no admitir cualquier texto enviado por el navegador.	`Gateway` y `bridge.R`, hoy fijados a PRESIDENCIA.
7. Preparar el adaptador	Cargar solo el módulo nuevo, sus recursos autorizados y su UI original. No eliminar el login del aplicativo convencional.	`shiny_app.R`, archivos del módulo destino y su mecanismo de autenticación.
8. Publicar rutas y servicios	Elegir rutas sin colisión, puertos internos disponibles y servicios propios; conservar la publicación HTTPS. Si se requiere otro proceso, planificarlo expresamente.	Plantillas systemd, `servidor.py` y los dos includes de Nginx.
9. Ajustar cliente y pruebas	Cambiar contrato y rutas del cliente, no solo el título. Probar autenticación, denegación entre módulos, SID, datos, WebSocket, expiración e iframe en el origen autorizado.	`cliente.R`, `TEST_API.R`, pytest y prueba de puente.
10. Documentar y publicar	Registrar archivos, dependencias, migraciones si existen, versión, evidencia y retorno a la versión anterior; publicar después de pruebas autorizadas.	Este manual y la documentación del módulo destino.

9.9.1. Dependencias y aislamiento que se deben revisar

1. Inventariar paquetes R y recursos reales del módulo destino; no asumir que cargar `global.R` de toda la aplicación sea necesario o seguro. En Presidencia, el bootstrap carga solo lo requerido y las conexiones de lectura; el adaptador incorpora los assets y UI/server originales.
2. Declarar un contrato de datos propio. Los modelos actuales fijan PRESIDENCIA, cinco IDs, unidad Porcentaje y filtro de año 2026. Un módulo nominal requiere una evaluación específica de campos y alcance; no copiar automáticamente el corte agregado de este ejemplo.
3. Revisar `Gateway.headers()`, `SQL_IDENTITY` y `bridge.R` de manera coordinada: el módulo autorizado, la cuenta y el contenido firmado deben coincidir. Cambiar una etiqueta de URL no otorga permisos. Mantener denegación cruzada entre módulos en fixtures controlados.
4. Separar credenciales cliente, cuenta institucional y clave de firma. Provisionar por un canal autorizado; no crear personas ficticias ni reutilizar el token de Presidencia por conveniencia.
5. Decidir si el nuevo módulo reutiliza un proceso o necesita un adaptador propio. La presente implementación no es un registro genérico de módulos y no se afirma que soporte el nuevo módulo sin cambios. Puertos, recursos PDF y rutas no deben colisionar.
6. Llevar los cambios a un clon de pruebas y ejecutar contrato, auth, datos, firma, iframe, WebSocket, controles, descargas, expiración y cierre. Solo después corresponde planificar una publicación autorizada, con revisión/hash y retorno. Nada de esta guía constituye evidencia de otra integración ya desplegada.

El resultado de esa adaptación debe demostrarse nuevamente. Que Presidencia tenga un POST operativo no acredita automáticamente que otro módulo pueda utilizarlo sin cambios.

10 Editar y regenerar este manual, solo en PDF

10.1. Archivo principal del manual

El documento fuente único es:

```
docs/manuales/api_presidencia/Manual_integral_API_Presidencia.qmd
```

`_quarto.yml` declara únicamente PDF y limita el renderizado a ese documento. El motor es XeLaTeX. El texto utiliza Latin Modern, una tipografía de la familia visual de LaTeX, y los recursos de portada, encabezados, pies y cuadrícula están en `latex/`. Quarto documenta esta forma de generar PDF mediante LaTeX (Posit, s. f.-a).

10.2. Inclusión y mantenimiento de las explicaciones de código

Los bloques con `fuente=` son procesados por `incluir_codigo.lua`. El filtro solo admite una lista de archivos de código públicos. Lee el archivo real, selecciona el rango solicitado e imprime números de línea. Nunca lee `.env`, credenciales JSON o el YAML privado.

Para cambiar una explicación, editar este QMD. Para cambiar el código de la aplicación, editar su fuente y luego revisar las explicaciones y rangos afectados. Incluir código actual no actualiza automáticamente la prosa: la numeración explicada corresponde a la revisión documentada.

Los ejemplos se muestran como código, con evaluación desactivada. Renderizar el manual no realiza POST, no consulta PostgreSQL y no arranca servicios.

10.3. Repetir las comprobaciones de esta revisión

Estos procedimientos se ejecutan desde la raíz de Windows y usan servicios ya existentes. No realizan despliegue ni configuración en el 197. Mantener el YAML privado fuera de Git. Usar una sesión de prueba autorizada y no ejecutar ensayos negativos con una cuenta real.

1. En una consola R, iniciar el cliente real con el navegador automático desactivado; cerrar antes otra instancia que ocupe 5010:

```
Sys.setlocale("LC_CTYPE", "Spanish_Ecuador.utf8")
app <- source("modulos/comprobacion_api/TEST_API.R", encoding = "UTF-8")$value
shiny::runApp(app, host = "127.0.0.1", port = 5010, launch.browser = FALSE)
```

2. En otra consola, comprobar HTTP y sesiones. El script lee la configuración privada pero solo guarda estados, tiempos y datos agregados de prueba en `tmp/`:

```
& 'C:/Program Files/R/R-4.6.0/bin/x64/Rscript.exe' docs/manuales/api_presidencia/recursos/validar_http_lectura.R
```

3. La revisión utilizó Node y Playwright ya disponibles en el runtime local, con Google Chrome instalado. Son dependencias del arnés de pruebas, no del servicio FastAPI. En otro equipo ajustar las rutas a herramientas autorizadas; no instalar nada en el 197 para usar este arnés:

```
$node = 'C:/Users/MARCELO/.cache/codex-runtimes/codex-primary-runtime/dependencies/node/bin/node.exe'
$env:SIVAS_PLAYWRIGHT_MODULE =
 ↪  'C:/Users/MARCELO/.cache/codex-runtimes/codex-primary-runtime/dependencies/node/node_modules/playwright'
& $node docs/manuales/api_presidencia/recursos/validar_navegador.cjs
& $node docs/manuales/api_presidencia/recursos/validar_controles.cjs
```

4. Fichas, en una prueba independiente para no contaminar el recorrido principal. La lectura directa no sustituye probar la vista previa y el botón:

```
& 'C:/Program Files/R/R-4.6.0/bin/x64/Rscript.exe'
 ↪  docs/manuales/api_presidencia/recursos/validar_fichas_lectura.R
& $node docs/manuales/api_presidencia/recursos/validar_fichas_navegador.cjs
```

5. Validar el contenido descargado con el script de lectura XML/PDF. Requiere Python y, para contar páginas PDF, `pypdf` en ese intérprete. En esta revisión se usó el Python de herramientas documentales existente; no se instaló ni modificó el venv del servicio:

```
& 'C:/Users/MARCELO/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/python.exe'
 ↪  docs/manuales/api_presidencia/recursos/validar_archivos_descargados.py
```

6. Contrastar los nuevos registros con las pruebas sintéticas y la inspección remota de lectura. No reutilizar fechas anteriores como si fueran una ejecución nueva. `actualizar_evidencia.py` depura registros ya obtenidos y construye el anexo; no vuelve a probar el servicio. El PDF se regenera a partir del QMD guardado sin ejecutar las pruebas ni necesitar sus descargas temporales.

Los scripts redactan SID, identificadores de sesión Shiny, cookies y cabeceras sensibles. Revisar siempre un registro antes de compartirlo. Al terminar, cerrar las sesiones de prueba y detener únicamente el cliente local creado para la prueba, nunca los servicios del 197. Algunos intentos PDF no confirmaron su cierre por timeout; no se afirma que su limpieza haya sido verificada. El vencimiento absoluto configurado sigue siendo el límite implementado.

10.4. Recursos gráficos y referencias

Los diagramas se elaboraron siguiendo las pautas de Diagram Design, con identidad azul/turquesa basada en SIVAS. Sus fuentes HTML y SVG están en `diagramas/`; los PNG tienen canal alfa y alta resolución. Se incluyen como figuras Quarto con identificadores, referencias cruzadas e índice de figuras. No son páginas HTML alternativas del manual.

Los logos son copias de los recursos institucionales existentes. La cuadrícula se dibuja con TikZ al generar cada página, no como una imagen de baja resolución. Los datos de autoría y contacto se centralizan en `latex/creditos.tex`.

Las referencias se guardan en `bibliography.bib` y se formatean con el estilo APA de Citation Style Language incluido en `recursos/apa.csl`. No se crearon DOI, fechas o enlaces de publicaciones inexistentes. Las referencias sin fecha de publicación usan la representación sin fecha del estilo.

10.5. Renderizado a PDF

Requisitos: Quarto, XeLaTeX/TinyTeX y las fuentes LaTeX. No se requiere ejecutar Python o R para renderizar este documento. Los códigos fuente del repositorio deben estar presentes para el filtro de inclusión.

Desde la raíz del proyecto Windows:

```
& 'C:/Program Files/Quarto/bin/quarto.exe' render
 ↪  .\docs\manuales\api_presidencia\Manual_integral_API_Presidencia.qmd --to pdf
```

O ejecutar el script de la carpeta:

```
& .\docs\manuales\api_presidencia\renderizar_pdf.ps1
```

En Positron, abrir el QMD y pulsar Render. Al reconocer el proyecto Quarto local, la salida predeterminada es PDF. No se declara un formato HTML o Word para este manual.

El resultado es `Manual_integral_API_Presidencia.pdf` en la misma carpeta. Si falta un paquete LaTeX, la instalación de TinyTeX debe poder resolverlo; en una máquina sin acceso a repositorios se deben aprovisionar previamente los paquetes requeridos. En el entorno de esta tarea Quarto y TinyTeX ya estaban instalados.

11 Anexo de evidencia verificable

Este anexo incorpora los resultados del registro depurado `recursos/evidencia_20261026.json`. Cada caso identifica su alcance; «aprobado» solo acredita lo esperado en ese caso. Una prueba sintética no certifica producción y una respuesta HTML no certifica el dashboard completo.

11.1. Entornos y trazabilidad temporal

Las fechas de las nuevas comprobaciones se registraron en UTC y se muestran también en America/Guayaquil (UTC−05:00). JUnit conservó fecha/hora con desplazamiento -05:00, convertida a UTC; el antecedente anterior solo tenía una fecha y no se reconstruyó su hora.

Equipo / intérprete	Versión observada	Uso
Windows .venv Presidencia	Python 3.12.14	32 pruebas sintéticas de API.
Windows Conda msp_01	Python 3.14.8	Herramientas locales; no se atribuye a él pytest.
197 venv systemd	Python 3.12.15	Servicio FastAPI instalado.
Windows R / Linux R	4.6.0 / 4.5.1	Consumidor local / adaptador publicado.
Navegador / Excel	Chrome 155.0.8059.86 / Excel 16.0	Iframe, controles y lectura de descargas.
Nginx del 197	1.20.1	Leído mediante nginx -v, sin recarga.

11.1.1. Paquetes observados

Paquete	Windows	197
shiny	1.14.0	1.11.1
highcharter	0.9.5	0.9.4
gt	1.3.0	1.1.0
DBI	1.3.0	1.2.3
RPostgres	1.4.10	1.4.8
openxlsx	4.2.8.1	4.2.8.1
digest	0.6.39	0.6.37
fastapi	0.159.1	0.159.1
uvicorn	0.54.0	0.54.0
pydantic	2.14.1	2.14.1
starlette	1.9.0	1.9.0
httpx	0.28.1	0.28.1
psycopg	3.3.6	3.3.6
websockets	16.1.1	16.1.1

Las consultas R no encontraron el paquete R sodium en esos intérpretes; la API verifica formatos institucionales mediante PyNaCl/Argon2 en Python. El helper R de la prueba usa el método disponible; su mensaje genérico no demuestra qué paquete se instaló. Los avisos de locale y de versiones compiladas se conservaron como avisos, no como fallos funcionales.

11.2. Matriz funcional de los cinco indicadores

A = aprobado en el alcance indicado; F = fallido observado; P = pendiente/no concluido; NA = no aplicable. «PDF» distingue vista y botón; la descarga estática de los cuatro archivos es un caso separado.

Indicador	Auth/REST	Iframe/WSS	Gráfico, KPI y tabla
Primer nivel	A	A	A
Salud mental	A	A	A
Paquete niños	A	A	A
Paquete gestantes	A	A	A
Medicamentos	A	A	A

Indicador	Controles	XLSX	Cierre REST
Primer nivel	A	A	A
Salud mental	A	A	A
Paquete niños	A	A	A
Paquete gestantes	A	A	A
Medicamentos	A	A	A

Indicador	Vista previa PDF	Botón PDF
Primer nivel	F	P
Salud mental	P	P
Paquete niños	F	P
Paquete gestantes	P	P
Medicamentos	NA	NA

11.2.1. Corte muestreado y coherencia de salidas

La generación reciente de un JSON no vuelve recientes los datos de sus fuentes. Los datos muestreados de REST, gráfico y Excel coincidieron dentro del redondeo; sus fechas máximas son distintas entre indicadores. Esta comparación no garantiza coincidencia futura entre dos exportaciones.

Indicador	Filas reales	Último mes de datos REST	Comparación puntual
Primer nivel	6	2026-06-01	aprobado
Salud mental	6	2026-06-01	aprobado
Paquete niños	9	2026-09-01	aprobado
Paquete gestantes	9	2026-09-01	aprobado
Medicamentos	3	2026-08-01	aprobado

Los puntos nulos de meses todavía sin valor no se cuentan como datos observados. El caso R08 comprueba transferencia/concordancia; la actualización de cada fuente y la auditoría independiente de sus cálculos permanecen fuera de esa aprobación.

El botón Salir fue comprobado en el primer indicador; el cierre REST y el rechazo de reutilización se comprobaron para cada uno. No se extiende el clic de Salir a cinco instancias sin haberlo repetido. Direcciones provinciales muestra el panel En construcción: abrirlo es verificable; filtrado geográfico no es una función operativa de este módulo.

11.3. Casos: procedimiento, esperado y observado

11.3.1. Pruebas sintéticas e identidades controladas

S01 — Autenticación, contrato, expiración y revocación simuladas

- **Fecha/hora:** 26/10/2026 06:45:12 -0500; UTC 11:45:12
- **Equipo/entorno:** Windows; .venv de Presidencia; TestClient; identidad PRUEBA, sin PostgreSQL real

- **Procedimiento:** `deploy/presidencia/.venv/Scripts/python.exe` -m pytest `deploy/presidencia/tests/test_api.py` -q –junitxml=tmp/auditoria_api_20261026/pytest.xml
- **Esperado:** Casos válidos aceptados; credenciales, firma o sesión inválidas rechazadas en fixtures.
- **Observado:** 32 aprobadas; 0 fallos; 0 errores. Incluye reloj simulado; no son ocho horas reales. JUnit incluye fecha/hora con desplazamiento -05:00; se conserva su duración y se convierte su inicio a UTC.
- **Estado:** aprobado. **Registro:** `recursos/registros/pytest_resumen.json`.

S02 — Cliente R con respuestas controladas

- **Fecha/hora:** 26/10/2026 06:51:55 -0500; UTC 11:51:55 → 26/10/2026 06:51:56 -0500; UTC 11:51:56
- **Equipo/entorno:** Windows; R 4.6.0; datos/identidades generados localmente
- **Procedimiento:** Rscript `deploy/presidencia/examples/shiny_windows/validar_cliente.R`
- **Esperado:** Validaciones sin invocar cuentas productivas ni escribir datos institucionales.
- **Observado:** [1] “Spanish_Ecuador.utf8” Cliente R/Shiny: URLs seguras, contrato de cinco indicadores, iframe, selección sin renovar, expiración y renovación manual correctos. Son respuestas controladas, no la sesión productiva.
- **Estado:** aprobado. **Registro:** `recursos/registros/local_y_ssh.json`.

S03 — Puente firmado y password local de prueba

- **Fecha/hora:** 26/10/2026 06:51:56 -0500; UTC 11:51:56 → 26/10/2026 06:51:56 -0500; UTC 11:51:56
- **Equipo/entorno:** Windows; R 4.6.0; datos/identidades generados localmente
- **Procedimiento:** Rscript `deploy/presidencia/tests/validar_puente.R`
- **Esperado:** Validaciones sin invocar cuentas productivas ni escribir datos institucionales.
- **Observado:** [1] “Spanish_Ecuador.utf8” Puente firmado, firma alterada, firma vencida, acceso anónimo y contraseña sodium: correctos. El mensaje genérico «sodium» no acredita la instalación de ese paquete R; se comprobó el helper disponible.
- **Estado:** aprobado. **Registro:** `recursos/registros/local_y_ssh.json`.

11.3.2. Comprobaciones locales

L01 — Intérpretes y Git del equipo Windows

- **Fecha/hora:** 26/10/2026 06:51:55 -0500; UTC 11:51:55
- **Equipo/entorno:** Windows; intérpretes existentes
- **Procedimiento:** sys.version, sys.executable, packageVersion(), git rev-parse HEAD y recuento de cambios rastreados
- **Esperado:** Distinguir Conda, .venv de pruebas y revisión de Git.
- **Observado:** .venv Python 3.12.14; Conda msp_01 Python 3.14.8; R 4.6.0. HEAD `e3c2e7094020ba6d49a88900b582d5e52c0e982c`; 39 archivos rastreados modificados. HEAD no describe por sí solo todo el árbol de trabajo.
- **Estado:** aprobado. **Registro:** `recursos/registros/local_y_ssh.json`.

L02 — Lectura de las descargas con Microsoft Excel

- **Fecha/hora:** 26/10/2026 07:04:02 -0500; UTC 12:04:02 → 26/10/2026 07:04:11 -0500; UTC 12:04:11
- **Equipo/entorno:** Windows; Excel 16.0
- **Procedimiento:** Abrir los cinco libros con UpdateLinks=0, ReadOnly=True, macros y eventos deshabilitados; cerrar sin guardar.
- **Esperado:** Cinco libros legibles, sin modificar los archivos ni datos.
- **Observado:** Cinco libros abiertos en solo lectura; filas incluyendo encabezado [7, 7, 10, 10, 4], columnas [7, 7, 7, 7, 4]. No se guardaron cambios; el lector XML y CRC contrastó el contenido.
- **Estado:** aprobado. **Registro:** `recursos/registros/excel_lector.json`.

D01 — Investigación del tiempo agotado del navegador

- **Fecha/hora:** 26/10/2026 06:50:34 -0500; UTC 11:50:34 → 26/10/2026 07:07:56 -0500; UTC 12:07:56
- **Equipo/entorno:** Chromote y Playwright/Chrome en Windows
- **Procedimiento:** Revisar marcos aislados, claves tipadas de Shiny, href de descarga y errores de red/consola.
- **Esperado:** Distinguir herramienta, red y servicio sin asignar una causa no demostrada.

- **Observado:** Se identificaron inspección de marco incorrecto y contador sin sufijo de tipo en instrumentos anteriores; corrigiendo solo los tests se observaron los cinco gráficos/controles. El Page.navigate original y los timeouts de lectura de cuerpos PDF no tienen causa raíz acreditada. Algunos timeouts PDF recibieron antes HTTP 200.
- **Estado:** aprobado. **Registro:** `recursos/registros/controles_instrumento_v1.json`.

11.3.3. Comprobaciones contra el 197 y su publicación

R01 — Salud, Swagger y protección REST

- **Fecha/hora:** 26/10/2026 07:09:36 -0500; UTC 12:09:36 → 26/10/2026 07:09:37 -0500; UTC 12:09:37
- **Equipo/entorno:** Windows → HTTPS público del 197
- **Procedimiento:** GET de live, ready, shiny, docs, openapi; GET del catálogo sin credenciales.
- **Esperado:** HTTP 200 en salud/documentación y 401 en catálogo anónimo.
- **Observado:** live/ready/shiny/docs/openapi respondieron HTTP 200; catálogo sin credenciales respondió 401.
- **Estado:** aprobado. **Registro:** `recursos/registros/salud_publica.json`.

R02 — Servicios, puertos y fuentes en disco

- **Fecha/hora:** 26/10/2026 06:52:46 -0500; UTC 11:52:46
- **Equipo/entorno:** SSH de lectura; 10.64.100.197
- **Procedimiento:** systemctl show/is-active, ss -ltn/ss -tn, lectura de includes Nginx y SHA256 de diez fuentes.
- **Esperado:** Servicios activos; 8000/8001 en loopback; 8595 escuchando; comparación sin sincronizar.
- **Observado:** API y Shiny activos; temporizador activo y última exportación success. Conexiones a 10.64.100.191:5432. Diez archivos comparados: siete idénticos, dos equivalentes LF y un cambio de ruta. No se inspeccionó el equipo que termina TLS.
- **Estado:** aprobado. **Registro:** `recursos/registros/servidor_lectura.json`.

R03 — Versiones y valores de configuración no secretos

- **Fecha/hora:** 26/10/2026 06:52:46 -0500; UTC 11:52:46 → 26/10/2026 07:18:59 -0500; UTC 12:18:59
- **Equipo/entorno:** SSH 197; venv de systemd y /usr/bin/Rscript
- **Procedimiento:** Consultar versiones y solo PGHOST/PGPORT, TTL, corte y rutas permitidas del entorno de los procesos.
- **Esperado:** No extraer passwords, tokens ni clave de firma. Confirmar intérpretes y límites configurados.
- **Observado:** Python 3.12.15; FastAPI 0.159.1; Uvicorn 0.54.0; R 4.5.1; Nginx 1.20.1. TTL 28800 s y corte máximo 24 h. Los límites configurados no son duraciones ensayadas.
- **Estado:** aprobado. **Registro:** `recursos/registros/runtime_no_secreto.json`. Soportes adicionales: `recursos/registros/servidor_lectura.json`, `recursos/registros/versiones_R.json`, `recursos/registros/componentes_197.json`.

R09 — Dependencia de ruta y archivo principal de Nginx

- **Fecha/hora:** 26/10/2026 07:35:03 -0500; UTC 12:35:03
- **Equipo/entorno:** SSH 197; solo lectura
- **Procedimiento:** Comprobar existencia de ambas rutas de Roles y Usuarios y leer únicamente líneas de escucha/include.
- **Esperado:** Acreditar la diferencia sin copiar ni renombrar archivos.
- **Observado:** En el 197 existe la ruta antigua de Roles y Usuarios y no la nueva; Nginx escucha en 8595 e incluye las rutas de Presidencia. No se cambió ningún archivo.
- **Estado:** aprobado. **Registro:** `recursos/registros/dependencias_ruta.json`.

H-primer-nivel — HTTP autorizado y cierre: primer-nivel

- **Fecha/hora:** 26/10/2026 06:43:36 -0500; UTC 11:43:36 → 26/10/2026 06:43:37 -0500; UTC 11:43:37
- **Equipo/entorno:** Windows/R → FastAPI público
- **Procedimiento:** validar_http_lectura.R: POST con cliente y cuenta institucional correctos, GET serie/HTML, POST cerrar y GET de reutilización.
- **Esperado:** Cinco enlaces; JSON y HTML correctos; cierre 200 y reutilización 401.

- **Observado:** Autenticación aprobado; REST 6 filas; HTML Shiny 200; cierre {‘estado’: ‘aprobado’, ‘respuesta’: ‘cerrada’, ‘reutilizacion’: 401}. No se ensayaron claves incorrectas reales.
- **Estado:** aprobado. **Registro:** `recursos/registros/http_real.json`.

H-salud-mental — HTTP autorizado y cierre: salud-mental

- **Fecha/hora:** 26/10/2026 06:43:37 -0500; UTC 11:43:37 → 26/10/2026 06:43:38 -0500; UTC 11:43:38
- **Equipo/entorno:** Windows/R → FastAPI público
- **Procedimiento:** validar_http_lectura.R: POST con cliente y cuenta institucional correctos, GET serie/HTML, POST cerrar y GET de reutilización.
- **Esperado:** Cinco enlaces; JSON y HTML correctos; cierre 200 y reutilización 401.
- **Observado:** Autenticación aprobado; REST 6 filas; HTML Shiny 200; cierre {‘estado’: ‘aprobado’, ‘respuesta’: ‘cerrada’, ‘reutilizacion’: 401}. No se ensayaron claves incorrectas reales.
- **Estado:** aprobado. **Registro:** `recursos/registros/http_real.json`.

H-paquete-ninos — HTTP autorizado y cierre: paquete-ninos

- **Fecha/hora:** 26/10/2026 06:43:38 -0500; UTC 11:43:38 → 26/10/2026 06:43:39 -0500; UTC 11:43:39
- **Equipo/entorno:** Windows/R → FastAPI público
- **Procedimiento:** validar_http_lectura.R: POST con cliente y cuenta institucional correctos, GET serie/HTML, POST cerrar y GET de reutilización.
- **Esperado:** Cinco enlaces; JSON y HTML correctos; cierre 200 y reutilización 401.
- **Observado:** Autenticación aprobado; REST 9 filas; HTML Shiny 200; cierre {‘estado’: ‘aprobado’, ‘respuesta’: ‘cerrada’, ‘reutilizacion’: 401}. No se ensayaron claves incorrectas reales.
- **Estado:** aprobado. **Registro:** `recursos/registros/http_real.json`.

H-paquete-gestantes — HTTP autorizado y cierre: paquete-gestantes

- **Fecha/hora:** 26/10/2026 06:43:39 -0500; UTC 11:43:39 → 26/10/2026 06:43:44 -0500; UTC 11:43:44
- **Equipo/entorno:** Windows/R → FastAPI público
- **Procedimiento:** validar_http_lectura.R: POST con cliente y cuenta institucional correctos, GET serie/HTML, POST cerrar y GET de reutilización.
- **Esperado:** Cinco enlaces; JSON y HTML correctos; cierre 200 y reutilización 401.
- **Observado:** Autenticación aprobado; REST 9 filas; HTML Shiny 200; cierre {‘estado’: ‘aprobado’, ‘respuesta’: ‘cerrada’, ‘reutilizacion’: 401}. No se ensayaron claves incorrectas reales.
- **Estado:** aprobado. **Registro:** `recursos/registros/http_real.json`.

H-medicamentos — HTTP autorizado y cierre: medicamentos

- **Fecha/hora:** 26/10/2026 06:43:44 -0500; UTC 11:43:44 → 26/10/2026 06:43:57 -0500; UTC 11:43:57
- **Equipo/entorno:** Windows/R → FastAPI público
- **Procedimiento:** validar_http_lectura.R: POST con cliente y cuenta institucional correctos, GET serie/HTML, POST cerrar y GET de reutilización.
- **Esperado:** Cinco enlaces; JSON y HTML correctos; cierre 200 y reutilización 401.
- **Observado:** Autenticación aprobado; REST 3 filas; HTML Shiny 200; cierre {‘estado’: ‘aprobado’, ‘respuesta’: ‘cerrada’, ‘reutilizacion’: 401}. No se ensayaron claves incorrectas reales.
- **Estado:** aprobado. **Registro:** `recursos/registros/http_real.json`.

R04 — Iframe, WebSocket, gráficos, KPIs y tablas de los cinco indicadores

- **Fecha/hora:** 26/10/2026 06:50:34 -0500; UTC 11:50:34 → 26/10/2026 06:51:31 -0500; UTC 11:51:31
- **Equipo/entorno:** Windows; Chrome 155.0.8059.86; TEST_API.R en 5010
- **Procedimiento:** validar_navegador.cjs: botón real; iframe con cada enlace individual; inspección del marco Shiny y mensajes WSS.
- **Esperado:** Tres marcos; sin segundo login; WSS con mensajes de ida/vuelta; un gráfico, KPIs y filas por indicador.
- **Observado:** Cinco indicadores aprobados. Render de gráficos, tablas y KPIs no equivale a auditar todos sus cálculos estadísticos.
- **Estado:** aprobado. **Registro:** `recursos/registros/navegador_individual.json`.

R05 — Descarga Excel por el botón de cada indicador

- **Fecha/hora:** 26/10/2026 06:50:34 -0500; UTC 11:50:34 → 26/10/2026 06:51:31 -0500; UTC 11:51:31
- **Equipo/entorno:** Chrome/Windows; sesión autorizada

- **Procedimiento:** Esperar href /download/ habilitado; pulsar botón; guardar, verificar CRC/XML y abrir con Excel solo lectura.
- **Esperado:** Cinco descargas completas con filas y columnas válidas.
- **Observado:** Cinco descargas; filas de datos 6/6/9/9/3; columnas 7/7/7/7/4; archivos abiertos en Excel sin guardar.
- **Estado:** aprobado. **Registro:** `recursos/registros/navegador_individual.json`. Soportes adicionales: `recursos/registros/excel_lector.json`, `recursos/registros/archivos_y_cortes.json`.

R06 — Selectores y controles disponibles

- **Fecha/hora:** 26/10/2026 07:06:15 -0500; UTC 12:06:15 → 26/10/2026 07:07:56 -0500; UTC 12:07:56
- **Equipo/entorno:** Chrome/Windows; una vista por indicador
- **Procedimiento:** validar_controles.cjs: selector, Consultar, Cifras → Direcciones → Cifras, Refrescar. Contadores con sufijo de tipo Shiny.
- **Esperado:** Evento de control, respuesta WSS y efecto visual esperado en los cinco.
- **Observado:** Los cinco controles de cada indicador fueron aprobados. Direcciones muestra «En construcción»; no se acreditan filtros provinciales operativos.
- **Estado:** aprobado. **Registro:** `recursos/registros/controles.json`.

R07 — Cierre y rechazo de reutilización

- **Fecha/hora:** 26/10/2026 06:33:01 -0500; UTC 11:33:01 → 26/10/2026 06:51:31 -0500; UTC 11:51:31
- **Equipo/entorno:** Chrome/API público y R/API público
- **Procedimiento:** Cerrar la sesión de prueba y volver a solicitar su URL.
- **Esperado:** POST cerrar 200; GET anterior 401.
- **Observado:** Navegador: {'estado': 'aprobado', 'post': 200, 'reutilizacion': 401}. El botón Salir se observó aprobado en el primer indicador durante la prueba aislada; no se repitió dicho clic en todos. El cierre REST sí se comprobó por cada indicador.
- **Estado:** aprobado. **Registro:** `recursos/registros/navegador_sesiones_independientes.json`. Soportes adicionales: `recursos/registros/navegador_individual.json`, `recursos/registros/http_real.json`.

R08 — Concordancia de corte REST, gráfico y Excel

- **Fecha/hora:** 26/10/2026 07:13:07 -0500; UTC 12:13:07 → 26/10/2026 07:13:07 -0500; UTC 12:13:07
- **Equipo/entorno:** Datos agregados obtenidos del 197; comparación local
- **Procedimiento:** validar_archivos_descargados.py: fechas/meses y valores, tolerancia 0.011 puntos porcentuales por redondeo.
- **Esperado:** Coincidencia de los datos observados, excluyendo puntos nulos del gráfico.
- **Observado:** Cinco comparaciones aprobadas. Corte REST generado 2026-10-26T11:07:27Z; 6/6/9/9/3 filas. Medicamentos tiene siete posiciones en gráfico, pero solo tres valores no nulos. No garantiza igualdad futura entre exportaciones.
- **Estado:** aprobado. **Registro:** `recursos/registros/archivos_y_cortes.json`. Soportes adicionales: `recursos/registros/serie_primer-nivel.json`, `recursos/registros/serie_salud-mental.json`, `recursos/registros/serie_paquete-ninos.json` y otros 7 identificados en el JSON.

P01 — Lectura directa de los cuatro PDF metodológicos

- **Fecha/hora:** 26/10/2026 06:51:45 -0500; UTC 11:51:45 → 26/10/2026 06:52:12 -0500; UTC 11:52:12
- **Equipo/entorno:** HTTPS público; R/Windows; lector PDF local
- **Procedimiento:** GET de las cuatro rutas /fichas/; verificar cuerpo PDF y páginas.
- **Esperado:** HTTP 200 y PDF completo, sin interpretar esto como éxito del visor embebido.
- **Observado:** Cuatro respuestas application/pdf y archivos completos. Todas incluyen X-Frame-Options SAMEORIGIN y CSP frame-ancestors self.
- **Estado:** aprobado. **Registro:** `recursos/registros/fichas_publicas.json`. Soportes adicionales: `recursos/registros/archivos_y_cortes.json`.

P02 — Vista previa PDF desde el sistema externo

- **Fecha/hora:** 26/10/2026 07:11:12 -0500; UTC 12:11:12 → 26/10/2026 07:14:29 -0500; UTC 12:14:29
- **Equipo/entorno:** Chrome; tres niveles de iframe; origen externo 5010
- **Procedimiento:** Abrir Ficha; contrastar requestfailed/consola con cabeceras de /fichas/.
- **Esperado:** PDF visible dentro del iframe autorizado.

- **Observado:** Se observó ERR_BLOCKED_BY_RESPONSE y violación frame-ancestors self en primer nivel y paquete de niños. Salud mental y gestantes no quedaron concluidos en esa prueba. La ruta absoluta /fichas/ sale del gateway. No se cambió CSP ni Nginx.
- **Estado:** fallido. **Registro:** `recursos/registros/fichas_navegador.json`.

P03 — Botón Descargar ficha PDF

- **Fecha/hora:** 26/10/2026 07:11:12 -0500; UTC 12:11:12 → 26/10/2026 07:14:29 -0500; UTC 12:14:29
- **Equipo/entorno:** Chrome/Windows; sesión de prueba
- **Procedimiento:** Intento inicial de descarga; repetición aislada con espera de href.
- **Esperado:** Descarga completa desde el botón de Shiny, no solamente desde la URL estática.
- **Observado:** El primer evento se canceló; en la repetición hubo timeouts antes de completar la secuencia. No se certifica el botón. La lectura directa de los cuatro archivos sí se comprobó por separado.
- **Estado:** pendiente. **Registro:** `recursos/registros/fichas_navegador.json`.

D02 — Conectividad SSH intermitente durante la revisión

- **Fecha/hora:** 26/10/2026 06:51:55 -0500; UTC 11:51:55 → 26/10/2026 06:52:46 -0500; UTC 11:52:46
- **Equipo/entorno:** Windows → 10.64.100.197:22
- **Procedimiento:** Intentos acotados de SSH, seguidos de lectura exitosa.
- **Esperado:** No confundir fallo de conexión de la prueba con caída del aplicativo.
- **Observado:** La salida de herramienta mostró timeouts en intentos previos, sin log primario completo guardado por intento. La consulta final registrada y la captura remota posterior sí fueron exitosas. No se reconstruyen las horas faltantes ni se atribuye causa a la conexión.
- **Estado:** aprobado. **Registro:** `recursos/registros/local_y_ssh.json`.

11.3.4. Funciones no aplicables

N01 — Direcciones provinciales y PDF de medicamentos

- **Fecha/hora:** 2026-10-26; Hora no registrada; no se reconstruye.
- **Equipo/entorno:** Módulo original
- **Procedimiento:** Contrastar UI y server de Presidencia.
- **Esperado:** No afirmar que una función inexistente fue probada.
- **Observado:** Direcciones permanece En construcción. Medicamentos no habilita PDF metodológico. No hay filtro provincial/temporal operativo adicional a los controles inventariados.
- **Estado:** no aplicable. **Registro:** `recursos/registros/controles.json`.

11.3.5. Antecedentes históricos

H00 — Registro previo a esta revisión

- **Fecha/hora:** 2026-10-26; Hora no registrada; no se reconstruye.
- **Equipo/entorno:** evidencia_20261026.json anterior; hora no registrada
- **Procedimiento:** Conservar como antecedente; no reconstruir timestamps, comandos ni pruebas de Linux faltantes.
- **Esperado:** Separar declaraciones históricas de verificaciones nuevas.
- **Observado:** El registro inicial no contenía tiempos ni procedimiento completo por caso. Los resultados Linux históricos, duración y descargas no acreditadas no se convierten en aprobaciones actuales.
- **Estado:** pendiente. **Registro:** `recursos/registros/antecedente_previo.json`.

11.4. Revisión Git y huellas de las fuentes

Windows y el 197 reportaron HEAD `e3c2e7094020ba6d49a88900b582d5e52c0e982c`. Se observaron 39 cambios rastreados en Windows y 6 en el 197. No se sincronizó para hacer coincidir los árboles. Los hashes son de código público, no de contraseñas. El registro conserva huellas locales, remotas y normalizadas a LF.

Archivo	Comparación	SHA256 en disco del 197
deploy/presidencia/api.py	idéntico	4fe17682b6c165133f168cb6f74f02f3f659c0c2bc6eaa92d054c3b2fcf23c35
deploy/presidencia/auth.py	idéntico	ef13cabd737c58426f76eb427a3c3c53ce4adaf55f6db57c7bd2033a299eaa3d
deploy/presidencia/shiny_app.R	diferencia de ruta en el adaptador	6ef9d72228f16855b13b1c09c2d7ac3577aff4ce87242459819e9584cd2cf1a3
deploy/presidencia/bridge.R	idéntico	e7ab14a11f68fd973f541b4d6c3ffd0a8f43ee60f3f47346edd79c7107bdb4cf
deploy/presidencia/bootstrap.R	idéntico	5d8046889c34272f07201bae3c30dff1a37a86724de419b6d2e673e87835801d
deploy/presidencia/linux/servidor.py	idéntico	9f699ff0f8be3b02ce565e5c8c4eff4acdb9e916188b158ffefb0e6d110c66bd
modulos/indicadores_presidencia/global.R	idéntico	6c3debab02c69aabe0f1cf78a9f4d4705cf441b37b752635706b0434c5a92a7f
modulos/indicadores_presidencia/indicadores_presidencia_ui.R	equivalente al normalizar LF	15108cdd80ada9c937ab6c7462a23399888794e536c73c6f8917e1dd8961a5e1
modulos/indicadores_presidencia/indicadores_presidencia_server.R	equivalente al normalizar LF	3b44da2112d743f418d8147212248e163bda59c4ef23dd1ae4818e78047df613
modulos/indicadores_presidencia/indicadores_presidencia_assets.R	idéntico	1c5d2f13c73561246d5a24b027671ea210c486f894cde3bcd15ae4e90acc623d

SHA256 local del adaptador: `4f4c26e0a9bfe487caabc42ee8939c070ba643c0543e7e71d402d83184c6977c`. La diferencia de ruta se explica en la sección de versiones; no se aplicó corrección automática.

11.5. Pendientes y conclusión delimitada

- **Clic Salir en las otras cuatro vistas:** Se verificó el botón en primer nivel y el cierre REST en los cinco indicadores, no el mismo clic en cinco vistas.
- **Ocho horas reales:** 28800 s es configuración observada, no tiempo efectivamente transcurrido en prueba.
- **Inactividad y renovación con espera real:** El gateway implementa expiración absoluta; no se identificó un temporizador de inactividad propio en API/adaptador. El ciclo completo de espera real no se ejecutó.
- **Carga/concurrencia y capacidad:** No se ejecutaron k6 ni ensayos de saturación; 256 sesiones es límite de admisión, no capacidad certificada.
- **Otra red externa y otros navegadores:** Esta revisión usó el mismo equipo/red y Chrome; no acredita Firefox ni una segunda red.
- **Vista previa y botón PDF:** Vista bloqueada acreditada en dos casos; secuencia completa del botón no concluida; corrección propuesta no aplicada.
- **Menús de exportación propios de Highcharts y todos los gestos gráficos:** Se verificó render, datos y controles inventariados; no cada función de exportación/zoom.
- **Auditoría estadística independiente de todos los cálculos:** La concordancia de tres salidas no es una auditoría independiente de las reglas de negocio.
- **Equipo de terminación TLS:** HTTPS 443 está accesible; no se inspeccionó el terminador institucional.

Conclusión de la revisión: Se verificó el recorrido autorizado de cinco indicadores, REST, iframe, WebSocket,

render, controles, tablas, XLSX y cierre REST. Existe incidencia de PDF y permanecen límites explícitos; no se declara funcionamiento integral.

12 Anexos de código Python completo

12.1. Guía de api.py y correspondencia de líneas

En Python, `def` define una función, `async def` permite esperar operaciones de red sin bloquear esa espera y `class` define un modelo. `Depends()` declara una comprobación que FastAPI ejecuta antes de la ruta; `return` produce la respuesta. Los decoradores `@app.get()` y `@app.post()` enlazan las funciones con HTTP. Las líneas siguientes corresponden al código incluido debajo.

Líneas	Elemento	Función en la integración
30–40	`Punto`	Valida una observación mensual y sus componentes.
43–51	`Indicador`	Define los campos comunes de un indicador autorizado.
54–59	`Corte`	Define el archivo de datos autorizado que exporta R.
62–66	`EnlaceIndicador`	Describe las rutas de un indicador del catálogo.
69–72	`Catalogo`	Agrupa los cinco enlaces y la fecha del corte.
75–77	`Serie`	Amplía el indicador con el origen y la fecha de generación.
80–84	`SolicitudSesion`	Valida el cuerpo del POST y trata la contraseña como secreto.
87–366	`create_app`	Construye FastAPI, lee configuración y registra rutas. Sus funciones internas comparten el mismo gateway.
120–135	`authorize`	Comprueba token Bearer, cliente API y vigencia del usuario institucional.
137–143	`view`	Resuelve la capacidad temporal y traduce errores de identidad a estados HTTP.
146–149	`invalid_input`	Devuelve un 422 genérico sin repetir contraseñas enviadas en el cuerpo.
151–169	`read_snapshot`	Lee y valida el corte JSON; rechaza un corte ausente, inválido o vencido.
172–178	`headers`	Agrega políticas de seguridad a las respuestas HTTP.
181–182	`live`	Confirma que el proceso API responde.
185–187	`ready`	Comprueba que el corte está disponible y es válido.
190–199	`shiny_health`	Comprueba la respuesta HTTP del adaptador Shiny.
204–218	`open_session`	Valida el segundo nivel y emite la URL temporal de visualización.
224–237	`shiny_http`	Intermedia HTTP hacia Shiny usando la identidad firmada.
240–288	`shiny_websocket`	Intermedia la conexión interactiva bidireccional y comprueba la sesión durante su uso.
260–265	`to_upstream`	Operación interna del servicio; véase el código completo.
267–272	`to_browser`	Operación interna del servicio; véase el código completo.
274–281	`check_session`	Operación interna del servicio; véase el código completo.
291–294	`catalog`	Entrega el catálogo protegido; sus plantillas de enlace requieren una sesión real.
297–306	`series`	Entrega los puntos autorizados de un indicador y filtra por mes.
309–311	`module`	Redirige la sesión al indicador inicial.
314–316	`embed`	Abre un indicador concreto usando una sesión ya autorizada.
319–324	`close_view`	Cierra una sesión después de validar el origen de la solicitud.
327–330	`authorized_demo`	Muestra la demostración autorizada para una sesión.
335–336	`no_anonymous`	Rechaza accesos HTTP directos sin capacidad temporal.
339–340	`no_anonymous_ws`	Rechaza WebSocket directo sin autorización.
343–344	`demo`	Entrega la página de demostración inicial.
347–348	`root`	Redirige la raíz de este servicio a `/embed/presidencia`.
350–363	`schema`	Completa el esquema OpenAPI con las tres cabeceras de autenticación.

Líneas	Elemento	Función en la integración
369–370	`page`	Construye una página HTML auxiliar; no reemplaza el dashboard Shiny.

12.2. Archivo api.py: código y explicación de cada línea

Las tablas de explicación se construyen a partir de `recursos/explicaciones_python.json`. Cada fila se vincula con el número y el texto exacto de una línea del archivo. Si cambia el código y no se actualiza la explicación, el renderizado se detiene para evitar referencias incorrectas. Las líneas vacías solo separan bloques y no requieren una instrucción de lectura.

```python
"""API de solo lectura sobre un corte autorizado creado por el módulo R."""
import html
import asyncio
import os
import secrets
from datetime import date, datetime, timezone
from pathlib import Path
from typing import Annotated, Literal
from urllib.parse import urlsplit

import httpx
from fastapi import Depends, FastAPI, HTTPException, Query, Request, WebSocket
from fastapi.responses import HTMLResponse, RedirectResponse, Response
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer, APIKeyHeader
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from fastapi.exceptions import RequestValidationError
from fastapi.openapi.utils import get_openapi
from pydantic import BaseModel, ConfigDict, ValidationError, SecretStr, Field
from websockets.asyncio.client import connect
from .auth import Gateway, PostgresIdentity, AccessDenied, IdentityUnavailable

IDS = ('primer-nivel', 'salud-mental', 'paquete-ninos', 'paquete-gestantes', 'medicamentos')
LABELS = dict(zip(IDS, ('Cobertura del primer nivel', 'Cobertura de salud mental',
                        'Paquete priorizado en niños', 'Paquete priorizado en gestantes',
                        'Abastecimiento de medicamentos')))
IndicatorId = Literal['primer-nivel', 'salud-mental', 'paquete-ninos', 'paquete-gestantes', 'medicamentos']
BASE = Path(__file__).resolve().parent


class Punto(BaseModel):
    model_config = ConfigDict(extra='forbid', allow_inf_nan=False)
    fecha: date
    mes: str
    valor: float
    meta_mensual: float | None
    numerador: float | None
    numerador_puntual: float | None
    denominador: float | None
    fecha_corte: date | None
    fecha_actualizacion: datetime | None


class Indicador(BaseModel):
    model_config = ConfigDict(extra='forbid', allow_inf_nan=False)
    id: IndicatorId
    titulo: str
    objetivo: str
    unidad: Literal['Porcentaje']
    linea_base: float
    meta_2026: float
    datos: list[Punto]


class Corte(BaseModel):
    model_config = ConfigDict(extra='forbid')
    modulo: Literal['PRESIDENCIA']
    origen: str
    generado_en: datetime
```

```
    indicadores: list[Indicador]


class EnlaceIndicador(BaseModel):
    id: IndicatorId
    titulo: str
    dashboard: str
    datos: str


class Catalogo(BaseModel):
    modulo: Literal['PRESIDENCIA']
    generado_en: datetime
    indicadores: list[EnlaceIndicador]


class Serie(Indicador):
    generado_en: datetime
    origen: str


class SolicitudSesion(BaseModel):
    model_config = ConfigDict(extra='forbid')
    usuario_sivas: str = Field(min_length=1, max_length=80)
    clave_sivas: SecretStr = Field(min_length=1, max_length=256)
    indicador: IndicatorId = 'primer-nivel'


def create_app(*, key: str | None = None, snapshot: Path | None = None,
               shiny_url: str | None = None, max_age: float | None = None,
               gateway: Gateway | None = None) -> FastAPI:
    api_key = key if key is not None else os.getenv('PRESIDENCIA_API_KEY', '')
    if len(api_key) < 32:
        raise RuntimeError('Configure PRESIDENCIA_API_KEY con al menos 32 caracteres aleatorios.')
    gate = gateway or Gateway(os.getenv('PRESIDENCIA_CLIENT_USER',''),
        os.getenv('PRESIDENCIA_CLIENT_HASH',''), os.getenv('PRESIDENCIA_BRIDGE_KEY',''),
        PostgresIdentity(os.getenv('PRESIDENCIA_SIVAS_USER','')),
        username=os.getenv('PRESIDENCIA_SIVAS_USER',''),
        seconds=int(os.getenv('PRESIDENCIA_VIEW_SECONDS','28800')))
    source = snapshot or Path(os.getenv('PRESIDENCIA_SNAPSHOT', str(BASE / 'runtime/presidencia.json')))
    shiny = (shiny_url or os.getenv('PRESIDENCIA_SHINY_PUBLIC_URL', 'http://127.0.0.1:8001')).rstrip('/')
    parsed = urlsplit(shiny)
    if parsed.scheme not in ('http', 'https') or not parsed.hostname or parsed.username or parsed.password or
     ↪  parsed.query or parsed.fragment:
        raise RuntimeError('La URL pública Shiny debe ser HTTP/HTTPS sin credenciales, query ni fragmento.')
    ttl = max_age if max_age is not None else float(os.getenv('PRESIDENCIA_MAX_AGE_HOURS', '24'))
    if ttl <= 0:
        raise RuntimeError('La vigencia del corte debe ser positiva.')
    ancestors = os.getenv('PRESIDENCIA_FRAME_ANCESTORS', "'self' http://127.0.0.1:8000 http://localhost:8000")
    if any(c in ancestors for c in (';', '\n', '\r', '*')):
        raise RuntimeError('Configure orígenes iframe explícitos, sin comodines.')
    environment = os.getenv('PRESIDENCIA_ENVIRONMENT', 'desarrollo')
    app = FastAPI(title=f'SIVAS - Integración de Presidencia ({environment})', version='1.0.0',
                  description='Solo Presidencia. Cliente API y token permanente revocable; sesión iframe tras
 ↪  validar la cuenta institucional.',
                  docs_url=os.getenv('PRESIDENCIA_DOCS_URL', '/docs'),
                  openapi_url=os.getenv('PRESIDENCIA_OPENAPI_URL', '/openapi.json'), redoc_url=None)
    app.add_middleware(TrustedHostMiddleware, allowed_hosts=os.getenv(
        'PRESIDENCIA_ALLOWED_HOSTS', '127.0.0.1,localhost,testserver').split(','))
    bearer = HTTPBearer(auto_error=False)
    client_user = APIKeyHeader(name='X-API-Usuario',scheme_name='UsuarioAPI',auto_error=False)
    client_password = APIKeyHeader(name='X-API-Clave',scheme_name='ClaveAPI',auto_error=False)

    def authorize(request: Request, credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(bearer)],
                  username: Annotated[str | None, Depends(client_user)], password: Annotated[str | None,
 ↪  Depends(client_password)]):
        try:
            gate.throttle(request.client.host if request.client else 'unknown')
            if credentials is None or credentials.scheme.lower() != 'bearer' or not
             ↪  secrets.compare_digest(credentials.credentials.encode(), api_key.encode()):
                raise AccessDenied()
            gate.verify_client(username,password)
            gate.principal()
        except AccessDenied:
```

```
            try:
                gate.throttle(request.client.host if request.client else 'unknown',fail=True)
            except AccessDenied:
                pass
           raise HTTPException(401, 'Credencial de integración inválida.', headers={'WWW-Authenticate': 'Bearer'})
        except IdentityUnavailable:
            raise HTTPException(503,'No se pudo comprobar la autorización institucional.') from None

    def view(sid):
        try:
            return gate.resolve(sid)
        except AccessDenied:
            raise HTTPException(401,'Sesión de visualización inválida o vencida.') from None
        except IdentityUnavailable:
            raise HTTPException(503,'No se pudo comprobar la autorización institucional.') from None

    @app.exception_handler(RequestValidationError)
    async def invalid_input(request, exc):
        # No devolver el campo clave_sivas ni otros secretos en errores de validación.
        return Response('{"detail":"Solicitud inválida; revise los campos y rangos permitidos."}',
                        status_code=422,media_type='application/json')

    def read_snapshot() -> Corte:
        try:
            if source.stat().st_size > 2_000_000:
                raise ValueError('Corte demasiado grande')
            cut = Corte.model_validate_json(source.read_text(encoding='utf-8'))
            if {i.id for i in cut.indicadores} != set(IDS) or len(cut.indicadores) != 5:
                raise ValueError('Catálogo incompleto')
            if cut.generado_en.tzinfo is None:
                raise ValueError('Fecha sin zona')
            age = (datetime.now(timezone.utc) - cut.generado_en).total_seconds()
            if age < -300 or age > ttl * 3600:
                raise ValueError('Corte vencido')
            for indicator in cut.indicadores:
                dates = [p.fecha for p in indicator.datos]
                if len(set(dates)) != len(dates) or any(d.year != 2026 for d in dates):
                    raise ValueError('Períodos inválidos')
            return cut
        except (OSError, ValueError, ValidationError):
            raise HTTPException(503, 'Corte no disponible, inválido o vencido. Ejecute la actualización de
            ↪ desarrollo.') from None

    @app.middleware('http')
    async def headers(request, call_next):
        response = await call_next(request)
        response.headers['Cache-Control'] = 'no-store'
        response.headers['X-Content-Type-Options'] = 'nosniff'
        response.headers['Referrer-Policy'] = 'no-referrer'
        response.headers['Content-Security-Policy'] = f"frame-ancestors {ancestors}; frame-src 'self'
↪   {parsed.scheme}://{parsed.netloc}; object-src 'none'; base-uri 'self'"
        return response

    @app.get('/health/live', tags=['Salud'])
    def live() -> dict:
        return {'estado': 'activo', 'modulo': 'PRESIDENCIA', 'ambiente': environment}

    @app.get('/health/ready', tags=['Salud'])
    def ready() -> dict:
        cut = read_snapshot()
        return {'estado': 'listo', 'indicadores': len(cut.indicadores), 'generado_en': cut.generado_en}

    @app.get('/health/shiny', tags=['Salud'])
    async def shiny_health() -> dict:
        target = os.getenv('PRESIDENCIA_SHINY_INTERNAL_URL', shiny)
        try:
            async with httpx.AsyncClient(timeout=5, follow_redirects=False) as client:
                response = await client.get(target + '/')
            if response.status_code != 200:
                raise ValueError('Shiny no disponible')
        except (httpx.HTTPError, ValueError):
            raise HTTPException(503, 'Shiny de desarrollo no responde.') from None
        return {'estado': 'disponible', 'autenticacion': 'cuenta institucional validada antes del iframe'}
```

```python
    prefix = '/api/v1/presidencia'

    @app.post(prefix + '/sesiones', dependencies=[Depends(authorize)],tags=['Presidencia'])
    def open_session(body: SolicitudSesion, request: Request):
        try:
            sid = gate.issue(body.usuario_sivas,body.clave_sivas.get_secret_value(),body.indicador)
        except AccessDenied:
            try:
                gate.throttle(request.client.host if request.client else 'unknown',fail=True)
            except AccessDenied:
                pass
            raise HTTPException(401,'Cuenta institucional no autorizada.') from None
        except IdentityUnavailable:
            raise HTTPException(503,'No se pudo comprobar la autorización institucional.') from None
        base = str(request.base_url).rstrip('/') + '/vista/' + sid
        return {'url':base+'/embed/presidencia','demo':base+'/demo',
                'vigencia_segundos':gate.seconds,'indicadores':[
                    {'id':i,'url':base+'/embed/presidencia/'+i} for i in IDS]}

    # Capacidad temporal en la ruta: no requiere cookies de terceros en iframe.
    internal = os.getenv('PRESIDENCIA_SHINY_INTERNAL_URL', shiny).rstrip('/')

    @app.api_route('/vista/{sid}/shiny/{path:path}', methods=['GET', 'POST'], include_in_schema=False)
    async def shiny_http(sid: str, path: str, request: Request):
        await asyncio.to_thread(view,sid)
        signed = await asyncio.to_thread(gate.headers,sid)
        target = internal + '/' + path
        try:
            async with httpx.AsyncClient(timeout=40, follow_redirects=False) as client:
                response = await client.request(request.method, target, params=request.query_params,
                    content=await request.body(), headers={**{k:v for k,v in request.headers.items()
                        if k.lower() in ('content-type', 'cookie', 'accept', 'range')},**signed})
        except httpx.HTTPError:
            raise HTTPException(503, 'Shiny de desarrollo no responde.') from None
        return Response(response.content, status_code=response.status_code,
            headers={k:v for k,v in response.headers.items() if k.lower() in (
                'content-type', 'content-disposition', 'accept-ranges', 'content-range')})

    @app.websocket('/vista/{sid}/shiny/{path:path}')
    async def shiny_websocket(sid: str, path: str, socket: WebSocket):
        try:
            signed = await asyncio.to_thread(gate.headers,sid)
        except (AccessDenied,IdentityUnavailable):
            await socket.close(code=1008)
            return
        # Bloquear conexiones WS originadas por otros sitios; no se aceptan tokens en URL.
        origin = socket.headers.get('origin', '')
        expected = str(socket.url).replace('ws://', 'http://').replace('wss://', 'https://')
        expected_parts = urlsplit(expected)
        if origin != f'{expected_parts.scheme}://{expected_parts.netloc}':
            await socket.close(code=1008)
            return
        backend = internal.replace('https://', 'wss://').replace('http://', 'ws://') + '/' + path
        if socket.url.query:
            backend += '?' + socket.url.query
        try:
            async with connect(backend, additional_headers=signed, max_size=4*1024*1024, open_timeout=10) as
            ↪ upstream:
                await socket.accept()

                async def to_upstream():
                    while True:
                        message = await socket.receive()
                        if message['type'] == 'websocket.disconnect':
                            break
                        await upstream.send(message.get('text') if message.get('text') is not None else
                        ↪ message['bytes'])

                async def to_browser():
                    async for message in upstream:
                        if isinstance(message, str):
                            await socket.send_text(message)
                        else:
                            await socket.send_bytes(message)
```

```
                async def check_session():
                    while True:
                        await asyncio.sleep(10)
                        try:
                            await asyncio.to_thread(gate.resolve,sid)
                        except (AccessDenied,IdentityUnavailable):
                            await socket.close(code=1008)
                            return
                tasks = [asyncio.create_task(to_upstream()),
↪  asyncio.create_task(to_browser()),asyncio.create_task(check_session())]
                done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
                for task in pending:
                    task.cancel()
                await asyncio.gather(*tasks, return_exceptions=True)
        except (OSError, TimeoutError):
            await socket.close(code=1011)

    @app.get(prefix, response_model=Catalogo, dependencies=[Depends(authorize)], tags=['Presidencia'])
    def catalog():
        cut = read_snapshot()
        return {'modulo': cut.modulo, 'generado_en': cut.generado_en,
                'indicadores': [dict(id=i.id, titulo=i.titulo,
                 ↪  dashboard=f'/vista/{{sesion}}/embed/presidencia/{i.id}', datos=f'{prefix}/{i.id}/serie') for
                 ↪  i in cut.indicadores]}

    @app.get(prefix + '/{indicator_id}/serie', response_model=Serie, dependencies=[Depends(authorize)],
     ↪  tags=['Presidencia'])
    def series(indicator_id: IndicatorId, anio: Annotated[int, Query(ge=2026, le=2026)] = 2026,
               mes_desde: Annotated[int, Query(ge=1, le=12)] = 1,
               mes_hasta: Annotated[int, Query(ge=1, le=12)] = 12):
        if mes_desde > mes_hasta:
            raise HTTPException(422, 'mes_desde debe ser menor o igual a mes_hasta.')
        cut = read_snapshot()
        indicator = next(i for i in cut.indicadores if i.id == indicator_id)
        data = indicator.model_dump(mode='json')
        data['datos'] = [p.model_dump(mode='json') for p in indicator.datos if p.fecha.year == anio and mes_desde
↪  <= p.fecha.month <= mes_hasta]
        return {'generado_en': cut.generado_en, 'origen': cut.origen, **data}

    @app.get('/vista/{sid}/embed/presidencia', response_class=HTMLResponse, include_in_schema=False)
    def module(sid: str):
        session = view(sid)
        return embed(sid,session['indicator'])

    @app.get('/vista/{sid}/embed/presidencia/{indicator_id}', response_class=HTMLResponse,
     ↪  include_in_schema=False)
    def embed(sid: str, indicator_id: IndicatorId):
        view(sid)
        return page(f'<iframe title="{html.escape(LABELS[indicator_id])}"
         ↪  src="/vista/{sid}/shiny/?indicador={indicator_id}"></iframe>')

    @app.post('/vista/{sid}/cerrar',include_in_schema=False)
    def close_view(sid: str,request: Request):
        view(sid)
        if request.headers.get('origin') != str(request.base_url).rstrip('/'):
            raise HTTPException(403,'Origen no autorizado.')
        gate.revoke(sid)
        return {'estado':'cerrada'}

    @app.get('/vista/{sid}/demo', response_class=HTMLResponse, include_in_schema=False)
    def authorized_demo(sid: str):
        view(sid)
        return (BASE / 'examples/demo.html').read_text(encoding='utf-8').replace('__IFRAME_URL__',
            '/vista/'+sid+'/embed/presidencia')

    @app.api_route('/shiny/{path:path}',methods=['GET','POST'],include_in_schema=False)
    @app.get('/embed/presidencia',include_in_schema=False)
    @app.get('/embed/presidencia/{path:path}',include_in_schema=False)
    def no_anonymous(path: str = ''):
        raise HTTPException(401,'Genere una sesión con POST /api/v1/presidencia/sesiones.')

    @app.websocket('/shiny/{path:path}')
    async def no_anonymous_ws(socket: WebSocket,path: str):
```

```
340         await socket.close(code=1008)
341
342     @app.get('/demo', response_class=HTMLResponse, tags=['Dashboards'], include_in_schema=environment ==
        ↪ 'desarrollo')
343     def demo():
344         return '<!doctype html><html lang="es"><meta charset="utf-8"><title>Integración
            ↪ Presidencia</title><p>Genere una sesión mediante POST /api/v1/presidencia/sesiones. Abra el enlace
            ↪ demo de la respuesta.</p></html>'
345
346     @app.get('/', include_in_schema=False)
347     def root():
348         return RedirectResponse('/embed/presidencia')
349
350     def schema():
351         if app.openapi_schema is None:
352             result =
    ↪ get_openapi(title=app.title,version=app.version,description=app.description,routes=app.routes)
353             health_prefix = os.getenv('PRESIDENCIA_HEALTH_PUBLIC_PREFIX', '')
354             if health_prefix:
355                 for path in list(result['paths']):
356                     if path.startswith('/health/'):
357                         result['paths'][health_prefix + path] = result['paths'].pop(path)
358             for path, operations in result['paths'].items():
359                 if path.startswith(prefix):
360                     for operation in operations.values():
361                         operation['security'] = [{'HTTPBearer':[], 'UsuarioAPI':[], 'ClaveAPI':[]}]
362             app.openapi_schema = result
363         return app.openapi_schema
364     app.openapi = schema
365
366     return app
367
368
369 def page(content: str) -> str:
370     return '<!doctype html><html lang="es"><head><meta charset="utf-8"><meta name="viewport"
        ↪ content="width=device-width,initial-scale=1"><title>SIVAS -
        ↪ Presidencia</title><style>html,body{width:100%;height:100%;margin:0;overflow:hidden}iframe{display:bloc
        ↪ k;width:100%;height:100%;border:0}</style></head><body>' + content + '</body></html>'
```

Lectura línea por línea: deploy/presidencia/api.py

Línea	Explicación
1	Texto descriptivo del componente; no ejecuta una operación de red ni abre un servicio.
2	Importa html para escape de títulos que se insertan en HTML.
3	Importa asyncio para tareas y esperas asíncronas del proxy WebSocket.
4	Importa os para variables de entorno privadas del proceso.
5	Importa secrets para comparación de credenciales y generación aleatoria de SID.
6	Importa datetime para fechas, horas y cálculos de vigencia UTC.
7	Importa pathlib para rutas de archivos independientes del directorio desde el que se invoca el código.
8	Importa typing para anotaciones de tipo y valores de indicador permitidos.
9	Importa urllib.parse para separación y validación de direcciones URL.
11	Importa httpx para peticiones HTTP asíncronas al visualizador interno.
12	Importa fastapi para aplicación, rutas, dependencias, excepciones y contrato HTTP.
13	Importa fastapi.responses para respuestas HTML, redirección y cuerpos HTTP.
14	Importa fastapi.security para lectores de los tres encabezados de autenticación API.
15	Importa fastapi.middleware.trustedhost para comprobación de nombres de host permitidos.
16	Importa fastapi.exceptions para manejo de errores de validación de entradas.
17	Importa fastapi.openapi.utils para generación de OpenAPI para Swagger.
18	Importa pydantic para modelos de entrada/salida y tratamiento de contraseñas como secretos.
19	Importa websockets.asyncio.client para conexión WebSocket hacia el visualizador.
20	Importa auth para gateway y proveedor de identidad institucional reutilizados por api.py.
22	En `módulo`, declara los cinco identificadores que admite la integración.
23	En `módulo`, relaciona esos identificadores con sus títulos institucionales.

Línea	Explicación
24	Continuación de la instrucción de la línea 23: En `módulo`, relaciona esos identificadores con sus títulos institucionales.
25	Continuación de la instrucción de la línea 23: En `módulo`, relaciona esos identificadores con sus títulos institucionales.
26	En `módulo`, restringe mediante Literal los valores de indicador admitidos.
27	En `módulo`, resuelve la carpeta real del archivo para encontrar recursos relativos.
30	Declara `Punto`: modelo de datos validado de FastAPI/Pydantic.
31	En `módulo`, rechaza campos ajenos al contrato; cuando corresponde también impide NaN e infinito.
32	Define `fecha`: fecha del punto mensual, validada como fecha de calendario.
33	Define `mes`: nombre del mes mostrado en la serie.
34	Define `valor`: resultado numérico del indicador para ese período.
35	Define `meta_mensual`: meta del mes; puede ser nula.
36	Define `numerador`: numerador acumulado del indicador; admite nulo.
37	Define `numerador_puntual`: numerador del período puntual; admite nulo.
38	Define `denominador`: denominador usado en la medición; admite nulo.
39	Define `fecha_corte`: fecha hasta la que corresponde el dato; admite nulo.
40	Define `fecha_actualizacion`: fecha y hora de actualización; admite nulo.
43	Declara `Indicador`: modelo de datos validado de FastAPI/Pydantic.
44	En `módulo`, rechaza campos ajenos al contrato; cuando corresponde también impide NaN e infinito.
45	Define `id`: uno de los cinco identificadores de indicador permitidos.
46	Define `titulo`: título institucional del indicador.
47	Define `objetivo`: descripción del objetivo del indicador.
48	Define `unidad`: unidad de medida, restringida al texto Porcentaje.
49	Define `linea_base`: valor de línea base del indicador.
50	Define `meta_2026`: meta anual del año 2026.
51	Define `datos`: lista validada de puntos mensuales del modelo Punto.
54	Declara `Corte`: modelo de datos validado de FastAPI/Pydantic.
55	En `módulo`, rechaza campos ajenos al contrato; cuando corresponde también impide NaN e infinito.
56	Define `modulo`: módulo, restringido al identificador PRESIDENCIA.
57	Define `origen`: identificación de la fuente de datos.
58	Define `generado_en`: fecha y hora en que se produjo el corte autorizado.
59	Define `indicadores`: lista de indicadores o enlaces del catálogo.
62	Declara `EnlaceIndicador`: modelo de datos validado de FastAPI/Pydantic.
63	Define `id`: uno de los cinco identificadores de indicador permitidos.
64	Define `titulo`: título institucional del indicador.
65	Define `dashboard`: plantilla de URL de visualización del indicador.
66	Define `datos`: URL del endpoint REST que entrega la serie de este indicador.
69	Declara `Catalogo`: modelo de datos validado de FastAPI/Pydantic.
70	Define `modulo`: módulo, restringido al identificador PRESIDENCIA.
71	Define `generado_en`: fecha y hora en que se produjo el corte autorizado.
72	Define `indicadores`: lista de indicadores o enlaces del catálogo.
75	Declara `Serie`: modelo de datos validado de FastAPI/Pydantic.
76	Define `generado_en`: fecha y hora en que se produjo el corte autorizado.
77	Define `origen`: identificación de la fuente de datos.
80	Declara `SolicitudSesion`: modelo de datos validado de FastAPI/Pydantic.
81	En `módulo`, rechaza campos ajenos al contrato; cuando corresponde también impide NaN e infinito.
82	Define `usuario_sivas`: nombre institucional obligatorio, de uno a ochenta caracteres.
83	Define `clave_sivas`: contraseña institucional tratada como SecretStr, de uno a 256 caracteres.
84	Define `indicador`: indicador inicial autorizado; por defecto primer-nivel.
87	Define `create_app`, que construye la aplicación FastAPI, su configuración, el gateway y sus rutas.
88	Completa los parámetros de `create_app`: shiny_url: str | None = None, max_age: float | None = None,.
89	Completa los parámetros de `create_app`: gateway: Gateway | None = None) -> FastAPI:.
90	En `create_app`, obtiene el Bearer del argumento de pruebas o de PRESIDENCIA_API_KEY.
91	Exige un Bearer de al menos 32 caracteres antes de crear la aplicación.
92	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.

Línea	Explicación
93	Usa el gateway recibido en pruebas o crea uno con el usuario del cliente API configurado.
94	Entrega al gateway el hash Argon2 de la clave API y la clave HMAC del puente; no son la contraseña institucional.
95	Construye PostgresIdentity para consultar únicamente la cuenta institucional autorizada.
96	Fija en el gateway el nombre institucional que podrá generar sesiones de Presidencia.
97	Lee PRESIDENCIA_VIEW_SECONDS como entero; el valor por defecto es 28.800 segundos.
98	En `create_app`, resuelve la ubicación del corte JSON de indicadores desde la configuración.
99	En `create_app`, normaliza la dirección del visualizador quitando la barra final.
100	En `create_app`, separa esquema, host y otras partes de la URL para validarlas.
101	Rechaza una dirección del visualizador sin host o protocolo HTTP/HTTPS, o que incluya credenciales, consulta o fragmento.
102	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
103	En `create_app`, obtiene la antigüedad máxima permitida al corte, expresada en horas.
104	Comprueba que la antigüedad máxima del corte sea mayor que cero.
105	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
106	En `create_app`, lee la lista explícita de orígenes que pueden embeber el visualizador.
107	Impide comodines, saltos de línea y punto y coma en la configuración de orígenes iframe.
108	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
109	En `create_app`, lee el nombre del ambiente para identificarlo en FastAPI y OpenAPI.
110	Crea FastAPI con título identificable del ambiente y versión 1.0.0.
111	Describe en OpenAPI el alcance del módulo y la diferencia entre credenciales API y sesión temporal.
112	Lee la ruta de Swagger desde PRESIDENCIA_DOCS_URL; su valor inicial es /docs.
113	Configura la URL de OpenAPI y desactiva la interfaz ReDoc.
114	Instala el control de host de FastAPI para rechazar dominios no autorizados.
115	Convierte la cadena de hosts permitidos en una lista separando por comas.
116	En `create_app`, prepara el lector del encabezado Authorization sin emitir un rechazo automático.
117	En `create_app`, prepara el lector del encabezado X-API-Usuario.
118	En `create_app`, prepara el lector del encabezado X-API-Clave.
120	Declara authorize como dependencia de acceso: recibe la petición y el Bearer que FastAPI extrajo de Authorization.
121	Recibe X-API-Usuario y X-API-Clave mediante Depends; ninguno se obtiene del JavaScript del iframe.
122	Agrupa operaciones de `authorize` cuyos errores se traducen a una respuesta controlada.
123	Consulta o incrementa el contador de fallos del origen, antes de aceptar más intentos.
124	Rechaza un Bearer ausente, un esquema distinto de Bearer o un token diferente al configurado; compara el token en tiempo constante.
125	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
126	Verifica X-API-Usuario y X-API-Clave contra el cliente y su hash configurados.
127	Comprueba que la cuenta institucional configurada siga activa, vigente y autorizada para PRESIDENCIA.
128	Captura `AccessDenied` en `authorize` para responder sin mostrar detalles internos ni credenciales.
129	Agrupa operaciones de `authorize` cuyos errores se traducen a una respuesta controlada.
130	Registra el intento fallido asociado al origen para limitar repeticiones dentro de la ventana configurada.
131	Captura `AccessDenied` en `authorize` para responder sin mostrar detalles internos ni credenciales.
132	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
133	Devuelve 401 y el encabezado WWW-Authenticate: Bearer, sin especificar qué secreto falló.
134	Captura `IdentityUnavailable` en `authorize` para responder sin mostrar detalles internos ni credenciales.
135	Devuelve 503 cuando la identidad institucional no puede comprobarse; no concede acceso por omisión.
137	Define `view`, que resuelve un SID y convierte rechazos o fallos de identidad en respuestas HTTP.
138	Agrupa operaciones de `view` cuyos errores se traducen a una respuesta controlada.
139	Devuelve una copia de la sesión autorizada para ese SID.

Línea	Explicación
140	Captura `AccessDenied` en `view` para responder sin mostrar detalles internos ni credenciales.
141	Interrumpe esta ruta con HTTP 401: rechaza la autenticación o sesión inválida. No devuelve el secreto recibido.
142	Captura `IdentityUnavailable` en `view` para responder sin mostrar detalles internos ni credenciales.
143	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
145	Aplica el decorador `app.exception_handler(RequestValidationError)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
146	Define `invalid_input`, que responde 422 sin devolver los secretos que causaron un error de validación.
147	Comentario del código: No devolver el campo clave_sivas ni otros secretos en errores de validación.
148	Devuelve JSON genérico con estado 422; no refleja clave_sivas en el error.
149	Continuación de la instrucción de la línea 148: Devuelve JSON genérico con estado 422; no refleja clave_sivas en el error.
151	Define `read_snapshot`, que lee el corte REST autorizado y valida tamaño, indicadores, fecha y vigencia.
152	Agrupa operaciones de `read_snapshot` cuyos errores se traducen a una respuesta controlada.
153	Comprueba `source.stat().st_size > 2000000` para asegurar integridad y vigencia del corte antes de devolver datos.
154	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
155	En `read_snapshot`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
156	Comprueba `{i.id for i in cut.indicadores} != set(IDS) or len(cut.indicadores) != 5` para asegurar integridad y vigencia del corte antes de devolver datos.
157	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
158	Comprueba `cut.generado_en.tzinfo is None` para asegurar integridad y vigencia del corte antes de devolver datos.
159	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
160	En `read_snapshot`, calcula en segundos la diferencia entre el instante actual UTC y la generación del corte.
161	Comprueba `age < -300 or age > ttl * 3600` para asegurar integridad y vigencia del corte antes de devolver datos.
162	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
163	Recorre `cut.indicadores` asignando cada elemento a `indicator` dentro de `read_snapshot`.
164	En `read_snapshot`, extrae las fechas del indicador para detectar duplicados o años no admitidos.
165	Comprueba `len(set(dates)) != len(dates) or any((d.year != 2026 for d in dates))` para asegurar integridad y vigencia del corte antes de devolver datos.
166	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
167	Devuelve el objeto Corte validado para los endpoints de lectura.
168	Captura `(OSError, ValueError, ValidationError)` en `read_snapshot` para responder sin mostrar detalles internos ni credenciales.
169	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
171	Aplica el decorador `app.middleware('http')`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
172	Define `headers`, que añade políticas de no caché, tipos de contenido, referencia y ancestros iframe.
173	En `headers`, guarda la respuesta del visualizador o de la siguiente capa HTTP.
174	Establece `response.headers['Cache-Control']` en la respuesta; esta política limita caché, interpretación de tipos, referencias u orígenes iframe según la línea de código.
175	Establece `response.headers['X-Content-Type-Options']` en la respuesta; esta política limita caché, interpretación de tipos, referencias u orígenes iframe según la línea de código.
176	Establece `response.headers['Referrer-Policy']` en la respuesta; esta política limita caché, interpretación de tipos, referencias u orígenes iframe según la línea de código.
177	Establece `response.headers['Content-Security-Policy']` en la respuesta; esta política limita caché, interpretación de tipos, referencias u orígenes iframe según la línea de código.

Línea	Explicación
178	Devuelve la respuesta HTTP con las políticas de seguridad incorporadas.
180	Aplica el decorador `app.get('/health/live', tags=['Salud'])`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
181	Define `live`, que informa que el proceso FastAPI atiende peticiones.
182	Devuelve estado activo, módulo y ambiente como JSON.
184	Aplica el decorador `app.get('/health/ready', tags=['Salud'])`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
185	Define `ready`, que comprueba la disponibilidad y vigencia del corte de indicadores.
186	En `ready`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
187	Devuelve estado listo, número de indicadores y fecha del corte.
189	Aplica el decorador `app.get('/health/shiny', tags=['Salud'])`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
190	Define `shiny_health`, que consulta por HTTP el visualizador interno para comprobar su disponibilidad.
191	En `shiny_health`, construye la dirección interna a la que se enviará la petición HTTP del visualizador.
192	Agrupa operaciones de `shiny_health` cuyos errores se traducen a una respuesta controlada.
193	Abre un cliente HTTP asíncrono con el timeout indicado y sin seguir redirecciones automáticamente.
194	En `shiny_health`, guarda la respuesta del visualizador o de la siguiente capa HTTP.
195	Comprueba `response.status_code != 200` dentro de `shiny_health`, que consulta por HTTP el visualizador interno para comprobar su disponibilidad.
196	Interrumpe la validación del recurso y deja que el manejador del bloque traduzca el error.
197	Captura `(httpx.HTTPError, ValueError)` en `shiny_health` para responder sin mostrar detalles internos ni credenciales.
198	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
199	Devuelve disponibilidad del visualizador después de comprobar su respuesta HTTP.
201	En `create_app`, define /api/v1/presidencia, base de los endpoints REST.
203	Registra POST /api/v1/presidencia/sesiones y exige que authorize termine correctamente antes de ejecutar open_session.
204	Recibe el cuerpo SolicitudSesion ya validado y Request para conocer la dirección pública de la solicitud.
205	Agrupa operaciones de `open_session` cuyos errores se traducen a una respuesta controlada.
206	Extrae la contraseña de SecretStr solo dentro del backend y llama a Gateway.issue con usuario, contraseña e indicador inicial.
207	Captura `AccessDenied` en `open_session` para responder sin mostrar detalles internos ni credenciales.
208	Agrupa operaciones de `open_session` cuyos errores se traducen a una respuesta controlada.
209	Consulta o incrementa el contador de fallos del origen, antes de aceptar más intentos.
210	Captura `AccessDenied` en `open_session` para responder sin mostrar detalles internos ni credenciales.
211	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
212	Interrumpe esta ruta con HTTP 401: rechaza la autenticación o sesión inválida. No devuelve el secreto recibido.
213	Captura `IdentityUnavailable` en `open_session` para responder sin mostrar detalles internos ni credenciales.
214	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
215	Construye la base pública /vista/ usando esquema y host de Request; Nginx debe aportar los encabezados correctos.
216	Devuelve url para el módulo y demo para la página de prueba; ambas incorporan el SID temporal.
217	Incluye vigencia_segundos y empieza la lista de enlaces individuales de los cinco indicadores.
218	Asocia cada ID autorizado con su URL /embed/presidencia/; no devuelve contraseñas ni hashes.
220	Comentario del código: Capacidad temporal en la ruta: no requiere cookies de terceros en iframe.
221	En `create_app`, lee la URL interna del visualizador para reenviar HTTP y WebSocket.

Línea	Explicación
223	Aplica el decorador `app.api_route('/vista/{sid}/shiny/{path:path}', methods=['GET', 'POST'], include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
224	Declara el proxy HTTP que recibe SID, recurso interno solicitado y petición HTTP del navegador.
225	Comprueba la sesión antes de reenviar el recurso; ejecuta esa validación síncrona mediante un hilo.
226	Obtiene los encabezados firmados para que el dashboard reconozca al usuario ya autenticado.
227	En `shiny_http`, construye la dirección interna a la que se enviará la petición HTTP del visualizador.
228	Agrupa operaciones de `shiny_http` cuyos errores se traducen a una respuesta controlada.
229	Abre un cliente HTTP con espera máxima de cuarenta segundos y sin seguir redirecciones.
230	Reenvía el método y la consulta recibidos al recurso interno autorizado.
231	Reenvía el cuerpo y prepara una lista limitada de encabezados del navegador.
232	Solo admite content-type, cookie, accept y range del navegador; añade después los encabezados firmados del gateway.
233	Captura `httpx.HTTPError` en `shiny_http` para responder sin mostrar detalles internos ni credenciales.
234	Interrumpe esta ruta con HTTP 503: informa indisponibilidad de identidad, corte o visualizador. No devuelve el secreto recibido.
235	Construye la respuesta con el cuerpo y el estado HTTP del visualizador.
236	Filtra los encabezados de retorno para no propagar encabezados internos arbitrarios.
237	Conserva tipo de contenido, disposición de descarga y encabezados de rango necesarios para recursos y archivos.
239	Aplica el decorador `app.websocket('/vista/{sid}/shiny/{path:path}')`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
240	Define `shiny_websocket`, que mantiene el canal interactivo autorizado entre navegador y visualizador.
241	Agrupa operaciones de `shiny_websocket` cuyos errores se traducen a una respuesta controlada.
242	En `shiny_websocket`, obtiene los encabezados internos firmados que acreditan la sesión ante el dashboard.
243	Captura `(AccessDenied, IdentityUnavailable)` en `shiny_websocket` para responder sin mostrar detalles internos ni credenciales.
244	Cierra el WebSocket con código 1008 (política) o 1011 (fallo interno), según el bloque.
245	Devuelve el resultado de esta operación de shiny_websocket.
246	Comentario del código: Bloquear conexiones WS originadas por otros sitios; no se aceptan tokens en URL.
247	Lee Origin del WebSocket para identificar la página que intenta establecer la conexión.
248	Convierte ws/wss a http/https para comparar el origen contra la URL pública del socket.
249	En `shiny_websocket`, extrae esquema y host que deben coincidir con el origen del navegador.
250	Exige igualdad exacta de esquema y host del origen; si no coinciden se cierra la conexión.
251	Cierra el WebSocket con código 1008 (política) o 1011 (fallo interno), según el bloque.
252	Devuelve el resultado de esta operación de shiny_websocket.
253	En `shiny_websocket`, construye la URL ws/wss del visualizador interno, conservando ruta y consulta.
254	Compara Origin con el esquema y host públicos esperados; una diferencia impide abrir el WebSocket.
255	En `shiny_websocket`, construye la URL ws/wss del visualizador interno, conservando ruta y consulta.
256	Agrupa operaciones de `shiny_websocket` cuyos errores se traducen a una respuesta controlada.
257	Conecta al WebSocket interno con firma, máximo de 4 MiB por mensaje y diez segundos para abrir la conexión.
258	Acepta el WebSocket del navegador después de comprobar SID, origen y conexión interna.
260	Define `to_upstream`, que reenvía al visualizador los mensajes WebSocket recibidos del navegador.
261	Mantiene el bucle de `to_upstream` hasta desconexión, revocación o cancelación coordinada.
262	En `to_upstream`, recibe un mensaje WebSocket para enviarlo al otro extremo sin convertir su contenido.

Línea	Explicación
263	Comprueba `message['type'] == 'websocket.disconnect'` dentro de `to_upstream`, que reenvía al visualizador los mensajes WebSocket recibidos del navegador.
264	Finaliza el bucle al detectar desconexión del navegador.
265	Reenvía el mensaje de texto o binario al extremo indicado, conservando su tipo.
267	Define `to_browser`, que envía al navegador los mensajes de texto o binarios del visualizador.
268	Recorre `upstream` asignando cada elemento a `message` dentro de `to_browser`.
269	Comprueba `isinstance(message, str)` dentro de `to_browser`, que envía al navegador los mensajes de texto o binarios del visualizador.
270	Reenvía el mensaje de texto o binario al extremo indicado, conservando su tipo.
271	Continuación de la instrucción de la línea 269: Comprueba `isinstance(message, str)` dentro de `to_browser`, que envía al navegador los mensajes de texto o binarios del visualizador.
272	Reenvía el mensaje de texto o binario al extremo indicado, conservando su tipo.
274	Define `check_session`, que revalida la sesión cada diez segundos mientras se mantiene el WebSocket.
275	Mantiene el bucle de `check_session` hasta desconexión, revocación o cancelación coordinada.
276	Espera diez segundos antes de revalidar la sesión del WebSocket.
277	Agrupa operaciones de `check_session` cuyos errores se traducen a una respuesta controlada.
278	Revalida el SID mientras el WebSocket sigue abierto, permitiendo aplicar vencimiento o revocación.
279	Captura `(AccessDenied, IdentityUnavailable)` en `check_session` para responder sin mostrar detalles internos ni credenciales.
280	Cierra el WebSocket con código 1008 (política) o 1011 (fallo interno), según el bloque.
281	Devuelve el resultado de esta operación de check_session.
282	En `shiny_websocket`, crea las tres tareas: navegador a visualizador, retorno y comprobación periódica de sesión.
283	En `shiny_websocket`, asigna `(done, pending)` al resultado de la expresión mostrada.
284	Recorre `pending` asignando cada elemento a `task` dentro de `shiny_websocket`.
285	Cancela una tarea pendiente cuando otro extremo ya terminó.
286	Espera el cierre de las tareas y recoge sus excepciones para terminar coordinadamente.
287	Captura `(OSError, TimeoutError)` en `shiny_websocket` para responder sin mostrar detalles internos ni credenciales.
288	Cierra el WebSocket con código 1008 (política) o 1011 (fallo interno), según el bloque.
290	Registra GET /api/v1/presidencia, aplica authorize y valida su salida con el modelo Catalogo.
291	Define `catalog`, que entrega el catálogo REST protegido de los cinco indicadores.
292	En `catalog`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
293	Devuelve módulo, fecha del corte y enlaces del catálogo REST.
294	Construye un enlace REST y una plantilla de URL de dashboard para cada indicador; la plantilla exige reemplazar la sesión por un SID real.
296	Registra GET /api/v1/presidencia/{indicator_id}/serie; exige authorize y valida la respuesta como Serie.
297	Limita indicator_id a los cinco IDs y anio a 2026 mediante Query; no habilita otros años por cambiar la URL.
298	Declara mes_desde entre uno y doce; su valor inicial es uno.
299	Declara mes_hasta entre uno y doce; su valor inicial es doce.
300	Comprueba que el primer mes del rango no sea posterior al último.
301	Interrumpe esta ruta con HTTP 422: rechaza parámetros fuera de contrato. No devuelve el secreto recibido.
302	En `series`, lee y valida el corte mediante el contrato Pydantic o reutiliza la lectura validada.
303	En `series`, localiza el indicador cuyo ID coincide con el solicitado.
304	En `series`, convierte el indicador validado a tipos JSON y conserva únicamente los puntos del rango pedido.
305	Conserva únicamente puntos cuyo año y mes pertenecen al rango autorizado de la petición.
306	Devuelve los puntos del indicador dentro del año y meses solicitados, con origen y fecha del corte.
308	Aplica el decorador `app.get('/vista/{sid}/embed/presidencia', response_class=HTMLResponse, include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.

Línea	Explicación
309	Define `module`, que abre el indicador inicial de una sesión autorizada.
310	En `module`, consulta la sesión actual del SID o recibe una copia de sus datos ya validados.
311	Invoca la vista del indicador inicial guardado en la sesión.
313	Aplica el decorador `app.get('/vista/{sid}/embed/presidencia/{indicator_id}', response_class=HTMLResponse, include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
314	Define `embed`, que construye el iframe interno de un indicador después de validar el SID.
315	Ejecuta `view(sid)` dentro de `embed` para construye el iframe interno de un indicador después de validar el SID.
316	Construye el iframe interno con indicador y SID; escapa el título para que su texto no se interprete como HTML.
318	Aplica el decorador `app.post('/vista/{sid}/cerrar',include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
319	Define `close_view`, que revoca una sesión después de validar su origen HTTP.
320	Ejecuta `view(sid)` dentro de `close_view` para revoca una sesión después de validar su origen HTTP.
321	Comprueba `request.headers.get('origin') != str(request.base_url).rstrip('/')` dentro de `close_view`, que revoca una sesión después de validar su origen HTTP.
322	Interrumpe esta ruta con HTTP 403: rechaza el origen no autorizado. No devuelve el secreto recibido.
323	Ejecuta `gate.revoke(sid)` dentro de `close_view` para revoca una sesión después de validar su origen HTTP.
324	Confirma en JSON que la sesión fue cerrada.
326	Aplica el decorador `app.get('/vista/{sid}/demo', response_class=HTMLResponse, include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
327	Define `authorized_demo`, que carga la demostración con una URL de iframe de la sesión autorizada.
328	Ejecuta `view(sid)` dentro de `authorized_demo` para carga la demostración con una URL de iframe de la sesión autorizada.
329	Entrega el HTML de demostración sustituyendo su marcador por la URL autorizada.
330	Continuación de la instrucción de la línea 329: Entrega el HTML de demostración sustituyendo su marcador por la URL autorizada.
332	Aplica el decorador `app.api_route('/shiny/{path:path}',methods=['GET','POST'],include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
333	Aplica el decorador `app.get('/embed/presidencia',include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
334	Aplica el decorador `app.get('/embed/presidencia/{path:path}',include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
335	Define `no_anonymous`, que rechaza el acceso HTTP al visualizador sin SID.
336	Interrumpe esta ruta con HTTP 401: rechaza la autenticación o sesión inválida. No devuelve el secreto recibido.
338	Aplica el decorador `app.websocket('/shiny/{path:path}')`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
339	Define `no_anonymous_ws`, que rechaza conexiones WebSocket directas sin autorización.
340	Cierra el WebSocket con código 1008 (política) o 1011 (fallo interno), según el bloque.
342	Aplica el decorador `app.get('/demo', response_class=HTMLResponse, tags=['Dashboards'], include_in_schema=environment == 'desarrollo')`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
343	Define `demo`, que entrega una página inicial que indica cómo crear la sesión.
344	Entrega instrucciones HTML para solicitar una sesión de visualización.
346	Aplica el decorador `app.get('/', include_in_schema=False)`. Vincula el método y la ruta HTTP con la función que aparece a continuación.
347	Define `root`, que redirige la raíz de este servicio a su entrada de visualización.
348	Responde con redirección a /embed/presidencia.

Línea	Explicación
350	Define `schema`, que adapta OpenAPI a las rutas públicas y declara las tres credenciales API juntas.
351	Comprueba `app.openapi_schema is None` dentro de `schema`, que adapta OpenAPI a las rutas públicas y declara las tres credenciales API juntas.
352	En `schema`, genera o completa el documento OpenAPI que consulta Swagger.
353	En `schema`, lee el prefijo público de las comprobaciones de salud.
354	Comprueba `health_prefix` dentro de `schema`, que adapta OpenAPI a las rutas públicas y declara las tres credenciales API juntas.
355	Recorre `list(result['paths'])` asignando cada elemento a `path` dentro de `schema`.
356	Comprueba `path.startswith('/health/')` dentro de `schema`, que adapta OpenAPI a las rutas públicas y declara las tres credenciales API juntas.
357	En `schema`, asigna `result['paths'][health_prefix + path]` al resultado de la expresión mostrada.
358	Recorre `result['paths'].items()` asignando cada elemento a `(path, operations)` dentro de `schema`.
359	Comprueba `path.startswith(prefix)` dentro de `schema`, que adapta OpenAPI a las rutas públicas y declara las tres credenciales API juntas.
360	Recorre `operations.values()` asignando cada elemento a `operation` dentro de `schema`.
361	Declara en OpenAPI que Bearer, UsuarioAPI y ClaveAPI se exigen juntos para esta operación.
362	En `schema`, asigna `app.openapi_schema` al resultado de la expresión mostrada.
363	Devuelve el documento OpenAPI ya adaptado y almacenado en caché.
364	En `create_app`, asigna `app.openapi` al resultado de la expresión mostrada.
366	Devuelve el resultado de esta operación de create_app.
369	Define `page`, que envuelve el contenido en una página HTML que ocupa todo el espacio disponible.
370	Devuelve el documento HTML completo con metadatos, estilos y contenido del iframe.

12.3. Archivo auth.py: código y explicación de cada línea

La consulta SQL utiliza parámetros, no concatena el usuario recibido. La comprobación de vigencia y módulos ocurre en la identidad; el password se verifica en Python sin entregar el hash al consumidor.

```
"""Autorización de integración: cliente API, usuario SIVAS y sesiones de iframe."""
import hashlib
import hmac
import os
import secrets
import time
from dataclasses import dataclass
from threading import Lock

from argon2 import PasswordHasher
from argon2.exceptions import VerificationError, InvalidHashError
import psycopg
from nacl import pwhash
from nacl.exceptions import InvalidkeyError
from psycopg.rows import dict_row


class AccessDenied(Exception):
    pass


class IdentityUnavailable(Exception):
    pass


@dataclass(frozen=True)
class Identity:
    username: str
    user_id: int
    fingerprint: str


```

```
SQL_IDENTITY = """
SELECT u.id_usuario, u.nombre_usuario, u.password_hash
FROM credenciales.usuarios u
WHERE UPPER(u.nombre_usuario) = %s
  AND u.activo IS TRUE AND u.superusuario IS NOT TRUE
  AND u.fecha_inicio_vigencia <= CURRENT_DATE
  AND (u.vigente_indefinido IS TRUE OR u.fecha_fin_vigencia IS NULL
       OR u.fecha_fin_vigencia >= CURRENT_DATE)
  AND EXISTS (
    SELECT 1 FROM credenciales.usuario_modulo um
    JOIN credenciales.modulos m ON m.id_modulo=um.id_modulo AND m.activo IS TRUE
    JOIN credenciales.roles r ON r.id_rol=um.id_rol AND r.activo IS TRUE
    WHERE um.id_usuario=u.id_usuario AND um.activo IS TRUE
      AND UPPER(m.codigo_modulo)='PRESIDENCIA' AND r.nombre_rol IN ('USUARIO','ADMINISTRADOR')
  )
"""


class PostgresIdentity:
    """SELECT parametrizado, sin migraciones, permisos adicionales ni escrituras."""
    def __init__(self, username='PRESIDENCIA'):
        self.username = username.upper()

    def __call__(self, username, password=None):
        if username.upper() != self.username:
            raise AccessDenied()
        try:
            with psycopg.connect(dbname='productos_bm', host=os.environ['PGHOST'],
                 port=os.getenv('PGPORT','5432'), user=os.environ['PGUSER'],
                 password=os.environ['PGPASSWORD'], sslmode=os.getenv('PGSSLMODE','prefer'),
                 connect_timeout=5, options='-c default_transaction_read_only=on -c statement_timeout=5000',
                 row_factory=dict_row) as con:
                row = con.execute(SQL_IDENTITY, (self.username,)).fetchone()
        except (psycopg.Error, KeyError):
            raise IdentityUnavailable() from None
        if row is None:
            raise AccessDenied()
        if password is not None:
            try:
                verify_institutional_password(row['password_hash'],password)
            except (VerificationError, InvalidHashError, InvalidkeyError, ValueError):
                raise AccessDenied() from None
        return Identity(self.username, row['id_usuario'],
         ↪  hashlib.sha256(row['password_hash'].encode()).hexdigest())


def verify_institutional_password(stored,password):
    # R sodium::password_store genera scrypt ($7$); no confundir con Argon2.
    if stored.startswith('$7$'):
        return pwhash.verify(stored.encode(),password.encode())
    return PasswordHasher().verify(stored,password)


class Gateway:
    def __init__(self, client_user, client_hash, bridge_key, provider, username='PRESIDENCIA',
                 seconds=28800, clock=time.time):
        if not client_user or not client_hash.startswith('$argon2') or len(bridge_key) < 32:
            raise RuntimeError('Configure el cliente API, su hash y la clave interna del puente.')
        if not 60 <= seconds <= 86400:
            raise RuntimeError('La sesión de visualización debe durar entre 60 y 86400 segundos.')
        self.client_user, self.client_hash, self.bridge_key = client_user, client_hash, bridge_key
        self.provider, self.username, self.seconds, self.clock = provider, username.upper(), seconds, clock
        self.sessions = {}
        self.lock = Lock()
        self.failures = {}

    def verify_client(self, username, password):
        if not username or not password or not secrets.compare_digest(username.encode(),
         ↪  self.client_user.encode()):
            raise AccessDenied()
        try:
            PasswordHasher().verify(self.client_hash, password)
        except (VerificationError, InvalidHashError):
            raise AccessDenied() from None
```

```python

    def principal(self):
        return self.provider(self.username)

    def issue(self, username, password, indicator):
        if username.upper() != self.username:
            raise AccessDenied()
        identity = self.provider(username, password)
        now = self.clock()
        with self.lock:
            self.sessions = {key:value for key,value in self.sessions.items() if value['expires'] > now}
            if len(self.sessions) >= 256:
                raise IdentityUnavailable()
            sid = secrets.token_urlsafe(32)
            self.sessions[sid] = dict(identity=identity, indicator=indicator, expires=now+self.seconds,
                                      checked=now)
        return sid

    def resolve(self, sid):
        now = self.clock()
        with self.lock:
            session = self.sessions.get(sid)
            if not session or session['expires'] <= now:
                self.sessions.pop(sid, None)
                raise AccessDenied()
            # Revisar revocación y cambio de contraseña sin consultar por cada asset.
            if now - session['checked'] >= 10:
                identity = self.provider(session['identity'].username)
                if identity != session['identity']:
                    self.sessions.pop(sid, None)
                    raise AccessDenied()
                session['checked'] = now
            return dict(session)

    def revoke(self, sid):
        with self.lock:
            self.sessions.pop(sid, None)

    def headers(self, sid):
        session = self.resolve(sid)
        timestamp = str(int(self.clock()))
        username = session['identity'].username
        payload = '\n'.join((username, 'PRESIDENCIA', timestamp, sid))
        signature = hmac.new(self.bridge_key.encode(), payload.encode(), hashlib.sha256).hexdigest()
        return {'X-Presidencia-Usuario':username, 'X-Presidencia-Modulo':'PRESIDENCIA',
                'X-Presidencia-Tiempo':timestamp, 'X-Presidencia-Sesion':sid,
                'X-Presidencia-Firma':signature}

    def throttle(self, peer, *, fail=False):
        now = self.clock()
        with self.lock:
            self.failures = {key:value for key,value in self.failures.items() if value[0]+60 > now}
            start, count = self.failures.get(peer,(now,0))
            if count >= 10:
                raise AccessDenied()
            if fail:
                self.failures[peer] = (start,count+1)
```

Lectura línea por línea: deploy/presidencia/auth.py

Línea	Explicación
1	Texto descriptivo del componente; no ejecuta una operación de red ni abre un servicio.
2	Importa hashlib para SHA-256 para huellas y firmas.
3	Importa hmac para firma interna HMAC que acredita el acceso al dashboard.
4	Importa os para variables de entorno privadas del proceso.
5	Importa secrets para comparación de credenciales y generación aleatoria de SID.
6	Importa time para reloj del gateway y de sus sesiones.
7	Importa dataclasses para identidad inmutable con campos definidos.
8	Importa threading para exclusión mutua al consultar o modificar sesiones en memoria.

Línea	Explicación
10	Importa argon2 para verificación de hashes Argon2.
11	Importa argon2.exceptions para rechazos de hash o verificación sin propagar detalles sensibles.
12	Importa psycopg para lectura de identidad en PostgreSQL.
13	Importa nacl para verificación del formato scrypt creado por sodium en R.
14	Importa nacl.exceptions para detección de contraseña scrypt inválida.
15	Importa psycopg.rows para lectura de resultados SQL por nombre de columna.
18	Declara `AccessDenied`: excepción para denegar acceso.
19	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
22	Declara `IdentityUnavailable`: excepción por indisponibilidad de identidad.
23	No añade otra acción: conserva la excepción declarada o el rechazo principal del bloque anterior.
26	Aplica el decorador `dataclass(frozen=True)`. Define el comportamiento de la clase o función siguiente.
27	Declara `Identity`: identidad inmutable con usuario, ID y huella.
28	Define `username`: nombre canónico de la identidad institucional.
29	Define `user_id`: ID interno de la persona; no es su contraseña.
30	Define `fingerprint`: huella SHA-256 del hash para detectar cambios posteriores de credencial.
33	En `módulo`, declara la consulta institucional parametrizada de lectura; no crea ni modifica personas.
34	Selecciona ID, nombre de usuario y hash almacenado únicamente para verificar identidad.
35	Lee credenciales.usuarios, sin modificar esa tabla.
36	Compara el nombre mediante un parámetro %s, sin concatenarlo a SQL.
37	Exige usuario activo y excluye el uso de una cuenta superusuario para esta integración.
38	Exige que la fecha inicial de vigencia ya haya comenzado.
39	Admite vigencia indefinida o una fecha final nula;
40	en caso de fecha final definida, exige que aún no haya vencido.
41	Exige además la existencia de una asignación de módulo válida.
42	Lee las asignaciones de credenciales.usuario_modulo.
43	Exige que el módulo relacionado permanezca activo.
44	Exige que el rol relacionado permanezca activo.
45	Relaciona la asignación con esta persona y exige que esté activa.
46	Restringe el permiso al módulo PRESIDENCIA y a los roles USUARIO o ADMINISTRADOR.
47	Cierra la comprobación EXISTS de autorización institucional.
48	Finaliza la cadena SQL; todavía no ejecuta ninguna consulta.
51	Declara `PostgresIdentity`: proveedor de identidad institucional de lectura.
52	Texto descriptivo del componente; no ejecuta una operación de red ni abre un servicio.
53	Define `__init__`, que prepara la instancia y sus valores de configuración.
54	Actualiza `self.username`: el usuario institucional en mayúsculas.
56	Define `__call__`, que consulta la cuenta institucional, comprueba vigencia y módulo y verifica su contraseña cuando se recibe.
57	Comprueba `username.upper() != self.username` antes de aceptar la identidad institucional o verificar su contraseña.
58	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
59	Agrupa operaciones de `__call__` cuyos errores se traducen a una respuesta controlada.
60	Abre una conexión PostgreSQL de solo lectura con límites de conexión/consulta y resultados por nombre.
61	Continuación de la instrucción de la línea 60: Abre una conexión PostgreSQL de solo lectura con límites de conexión/consulta y resultados por nombre.
62	Continuación de la instrucción de la línea 60: Abre una conexión PostgreSQL de solo lectura con límites de conexión/consulta y resultados por nombre.
63	Continuación de la instrucción de la línea 60: Abre una conexión PostgreSQL de solo lectura con límites de conexión/consulta y resultados por nombre.
64	Continuación de la instrucción de la línea 60: Abre una conexión PostgreSQL de solo lectura con límites de conexión/consulta y resultados por nombre.
65	En `__call__`, obtiene una sola cuenta que cumple actividad, vigencia y permiso de módulo.

Línea	Explicación
66	Captura `(psycopg.Error, KeyError)` en `__call__` para responder sin mostrar detalles internos ni credenciales.
67	Señala que no es posible comprobar identidad o emitir otra sesión; no concede acceso por omisión.
68	Comprueba `row is None` antes de aceptar la identidad institucional o verificar su contraseña.
69	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
70	Comprueba `password is not None` antes de aceptar la identidad institucional o verificar su contraseña.
71	Agrupa operaciones de `__call__` cuyos errores se traducen a una respuesta controlada.
72	Comprueba la contraseña institucional con el algoritmo que corresponde al hash almacenado.
73	Captura `(VerificationError, InvalidHashError, InvalidkeyError, ValueError)` en `__call__` para responder sin mostrar detalles internos ni credenciales.
74	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
75	Devuelve Identity con usuario, ID y huella de credencial; no devuelve el hash original al consumidor.
78	Define `verify_institutional_password`, que selecciona scrypt o Argon2 según el formato del hash institucional.
79	Comentario del código: R sodium::password_store genera scrypt (7); no confundir con Argon2.
80	Comprueba `stored.startswith('$7$')` dentro de `verify_institutional_password`, que selecciona scrypt o Argon2 según el formato del hash institucional.
81	Devuelve el resultado de verificar la contraseña frente al formato del hash detectado.
82	Devuelve el resultado de verificar la contraseña frente al formato del hash detectado.
85	Declara `Gateway`: control del cliente API, identidad, SID, firma y fallos.
86	Define `__init__`, que prepara la instancia y sus valores de configuración.
87	Completa los parámetros de `__init__`: seconds=28800, clock=time.time):.
88	Comprueba `not client_user or not client_hash.startswith('$argon2') or len(bridge_key) < 32` dentro de `__init__`, que prepara la instancia y sus valores de configuración.
89	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
90	Comprueba `not 60 <= seconds <= 86400` dentro de `__init__`, que prepara la instancia y sus valores de configuración.
91	Detiene el arranque con un mensaje de configuración; evita servir una API mal configurada.
92	En `__init__`, asigna `(self.client_user, self.client_hash, self.bridge_key)` al resultado de la expresión mostrada.
93	En `__init__`, asigna `(self.provider, self.username, self.seconds, self.clock)` al resultado de la expresión mostrada.
94	Actualiza `self.sessions`: el registro de SID y sesiones en memoria.
95	Actualiza `self.lock`: un bloqueo para proteger sesiones e intentos concurrentes.
96	Actualiza `self.failures`: el contador por origen usado al limitar intentos.
98	Define `verify_client`, que comprueba el usuario del cliente API y verifica su contraseña contra el hash Argon2.
99	Comprueba `not username or not password or (not secrets.compare_digest(username.encode(), self.client_user.encode()))` dentro de `verify_client`, que comprueba el usuario del cliente API y verifica su contraseña contra el hash Argon2.
100	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
101	Agrupa operaciones de `verify_client` cuyos errores se traducen a una respuesta controlada.
102	Verifica la contraseña contra el hash Argon2; no recupera una contraseña desde el hash.
103	Captura `(VerificationError, InvalidHashError)` en `verify_client` para responder sin mostrar detalles internos ni credenciales.
104	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
106	Define `principal`, que revalida la identidad institucional configurada sin pedir su contraseña nuevamente.

Línea	Explicación
107	Solicita al proveedor comprobar la identidad configurada sin una contraseña nueva.
109	Define `issue`, que crea una sesión temporal después de comprobar la contraseña institucional.
110	Comprueba `username.upper() != self.username` dentro de `issue`, que crea una sesión temporal después de comprobar la contraseña institucional.
111	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
112	En `issue`, consulta la identidad institucional actual y su huella de credencial.
113	En `issue`, obtiene el instante actual del reloj configurado para vigencia, revalidación o intentos.
114	Adquiere el bloqueo de memoria durante la comprobación o modificación; se libera al salir del bloque.
115	Actualiza `self.sessions`: el registro de SID y sesiones en memoria.
116	Comprueba `len(self.sessions) >= 256` dentro de `issue`, que crea una sesión temporal después de comprobar la contraseña institucional.
117	Señala que no es posible comprobar identidad o emitir otra sesión; no concede acceso por omisión.
118	En `issue`, genera o recibe el identificador aleatorio temporal de la sesión; no es el Bearer permanente.
119	Actualiza `self.sessions[sid]`: la configuración de la instancia utilizada por issue.
120	Continuación de la instrucción de la línea 119: Actualiza `self.sessions[sid]`: la configuración de la instancia utilizada por issue.
121	Devuelve el SID que acaba de crearse; la contraseña no forma parte de ese identificador.
123	Define `resolve`, que consulta una sesión, comprueba caducidad y revalida revocación o cambio de contraseña.
124	En `resolve`, obtiene el instante actual del reloj configurado para vigencia, revalidación o intentos.
125	Adquiere el bloqueo de memoria durante la comprobación o modificación; se libera al salir del bloque.
126	En `resolve`, consulta la sesión actual del SID o recibe una copia de sus datos ya validados.
127	Comprueba `not session or session['expires'] <= now` para rechazar vencimiento, revisar revocación o detectar cambio de credencial.
128	Retira la sesión vencida/revocada del diccionario; si no existe, no añade un error secundario.
129	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
130	Comentario del código: Revisar revocación y cambio de contraseña sin consultar por cada asset.
131	Comprueba `now - session['checked'] >= 10` para rechazar vencimiento, revisar revocación o detectar cambio de credencial.
132	En `resolve`, consulta la identidad institucional actual y su huella de credencial.
133	Comprueba `identity != session['identity']` para rechazar vencimiento, revisar revocación o detectar cambio de credencial.
134	Retira la sesión vencida/revocada del diccionario; si no existe, no añade un error secundario.
135	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
136	Actualiza la marca de revalidación de esta sesión sin prolongar su vencimiento.
137	Devuelve una copia de la sesión después de las comprobaciones de vigencia y revocación.
139	Define `revoke`, que retira una sesión del registro en memoria.
140	Adquiere el bloqueo de memoria durante la comprobación o modificación; se libera al salir del bloque.
141	Retira la sesión vencida/revocada del diccionario; si no existe, no añade un error secundario.
143	Define `headers`, que firma el acceso interno al visualizador con HMAC-SHA256.
144	En `headers`, consulta la sesión actual del SID o recibe una copia de sus datos ya validados.
145	En `headers`, produce la marca temporal que forma parte de la firma interna.
146	En `headers`, recupera el usuario canónico de la sesión validada para firmar el acceso.
147	En `headers`, une usuario, módulo, marca temporal y SID en el mensaje a firmar.
148	En `headers`, calcula HMAC-SHA256 con la clave interna del puente sobre ese mensaje.
149	Devuelve los cinco encabezados internos que R validará antes de renderizar el dashboard.
150	Continuación de la instrucción de la línea 149: Devuelve los cinco encabezados internos que R validará antes de renderizar el dashboard.

Línea	Explicación
151	Continuación de la instrucción de la línea 149: Devuelve los cinco encabezados internos que R validará antes de renderizar el dashboard.
153	Define `throttle`, que limita fallos de acceso por origen dentro de una ventana de sesenta segundos.
154	En `throttle`, obtiene el instante actual del reloj configurado para vigencia, revalidación o intentos.
155	Adquiere el bloqueo de memoria durante la comprobación o modificación; se libera al salir del bloque.
156	Actualiza `self.failures`: el contador por origen usado al limitar intentos.
157	En `throttle`, asigna `(start, count)` al resultado de la expresión mostrada.
158	Comprueba `count >= 10` dentro de `throttle`, que limita fallos de acceso por origen dentro de una ventana de sesenta segundos.
159	Interrumpe la operación por acceso denegado; la ruta superior lo convierte en un rechazo de autenticación.
160	Comprueba `fail` dentro de `throttle`, que limita fallos de acceso por origen dentro de una ventana de sesenta segundos.
161	Actualiza `self.failures[peer]`: la configuración de la instancia utilizada por throttle.

12.4. servidor.py: lectura línea por línea

El archivo completo se imprimió en el capítulo de construcción. Esta guía permite reconocer cada instrucción y su relación con el arranque.

Línea	Explicación
1	Descripción del propósito del archivo; no inicia un proceso.
2	Importa el sistema de registro de Python.
3	Importa expresiones regulares para reconocer la capacidad temporal.
4	Separación visual sin instrucciones ejecutables.
5	Importa Uvicorn, el servidor ASGI que ejecutará FastAPI.
6	Separación visual sin instrucciones ejecutables.
7	Separación visual sin instrucciones ejecutables.
8	Define un filtro reutilizable para registros.
9	Recibe cada registro antes de que se formatee.
10	Busca una ruta de sesión con 43 caracteres; la expresión exige que termine antes de otra barra.
11	Sustituye esa parte por una etiqueta neutra; usa el mensaje ya formateado.
12	Vacía los argumentos originales para que no se intente formatear dos veces el mensaje.
13	Permite que el registro continúe después de depurarlo.
14	Separación visual sin instrucciones ejecutables.
15	Separación visual sin instrucciones ejecutables.
16	Solo arranca el servidor cuando se ejecuta este archivo como programa principal.
17	Toma la configuración estándar de registro de Uvicorn.
18	Registra la clase que ocultará identificadores temporales.
19	Recorre los manejadores de salida de logs.
20	Asocia el filtro a cada manejador.
21	Ejecuta la fábrica `create_app`; `factory=True` indica que debe llamarla para obtener la aplicación.
22	Escucha solo en loopback, puerto 8000, con un trabajador. Las sesiones de memoria no se comparten entre varios procesos.
23	Desactiva el registro habitual de accesos y usa la configuración depurada.
24	Acepta las cabeceras de proxy solo desde loopback, donde está el Nginx local.

12.5. Bootstrap y actualización del corte

```
# Carga exclusivamente Presidencia; no ejecuta global.R ni otros módulos de SIVAS.
Sys.setlocale('LC_CTYPE', if (.Platform$OS.type == 'windows') 'Spanish_Ecuador.utf8' else 'C.UTF-8')
```

```r
.libPaths(c(Sys.getenv('R_LIBS_USER'),.libPaths()))
if(file.exists('deploy/presidencia/.env'))readRenviron('deploy/presidencia/.env')
library(shiny)
library(dplyr)
library(highcharter)
library(glue)
library(gt)
library(shinyjs)
`%||%` <- function(x,y) if(is.null(x)||!length(x)) y else x
source('modulos/indicadores_presidencia/global.R',encoding='UTF-8')

con_pg_fun <- local({
 conexiones <- new.env(parent=emptyenv())
 function(dbname) {
  stopifnot(dbname %in% c('kpis','productos_bm'))
  if(exists(dbname,conexiones,inherits=FALSE)) {
   con<-conexiones[[dbname]]
   vivo<-tryCatch(DBI::dbIsValid(con)&&nrow(DBI::dbGetQuery(con,'SELECT 1'))==1L,error=function(e)FALSE)
   if(vivo)return(con)
   try(DBI::dbDisconnect(con),silent=TRUE)
  }
  cfg_path<-Sys.getenv('SIVAS_DEV_CONFIG','config.yml')
  cfg<-if(file.exists(cfg_path))config::get(file=cfg_path,value='postgresql_dneaisns')else list()
  parametro<-function(nombre,valor='') {v<-Sys.getenv(nombre);if(nzchar(v))v else valor}
  con<-DBI::dbConnect(RPostgres::Postgres(),dbname=dbname,
   host=parametro('PGHOST',cfg$host %||% '10.64.100.191'),
   port=as.integer(parametro('PGPORT',as.character(cfg$port %||% 5432))),
   user=parametro('PGUSER',cfg$user %||% ''),password=parametro('PGPASSWORD',cfg$password %||% ''),
   sslmode=parametro('PGSSLMODE','prefer'),connect_timeout=10,
   options='-c default_transaction_read_only=on -c statement_timeout=15000')
  conexiones[[dbname]]<-con
  con
 }
})

presidencia_exportar <- function(destino) {
 # Reutiliza las mismas consultas, normalización, metas y regla de publicación.
 reales<-presidencia_consultar_postgres(con_pg_fun('kpis'),con_pg_fun('productos_bm'))
 stopifnot(nrow(reales)>0)
 serie<-bind_rows(presidencia_crear_planificado(),reales)
 ids<-c('primer-nivel','salud-mental','paquete-ninos','paquete-gestantes','medicamentos')
 salida<-lapply(seq_along(ids),function(i) {
  meta<-presidencia_indicadores_base[i,]
  detalle<-presidencia_detalle_con_meta(filter(serie,indicador==meta$indicador))
  campos<-c('fecha','mes','valor','meta_mensual','numerador','numerador_puntual','denominador','fecha_corte','fe
  ↪  cha_actualizacion')
  list(id=ids[i],titulo=meta$indicador,objetivo=meta$objetivo,unidad='Porcentaje',
   linea_base=meta$linea_base,meta_2026=meta$meta_2026,
   datos=as.data.frame(detalle[,campos,drop=FALSE]))
 })
 dir.create(dirname(destino),recursive=TRUE,showWarnings=FALSE)
 temporal<-paste0(destino,'.tmp')
 jsonlite::write_json(list(modulo='PRESIDENCIA',origen='PostgreSQL; consultas institucionales de solo lectura',
  generado_en=format(Sys.time(),'%Y-%m-%dT%H:%M:%SZ',tz='UTC'),indicadores=salida),temporal,
  auto_unbox=TRUE,na='null',null='null',POSIXt='ISO8601',Date='ISO8601',digits=NA,pretty=TRUE)
 # Linux publica atómicamente para que el lector no observe un JSON incompleto.
 if(.Platform$OS.type=='windows') {
  if(!file.copy(temporal,destino,overwrite=TRUE))stop('No se pudo publicar el corte agregado')
  unlink(temporal)
 } else if(!file.rename(temporal,destino))stop('No se pudo publicar el corte agregado')
 cat('Corte agregado de Presidencia actualizado:',length(salida),'indicadores; sin datos nominales.\n')
}
```

12.6. Adaptador Shiny completo

Este archivo carga los componentes del módulo, exige identidad firmada y expone únicamente el adaptador de integración. La configuración del ambiente y de sus conexiones se recibe desde el proceso que lo ejecuta.

```r
Sys.setlocale('LC_CTYPE',if(.Platform$OS.type=='windows')'Spanish_Ecuador.utf8'else'C.UTF-8')
source('deploy/presidencia/bootstrap.R',encoding='UTF-8')
source('deploy/presidencia/bridge.R',encoding='UTF-8')
source('helpers/msp_ui_helpers.R',encoding='UTF-8')
source('modulos/indicadores_presidencia/indicadores_presidencia_assets.R',encoding='UTF-8')
source('modulos/indicadores_presidencia/indicadores_presidencia_ui.R',encoding='UTF-8')
source('modulos/indicadores_presidencia/indicadores_presidencia_server.R',encoding='UTF-8')

# Solo las funciones necesarias del autenticador institucional. Sin preparar
# esquemas, migrar roles ni escribir auditoría en la base compartida.
auth_defs<-c('roles_usuarios_schema_existe','roles_usuarios_verificar_password',
 'roles_usuarios_usuario_login_valido','roles_usuarios_codigo_modulo_valido','roles_usuarios_autenticar_modulo')
for(expr in parse('modulos/roles_usuarios/roles_usuarios_global.R',encoding='UTF-8')) {
 if(is.call(expr)&&identical(expr[[1]],as.name('<-'))&&is.symbol(expr[[2]])&&as.character(expr[[2]])%in%auth_de
  ↪ fs)eval(expr)
}
roles_usuarios_asegurar_rol_por_modulo<-function(...)invisible(NULL)
roles_usuarios_registrar_login<-function(...)message('Presidencia DEV: autenticación institucional validada; sin
 ↪ escritura en PostgreSQL.')
central_auth<-roles_usuarios_autenticar_modulo
roles_usuarios_autenticar_modulo<-function(con,nombre_usuario,password,codigo_modulo) {
 for(intento in 1:2) {
  resultado<-tryCatch(central_auth(con_pg_fun('productos_bm'),nombre_usuario,password,codigo_modulo),error=funct
 ↪ ion(e)NULL)
  if(!is.null(resultado))return(resultado)
 }
 list(ok=FALSE,usuario=NULL,mensaje='No fue posible validar el acceso institucional. Revise PostgreSQL o la VPN y
  ↪ vuelva a intentar.')
}
productos_bm<-NULL
addResourcePath('presidencia_assets',normalizePath('www',winslash='/'))
# Limitar recursos a assets institucionales; fichas son las cuatro del módulo.
original<-indicadores_presidencia_ui('indicadores_presidencia')
sin_login<-htmltools::tagQuery(original)$find('#indicadores_presidencia-login_panel')$remove()$allTags()
ui<-function(request) {
 if(!isTRUE(presidencia_validar_puente(request)$ok))return(fluidPage(
  tags$p('Acceda mediante la integración autorizada de Presidencia.')))
 fluidPage(shinyjs::useShinyjs(),tags$head(includeCSS('www/styles.css')),
  tags$script(HTML("Shiny.addCustomMessageHandler('presidencia-cerrar',function(message){var p=location.pathname
 ↪ .split('/shiny/')[0];fetch(p+'/cerrar',{method:'POST'}).finally(function(){location.reload()})});")),
  sin_login)
}
# La UI original registra fichas/. Sustituirla por solo las cuatro fichas autorizadas.
fichas_dev<-'deploy/presidencia/runtime/fichas'
dir.create(fichas_dev,recursive=TRUE,showWarnings=FALSE)
for(nombre in c('atenciones_establecimientos_primer_nivel.pdf','atenciones_salud_mental.pdf',
 'ninios_ninias_paquete_priorizado_edad.pdf','gestantes_paquete_priorizado_edad.pdf')) {
 stopifnot(file.copy(file.path('fichas',nombre),file.path(fichas_dev,nombre),overwrite=TRUE))
}
removeResourcePath('fichas')
addResourcePath('fichas',normalizePath(fichas_dev,winslash='/'))
server<-function(input,output,session) {
 ids<-c('primer-nivel','salud-mental','paquete-ninos','paquete-gestantes','medicamentos')
 inicial<-reactive({
  slug<-parseQueryString(session$clientData$url_search)$indicador %||% 'primer-nivel'
  match(slug,ids) %||% 1L
 })
 indicadores_presidencia_server('indicadores_presidencia',indicador_inicial=inicial,
  autenticacion_externa=function()presidencia_validar_puente(session$request))
 observeEvent(input[['indicadores_presidencia-btn_logout']],{
  session$sendCustomMessage('presidencia-cerrar',list())
 },ignoreInit=TRUE)
}
app<-shinyApp(ui,server)
# La conexión solo permite SELECT, incluso durante el login.
shiny::runApp(app,host=Sys.getenv('PRESIDENCIA_SHINY_HOST','127.0.0.1'),
 port=as.integer(Sys.getenv('PRESIDENCIA_SHINY_PORT','8001')),launch.browser=FALSE)
```

12.7. CI existente

El workflow construye y prueba artefactos; no contiene las claves del servidor ni un CD automático al servidor 10.64.100.197.

```yaml
name: Integración Presidencia - pruebas y paquete de desarrollo
on:
  pull_request:
    paths: ['deploy/presidencia/**', 'modulos/indicadores_presidencia/**',
 ↪  '.github/workflows/presidencia-integracion.yml']
  push:
    paths: ['deploy/presidencia/**', 'modulos/indicadores_presidencia/**',
 ↪  '.github/workflows/presidencia-integracion.yml']
  workflow_dispatch:
    inputs:
      construir_shiny:
        description: Construir también la imagen R/Shiny (descarga grande)
        type: boolean
        default: false
permissions:
  contents: read
jobs:
  validar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.14'
      - run: pip install -r deploy/presidencia/requirements.txt
      - run: python -m pytest deploy/presidencia/tests -q
      - run: docker build -f deploy/presidencia/Dockerfile.api -t sivas-presidencia-api:ci .
      - name: Empaquetar imagen de desarrollo
        if: github.event_name == 'workflow_dispatch'
        run: docker save sivas-presidencia-api:ci | gzip > presidencia-api.tar.gz
      - uses: actions/upload-artifact@v4
        if: github.event_name == 'workflow_dispatch'
        with:
          name: presidencia-api-desarrollo
          path: presidencia-api.tar.gz
  shiny:
    if: github.event_name == 'workflow_dispatch' && inputs.construir_shiny
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -f deploy/presidencia/Dockerfile.shiny -t sivas-presidencia-shiny:ci .
      - run: docker save sivas-presidencia-shiny:ci | gzip > presidencia-shiny.tar.gz
      - uses: actions/upload-artifact@v4
        with:
          name: presidencia-shiny-desarrollo
          path: presidencia-shiny.tar.gz
```

12.8. Actualización de las notas por línea

Las notas de Python se regeneran desde el entorno del proyecto y se revisan antes de renderizar:

```
& 'C:/ProgramData/anaconda3/envs/msp_01/python.exe' .\docs\manuales\api_presidencia\generar_explicaciones.py
```

El generador analiza únicamente los archivos públicos `api.py`, `auth.py` y `linux/servidor.py`; no lee credenciales ni ejecuta servicios. `incluir_codigo.lua` comprueba que el texto de cada línea coincida con su explicación. Los diagramas se regeneran con `diagramas/generar_diagramas.py`; sus PNG transparentes ya están incluidos para que el PDF se pueda reconstruir sin volver a capturarlos.

Referencias

Conda. (s. f.). *Managing environments*. Recuperado 25 de octubre de 2026, de https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html

FastAPI. (s. f.-a). *First Steps*. Recuperado 22 de octubre de 2026, de https://fastapi.tiangolo.com/tutorial/first-steps/

FastAPI. (s. f.-b). *Run a Server Manually*. Recuperado 22 de octubre de 2026, de https://fastapi.tiangolo.com/deployment/manually/

NGINX. (s. f.). *WebSocket proxying*. Recuperado 22 de octubre de 2026, de https://nginx.org/en/docs/http/websocket.html

Posit. (s. f.-a). *PDF Basics*. Recuperado 22 de octubre de 2026, de https://quarto.org/docs/output-formats/pdf-basics.html

Posit. (s. f.-b). *Welcome to Shiny*. Recuperado 22 de octubre de 2026, de https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/

PostgreSQL Global Development Group. (s. f.). *SET TRANSACTION*. Recuperado 22 de octubre de 2026, de https://www.postgresql.org/docs/current/sql-set-transaction.html

Sistema de Visualización Analítica en Salud (SIVAS). (2026). *Sistema de Visualización Analítica en Salud (SIVAS): módulo de Indicadores de Presidencia*.

The systemd Project. (s. f.). *systemd.service*. Recuperado 22 de octubre de 2026, de https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html

W3C. (s. f.). *Content Security Policy Level 3*. Recuperado 22 de octubre de 2026, de https://www.w3.org/TR/CSP3/